K L H UNIVERSITY

Department of Computer Science and Engineering

A PROJECT REPORT ON

KISAAN KRUSHI AI (FARMASSIST)

An Algorithm-Driven Agricultural Knowledge Search and Decision Support System

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Submitted by

Nimma Lokesh Reddy : 2520030366

Ramagiri Rishik Rao : 2520030333

Manne Yashwanth Manoj : 2520030369

Under the Guidance of

Dr. V. Sireesha

Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

2026-27

Department of Computer Science and Engineering

DECLARATION

The project report entitled "KISAAN KRUSHI AI (FARMASSIST): An Algorithm-Driven Agricultural Knowledge Search and Decision Support System" is a record of the bonafide work carried out by the following students of the Department of Computer Science and Engineering, K L H UNIVERSITY, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology during the academic year 2026-27.

Nimma Lokesh Reddy - 2520030366

Ramagiri Rishik Rao - 2520030333

Manne Yashwanth Manoj - 2520030369

We hereby declare that the results embodied in this report have not been copied from any other department, university, or institute, and that all sources of information used have been acknowledged in the reference section. We understand that any violation of academic integrity may lead to disciplinary action as per the rules of the university.

Place: Guntur

Date: ___ / ___ / 2026-27

Signature of the Students

Nimma Lokesh Reddy

Ramagiri Rishik Rao

Manne Yashwanth Manoj

CERTIFICATE

This is to certify that the project report entitled "KISAAN KRUSHI AI (FARMASSIST): An Algorithm-Driven Agricultural Knowledge Search and Decision Support System" is a bonafide record of the mini project work done and submitted by

Nimma Lokesh Reddy - 2520030366

Ramagiri Rishik Rao - 2520030333

Manne Yashwanth Manoj - 2520030369

in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of Computer Science Engineering, K L H UNIVERSITY, during the academic year 2026-27.

The work presented in this report has been carried out by the candidates under my supervision and guidance. The report has been examined and is recommended for acceptance.

Place: Guntur

Date: ___ / ___ / 2026-27

Signature of the Guide	**Signature of the HOD**	**Signature of the External Examiner**
Dr. V. Sireesha Assistant Professor	Dr. P. Venkateswara Rao Associate Professor & Head of the Department	[Name / Designation]

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. V. Sireesha, Professor, Department of Computer Science and Engineering, for the continuous guidance, valuable suggestions, and constant encouragement provided throughout the development of this project. The regular review discussions helped us understand how classical data structures and algorithms can be applied to a practical agricultural problem, and how the selection of an algorithm should be driven by the shape of the problem rather than by convenience.

We are thankful to Dr. P. Venkateswara Rao, Associate Professor & Head of the Department, for providing the academic environment, laboratory facilities, and resources required to complete this work. We also thank all the faculty members of the Department of Computer Science and Engineering, K L H UNIVERSITY, who taught the Data Structures and Algorithms sequence of courses and whose lectures directly shaped the technical content of this report.

We gratefully acknowledge the support of our classmates, friends, and family members, whose encouragement helped us complete the project on time. Finally, we acknowledge the use of the JLine library for terminal keyboard input handling and the OpenWeatherMap service for the optional live weather feature. Both are used strictly as supporting infrastructure and not for any algorithmic work, which is implemented entirely from first principles within the project source code.

The Project Team

ABSTRACT

Kisaan Krushi AI, also referred to as FarmAssist, is a console-based agricultural knowledge search and decision support system developed using core Java and classical data structures and algorithms. A farmer can put a question about crops, diseases, pests, symptoms, fertilizers, crop planning, resource allocation, or weather in ordinary natural language, and the application answers it from a local agricultural knowledge base. Unlike a basic keyword lookup tool, the system is designed to cope with spelling mistakes, joined words, regional crop names, several symptoms in one sentence, and requests that demand an optimization decision rather than a simple retrieval.

The system uses thirteen algorithms implemented from first principles. Aho-Corasick detects every agricultural entity in the sentence in a single pass, KMP performs exact symptom matching inside disease records, Rabin-Karp scans and scores articles using rolling hashes, Edit Distance repairs spelling, and a Trie supplies live autocomplete suggestions from the knowledge base. Dynamic programming is used at four different levels: character level for spelling repair, word level for segmentation, budget level through the 0/1 Knapsack problem for fertilizer selection, and subset level through Bitmask DP for crop to fertilizer assignment. Bipartite Matching, Maximum Flow, and Minimum Cut support one-bag-per-crop and limited-stock allocation tasks. Greedy Set Cover and Greedy Knapsack address NP-hard decision problems with documented approximation guarantees. Randomized QuickSort ranks results, while parallel map, reduce, and prefix-sum primitives score the crop table using work and span analysis.

The application uses a local pipe-separated agricultural knowledge base containing crops, diseases, pests, fertilizers, articles, symptoms, synonyms, and small-talk responses. An optional weather module retrieves current weather conditions and a five-day outlook using the OpenWeatherMap API and converts the readings into field advice. The project demonstrates that classical data structures and algorithms, combined carefully, can produce a useful, explainable, and mostly offline agricultural assistant without depending on any machine-learning model.

Keywords: Agriculture, FarmAssist, string matching, dynamic programming, graph algorithms, network flow, approximation algorithms, natural-language query processing, Java, data structures and algorithms.

TABLE OF CONTENTS

The table of contents below is generated as a Word field. After all figures have been inserted, right-click the field and select "Update Field" to refresh the chapter, section, and page numbers.

LIST OF FIGURES

The figure numbers and page numbers listed below are updated after the diagrams and screenshots are inserted in place of the marked boxes.

CHAPTER 1

Fig. 1.1 Overall Concept of FarmAssist

CHAPTER 2

Fig. 2.1 Use Case Diagram of FarmAssist

CHAPTER 3

Fig. 3.1 High-Level Layered Architecture of FarmAssist

Fig. 3.2 Level 0 Data Flow Diagram of FarmAssist

Fig. 3.3 Level 1 Data Flow Diagram of the Query Pipeline

Fig. 3.4 Query Processing Pipeline of FarmAssist

Fig. 3.5 Entity Relationship Diagram of the Knowledge Base

Fig. 3.6 Class Diagram of the Principal FarmAssist Classes

Fig. 3.7 Sequence Diagram for Processing a Diagnosis Question

CHAPTER 4

Fig. 4.1 KMP Prefix Table Construction and Pattern Matching

Fig. 4.2 Aho-Corasick Trie, Failure Links and Output Links

Fig. 4.3 Edit Distance Matrix for Converting tomatoe into tomato

Fig. 4.4 Trie Structure for Live Autocomplete

Fig. 4.5 Knapsack Dynamic Programming Table for a Fertilizer Budget

Fig. 4.6 Bitmask Dynamic Programming State Transition Diagram

Fig. 4.7 Maximum Flow Network with Minimum Cut for Stock Supply

Fig. 4.8 Greedy Set Cover Selection over Crops and Fertilizers

Fig. 4.9 Parallel Map-Reduce Tree for Scoring the Crop Table

CHAPTER 5

Fig. 5.1 Disease Diagnosis Using Aho-Corasick, KMP and Randomized QuickSort

Fig. 5.2 Budget-Based Fertilizer Recommendation

Fig. 5.3 Crop-to-Fertilizer Assignment Using Matching and Bitmask DP

Fig. 5.4 Maximum-Flow Stock Supply and the Bottleneck Reported by the Minimum Cut

Fig. 5.5 Farm Shopping List Produced by Greedy Set Cover

Fig. 5.6 Live Autocomplete Suggestions Under the Prompt

Fig. 5.7 Climate-Based Crop Planner Using Parallel Map and Reduce

Fig. 5.8 Weather Report and Derived Field Advice

Fig. 5.9 Algorithm Demonstration Output

LIST OF TABLES

The table numbers and page numbers listed below are updated after the final pagination of the report.

Table 2.1 Summary of the string algorithms and their use in FarmAssist

Table 2.2 Limitations of the existing approach and the response in FarmAssist

Table 2.3 Functional requirements

Table 2.4 Non-functional requirements

Table 2.5 Requirement traceability matrix

Table 3.1 Algorithm layer contents and the workload carried by each class

Table 3.2 Knowledge base files and their contents

Table 3.3 Package structure and responsibilities

Table 3.4 Question shape, detected intent, and the algorithm family used

Table 3.5 Data structures used in the design and the reason for each choice

Table 3.6 Trace facts contributed by each algorithm

Table 3.7 States and transitions of the interactive input

Table 4.1 Development and execution environment

Table 4.2 Error conditions handled explicitly by the application

Table 5.1 Independent validation performed on each algorithm

Table 5.2 Functional test cases and observed results

Table 5.3 Complexity of each algorithm with the bound that applies in FarmAssist

Table 5.4 Feature, algorithmic result, and value to the user

CHAPTER 1: INTRODUCTION

1.1 Overview

Agriculture continues to be the largest source of livelihood in India, and the quality of a farming season depends heavily on the timeliness and accuracy of the decisions taken by the farmer. Decisions about which crop to sow, how much water and fertilizer to apply, how to recognise a disease from the symptoms visible on a leaf, and how to protect the crop from a pest outbreak must all be taken within short windows of time. A delay of a few days in recognising a disease, or a mistake in the quantity of fertilizer applied, can reduce the yield of an entire field and with it the income of a family for a whole season.

At the same time, the channels through which a farmer can obtain advice are limited. A village may not have a resident agricultural extension officer, an internet connection may be unreliable or costly, and general search engines are not designed to answer agronomic questions about a local knowledge base that they do not hold. Even when connectivity is available, a farmer types a question the way it is spoken, not the way it is stored in a database. Spelling mistakes, joined words, colloquial crop names such as paddy for rice, bhindi for okra and dhan for paddy, and several symptoms described in one sentence are all normal features of a real question and all of them break a simple keyword lookup.

Kisaan Krushi AI, referred to throughout this report as FarmAssist, is a console-based agricultural knowledge search and decision support system that addresses this gap. It accepts a farmer question written in ordinary language and answers it from a local, offline knowledge base of crop profiles, disease records, pest records, fertilizer records, and field articles. The response is produced not by a machine-learning model but by thirteen classical data structures and algorithms that are implemented from first principles in core Java and that do visible work at every stage of the query pipeline.

The overall concept of the system is shown in Fig. 1.1. The farmer enters a question; the system repairs the words, normalises the local names, detects every agricultural entity in a single pass, decides what the farmer is actually asking, executes the algorithm that fits that question, and prints the answer on a consistent

terminal layout. An optional trace mode prints the name of each algorithm as it runs together with its intermediate result, so that the computational method behind an answer can be seen rather than merely trusted.

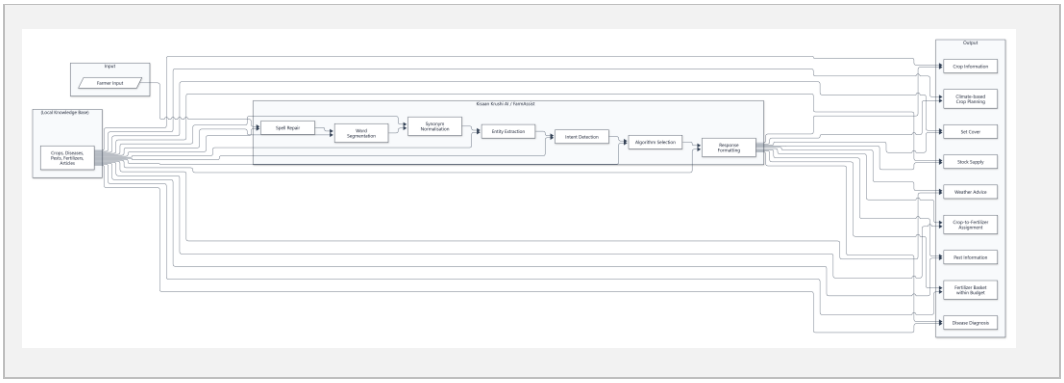


Fig. 1.1: Overall Concept of FarmAssist The concept diagram introduces the reader to the complete scope of the system before the individual algorithms are discussed.

1.2 Background and Motivation

The motivation for the project comes from a simple observation: a farmer question is not a database query. It is a sentence, usually short, often ungrammatical, and almost always containing more than one piece of information. In the sentence "my tomatoe has yelow leaves and brown spots", the word tomatoe is misspelled, the word yelow is misspelled, and two distinct symptoms are present. A conventional information system would either return nothing or return documents that merely happen to share a word. The question also implicitly asks a diagnostic question that requires the symptoms to be matched against stored disease descriptions and the candidate diseases to be ranked by how well they fit.

A second observation is that many of the farmer questions that matter most are not retrieval questions at all; they are optimisation questions. "Suggest a fertilizer for tomato under three thousand rupees" asks for the best basket of items whose total price does not exceed a budget, which is the 0/1 Knapsack problem. "Match fertilizers for rice, cotton and banana" asks for one distinct suitable fertilizer per crop, which is a bipartite matching problem and, when the benefit must be maximised, a dynamic program over subsets. "Supply three fields of rice and two fields of cotton from the shop stock" is a maximum flow problem whose residual graph also names the bottleneck through a minimum cut. "Fewest fertilizers for rice, wheat, cotton and banana" is the NP-hard set cover problem and needs an

approximation algorithm with a proven bound. Treating all of these as variations of keyword search would lose the decision support that the farmer actually needs.

The third motivation is educational. The Data Structures and Algorithms sequence of courses teaches string matching, dynamic programming, graph algorithms, network flow, approximation algorithms, randomised algorithms, and parallel algorithms as separate topics with abstract inputs. FarmAssist was designed to make each of those topics carry a real workload in a single application, so that the theory can be seen operating on genuine data. The project deliberately avoids the built-in Java searching and sorting methods for the algorithmic core, so that a reader who inspects the source code sees the algorithm rather than a library call. The following points summarise the motivation:

- Farmers need quick, understandable agricultural information and cannot always reach an expert or a reliable internet connection.
- Natural-language questions are rarely clean; they contain typos, joined words, regional crop names, and multiple entities in no fixed order.
- Several important agricultural decisions are optimisation problems rather than lookup problems and therefore need algorithms, not indexes.
- Classical algorithms produce results that are explainable and traceable, which matters for an advisory tool that must be trusted by its user.
- An offline-first local knowledge base removes the dependence on general web search and on connectivity.
- A single application that exercises string, dynamic-programming, graph, flow, approximation, randomised, and parallel algorithms makes the course outcomes directly demonstrable.

1.3 Problem Statement

Farmers require information about crops, diseases, pests, fertilizers, weather, and cultivation practices, and they frequently require decisions that depend on that information, such as which fertilizer to buy within a budget or how to distribute a limited stock of fertilizer across several fields. Agricultural information is however difficult to reach offline, and the ordinary search systems that are available cannot interpret a natural-language question containing spelling mistakes, regional synonyms, several symptoms, and multiple named entities. In

addition, the decision-making tasks that a farmer faces, namely budget-based fertilizer selection, crop-to-fertilizer assignment, limited-stock supply, and farm-wide product coverage, are combinatorial optimisation problems for which no simple index or lookup can produce an answer.

Therefore, a system is required that can accept a farmer question written in natural language, repair the language problems in it without asking the farmer to retype, identify every agricultural entity mentioned, determine what kind of answer the farmer is asking for, select and execute the classical algorithm that is appropriate to that kind of question, and present a clear and explainable result that is computed entirely from a local knowledge base. The system must remain usable without a continuous internet connection so that it is useful in a village setting, and it must expose the reasoning behind each answer instead of presenting a black-box recommendation.

1.4 Scope of the Project

The scope of FarmAssist covers a terminal-based conversational interface, a local agricultural knowledge base stored in plain text, an algorithmic query processing pipeline, crop, disease, pest, and fertilizer information retrieval, decision support for fertilizer purchase and allocation, climate-based crop planning, live autocomplete while typing, an optional trace of every algorithm executed, and an optional live weather module. The scope of the system is deliberately bounded in the following ways.

- **Interface:** the system provides a command-line, text-based chat interface designed for a terminal emulator. A graphical, web, or mobile front end is out of scope, although the engine is written so that it can be reused by such a front end in future work.
- **Language:** the system understands English sentences with English-script regional crop names. A synonym dictionary maps local names such as paddy, dhan, and bhindi to their canonical forms. Free-form Hindi or Telugu sentences, speech input, and translation are outside the current scope.
- **Knowledge base:** the agricultural content is a curated local data set stored as pipe-separated text files. The project does not scrape or synchronise live

agricultural databases, and the quality of an answer therefore depends on the completeness and accuracy of the local records.

- **Algorithms:** thirteen classical algorithms are implemented and used. The project does not implement every algorithm in the syllabus; only those that carry a genuine workload in the application are included.
- **Weather:** live weather is an optional module that requires an API key and an internet connection. The rest of the system is fully offline.
- **Advisory responsibility:** the system is an educational decision support prototype. It does not replace an agronomist, and it does not provide guaranteed medical, chemical, or financial advice.

The intended user of the system is a small or marginal farmer who has occasional access to a computer or a shared terminal facility, and the intended secondary user is a student or reviewer who wishes to examine how the algorithms behave on real data. Within this scope the system is complete and demonstrable end to end.

1.5 Objectives

The objectives of the project are stated below. Each objective is expressed as a capability that the finished system must provide, and each corresponds to at least one feature that can be demonstrated during a review.

1. To load agricultural knowledge, namely crops, diseases, pests, fertilizers, articles, symptoms, synonyms, and conversational replies, from a local plain-text knowledge base at start-up.
2. To accept a natural-language question typed by the user, including questions that contain spelling errors and joined words.
3. To correct common spelling errors using Edit Distance and to split joined words using word-segmentation dynamic programming before any interpretation takes place.
4. To normalise regional and alternate crop and disease names to their canonical forms using a synonym dictionary driven by a second automaton.
5. To detect crops, diseases, pests, symptoms, and fertilizers present in the question in a single pass over the sentence using Aho-Corasick.

6. To determine the intention of the question and route it to the appropriate feature using keyword rules, so that the correct algorithm family is selected for each question.
7. To search and rank agricultural records and articles using Rabin-Karp scoring, KMP exact matching, Suffix Array with LCP related-reading detection, and Randomized QuickSort ranking.
8. To diagnose possible diseases from the symptoms reported by the farmer using exact matching and to present the candidate diseases in ranked order with their treatment information.
9. To recommend the best basket of fertilizers for a crop within a stated budget using the 0/1 Knapsack dynamic program, and to display the greedy approximation beside the exact answer.
10. To assign a distinct suitable fertilizer to each named crop using bipartite matching, and to find the richest such assignment using Bitmask dynamic programming when the number of crops is small.
11. To model the supply of several fields from a limited stock of fertilizer bags using Maximum Flow and to explain the limiting resource using Minimum Cut.
12. To find a small set of fertilizer products that between them cover every crop on the farm using Greedy Set Cover, and to report the guarantee that accompanies the approximation.
13. To recommend crops suited to a given rainfall, temperature, or water profile by scoring the crop table with parallel map, reduce, and prefix-sum primitives and reporting the work and span of the computation.
14. To provide live autocomplete suggestions while the user types, using a Trie built over the knowledge base vocabulary.
15. To provide current weather conditions, a five-day outlook, and derived field advice when the optional weather module is configured.
16. To make every algorithm visible at run time through a trace mode and a demonstration command that runs each algorithm on small, hand-checkable inputs.
17. To package the whole application so that it compiles and runs with only a JDK, without any build tool or framework, on Windows, Linux, and macOS.

1.6 Contributions of the Project

The principal contributions of the project are listed below and are expanded in the later chapters.

- **Integration:** thirteen classical algorithms from six different algorithm families are integrated into one working application in which each algorithm carries a distinct workload.
- **Single query pipeline:** one pipeline performs repair, normalisation, extraction, routing, execution, and formatting, so that a new feature can be added by extending the routing rules rather than by rewriting the input handling.
- **Decision support beyond search:** the system answers budget, assignment, supply, coverage, and planning questions in addition to retrieval and diagnosis questions.
- **Explainability:** an algorithm trace names the algorithm that ran and reports its intermediate result, such as the number of augmenting paths, the number of trie states, or the approximation bound, so that every answer can be justified.
- **Extensible knowledge base:** the knowledge base is plain text with one record per line, so a new crop or disease can be added without changing any algorithm class.
- **Offline operation:** every feature except the optional weather module works without any network connection.
- **Honest treatment of hard problems:** the NP-hard decisions are solved exactly where the input is small enough and approximately where it is not, and the report always states which of the two was used together with its guarantee.

1.7 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 presents the system requirements: it summarises the related work that informed the design, describes the existing and proposed systems, and lists the functional, non-functional, hardware, and software requirements together with the use case view of the system. Chapter 3 presents the system design: the layered architecture, the data

flow diagrams, the query processing pipeline, the knowledge base design, the class and package design, and the sequence of interactions for a representative query. Chapter 4 presents the implementation with pseudocode for each of the thirteen algorithms, followed by the module-wise implementation, the front-end logic, the external API integration, and the installation and execution procedure. Chapter 5 presents the results: the testing strategy, the algorithm validation, the functional and boundary test cases, the results of each feature, and an analysis of the outcomes. Chapter 6 records the challenges faced during development, the solutions that were applied, and the limitations that remain. Chapter 7 concludes the report and sets out the future work, and is followed by the list of references.

CHAPTER 2: SYSTEM REQUIREMENTS

This chapter establishes what the system must do and why those requirements are the right ones. It begins with a survey of the published work that the project builds upon, covering agricultural information systems and each of the algorithm families that FarmAssist uses. It then contrasts the existing way in which a farmer obtains advice with the proposed system, and finally states the requirements themselves: functional, non-functional, hardware, and software, followed by the use case view of the completed application.

2.1 Literature Survey

The literature relevant to FarmAssist falls into six groups. The first group concerns agricultural information systems and advisory services. The remaining five concern the algorithm families that the project applies: string algorithms, dynamic programming, graph and network-flow algorithms, approximation algorithms, and randomised and parallel algorithms. The survey below is organised in the same order and closes with the research gap that the project addresses.

2.1.1 Agricultural Information Systems and Advisory Services

Agricultural information systems in common use provide crop information, disease descriptions and photographs, weather data, market prices, or general advisory content. They are typically organised as web portals, mobile applications, or fixed questionnaires, and they are effective when the user already knows which record to open. Their limitation, from the point of view of this project, is the interaction model: the user must navigate a fixed structure, and the burden of interpreting a messy question falls entirely on the user. Advisory services delivered by telephone or by a visiting officer avoid that problem but do not scale to the number of questions that arise daily, and they are difficult to reach at the moment the question actually occurs.

Research on agricultural decision support has also explored expert systems based on rule bases and, more recently, machine-learning classifiers trained on leaf images or on crop-disease data sets. Those approaches can be accurate, but they

require training data, they produce recommendations whose reasoning is difficult to inspect, and they are comparatively heavy to run on the modest hardware available in a village setting. FarmAssist takes a deliberately different position: the intelligence of the system lies in the correct choice and execution of classical algorithms over a curated local knowledge base, which is explainable at every step and needs no training.

2.1.2 Traditional String Searching

The naive or brute-force string matching method compares a pattern against the text at every possible alignment. Its worst-case cost is O(n x m) for a text of length n and a pattern of length m, and when k patterns must be searched the cost becomes O(k x n x m) because each pattern repeats the scan. For an application that must find many agricultural terms in a short sentence and many query terms in a large article collection, that repeated work is the dominant cost.

Knuth, Morris and Pratt presented the first linear-time exact matching algorithm in 1977. The algorithm preprocesses the pattern into a longest proper prefix that is also a suffix table, usually abbreviated as the LPS or failure table, and uses it to shift the pattern after a mismatch without ever moving the text pointer backwards. The cost becomes O(n + m) time and O(m) extra space. In FarmAssist, KMP is the exact matcher used to test whether a reported symptom occurs inside the description of a disease, and to locate a term so that a readable snippet can be cut around it.

Karp and Rabin presented the randomised rolling-hash method in 1987. The pattern and every window of the text of the same length are reduced to numbers by a polynomial hash, and the hash is rolled from one window to the next in constant time by removing the contribution of the outgoing character and adding the contribution of the incoming character. Because a hash match may be a collision rather than a real match, the candidate positions are verified character by character, which is why the method is described as a Monte Carlo idea made exact by verification. Its expected cost is O(n + m) and its extra space is constant. FarmAssist uses Rabin-Karp to scan every indexed document, count how often each query term occurs, and turn those counts into a relevance score.

Aho and Corasick presented the multi-pattern automaton in 1975. All patterns are inserted into a trie, failure links are added so that the automaton never restarts from the root after a mismatch, and output links are added so that every pattern ending at a position is reported. One pass over the text therefore finds all patterns at a cost of O(n + total length of the patterns + number of matches). FarmAssist uses Aho-Corasick twice: once to detect crops, diseases, pests, fertilizers, and symptoms in a single pass, and once to rewrite local names into their canonical forms.

Suffix structures complete the string group. A suffix array stores the starting positions of all suffixes of a string in sorted order, and the longest-common-prefix array, constructed in linear time by the Kasai method once the suffix array exists, records how much adjacent suffixes share. Together they allow the longest common substring of two texts to be found without pairwise comparison, and they are the basis of the related reading feature in FarmAssist. A trie is also used in its simplest form as a prefix tree for the live autocomplete suggestions that appear under the prompt while the user types.

Table 2.1: Summary of the string algorithms and their use in FarmAssist

Algorithm	Published work	Time	Space	Use in FarmAssist
Naive matching	Classical	O(n x m) per pattern	O(1)	Rejected: repeats work for every pattern
KMP	Knuth, Morris, Pratt (1977)	O(n + m)	O(m)	Exact symptom matching inside disease records; snippet location
Rabin-Karp	Karp, Rabin (1987)	O(n + m) expected	O(1)	Rolling-hash scan of all indexed documents to build relevance scores
Aho-Corasick	Aho, Corasick (1975)	O(n + total pattern length + matches)	O(total pattern length)	Single-pass detection of all entities; synonym rewriting
Trie	Classical	O(L) per prefix lookup	O(total word length)	Live autocomplete suggestions under the prompt
Suffix Array + LCP	Manber, Myers; Kasai et al.	O(n log^2 n) construction	O(n)	Related reading by longest common substring

2.1.3 Dynamic Programming in Decision Support

Dynamic programming applies when a problem can be divided into overlapping subproblems whose answers can be stored and reused. The classical reference is Bellman, and the standard modern treatment of the technique is given by Cormen, Leiserson, Rivest and Stein. FarmAssist applies dynamic programming at four distinct levels of granularity, which is one of the design ideas that distinguishes it from a typical course exercise.

- **Character level:** the Levenshtein edit distance between the typed word and a dictionary word is computed with a table of prefix distances and used to propose spelling corrections.
- **Word level:** word-segmentation dynamic programming decides how a joined string such as lateblight should be split into dictionary words, which edit distance alone cannot repair because no substitution explains a missing space.
- **Budget level:** the 0/1 Knapsack problem selects the combination of fertilizers that maximises agricultural benefit while respecting a rupee budget. The exact table-based dynamic program is pseudo-polynomial in the budget value, which is precisely the case in which it is practical.
- **Subset level:** Bitmask dynamic programming solves the crop to fertilizer assignment problem when the objective is to maximise total benefit rather than merely to find any feasible assignment. The state records which fields have already been served, encoded as bits of an integer.

The subset level is the most interesting of the four because it illustrates a general technique rather than a specific problem. Whenever the state of a computation is a set of already-processed items and the set is small, an integer mask can represent that set, and a table indexed by the mask can be filled by dynamic programming. The approach is used in the literature for the travelling salesman problem, for assignment problems, and for scheduling. In FarmAssist the number of named crops is limited by the length of a sentence, which keeps the mask dimension, and hence the table, small enough to display.

2.1.4 Graph and Network-Flow Algorithms

Many resource-allocation problems have a natural graph formulation, and FarmAssist uses that formulation for two of its features. Bipartite matching, solved by the augmenting-path method of Kuhn, gives each crop a distinct suitable fertilizer when such an assignment exists. The method repeatedly searches for an augmenting path and, when one is found, flips the matched and unmatched edges along it. Each flip increases the size of the matching by one, and when no augmenting path remains the matching is maximum by Berge's theorem.

Maximum flow extends the matching idea by attaching capacities to the edges. Ford and Fulkerson showed that the maximum flow equals the capacity of the minimum cut, the max-flow min-cut theorem, and Edmonds and Karp showed that always choosing the shortest augmenting path, found by breadth first search, makes the number of augmentations polynomial and therefore makes the algorithm polynomial in the size of the graph. FarmAssist models the supply of fertilizer to several fields as such a network: the source connects to each crop with a capacity equal to the number of fields of that crop, each crop connects to the fertilizers that suit it, and each fertilizer connects to the sink with a capacity equal to the number of bags in stock. The maximum flow is the number of fields that can actually be supplied, and the set of vertices still reachable from the source after the last augmentation names the bottleneck, because that set is a minimum cut whose capacity equals the flow. This converts a numeric answer into an explanation, which is exactly what a farmer needs in order to decide whether to buy more stock or to change the crop plan.

2.1.5 Approximation Algorithms

Some of the decisions that a farmer faces are NP-hard, so no polynomial exact algorithm is expected to exist and brute force does not scale. The standard response in the literature is to use an approximation algorithm that runs fast and carries a proven bound on how far its answer can be from the optimum.

The set cover problem asks for the smallest number of sets whose union contains a given universe. It is NP-hard, and it contains vertex cover as a special case. Chvatal analysed the greedy heuristic that always selects the set covering the

largest number of still-uncovered elements and proved that it uses at most a harmonic factor, that is one plus one half plus one third up to one over the size of the largest set, times the optimum. In FarmAssist the universe is the set of crops named by the farmer and the sets are fertilizer products, each of which suits some subset of the crops. The greedy choice is reported together with the harmonic factor so that the user can see the quality guarantee that accompanies the answer.

The knapsack problem is NP-hard in the weak sense and contains subset sum as a special case. The exact dynamic program is pseudo-polynomial: its running time depends on the numeric value of the budget rather than on the number of bits needed to write it. When the budget is large, a greedy method that sorts the items by benefit per rupee gives a fast approximate answer. Because taking the best single item as well and keeping the better of the two results raises the guarantee to at least one half of the optimum, FarmAssist computes both the greedy selection and the best single item and reports the better of the two. The exact dynamic program and the approximation are displayed side by side, which makes the trade-off visible and honest rather than hidden.

2.1.6 Randomised and Parallel Algorithms

Randomisation is used in algorithm design to remove the dependence of performance on the particular input. QuickSort with a deterministic pivot rule, such as always choosing the last element, degenerates to quadratic time on already-sorted input. Choosing the pivot uniformly at random makes the expected running time O(n log n) regardless of the input order, and the algorithm remains always correct, so it is a Las Vegas algorithm: only its running time is random. FarmAssist uses randomized QuickSort to rank search results, diagnosis candidates, pest results, and related articles, and the random choice is exactly what protects the ranking from a knowledge base that happens to be stored in sorted order.

Parallel algorithms are used where the work naturally decomposes. The map-reduce model applies an independent function to every element and then combines the results, and the parallel prefix-sum or scan operation, studied in depth by Blelloch, computes all running totals of a sequence using two sweeps up and down a binary tree. The model is analysed by stating two quantities: the work,

which is the total number of operations, and the span, which is the length of the longest dependency chain. Brent's theorem then bounds the running time on p processors by work divided by p plus span. FarmAssist scores the crop table against a climate profile with a parallel map and reduce over a fork-join tree, and computes running costs in the greedy fertilizer process with a parallel prefix sum, reporting the work and the span of each computation in the trace.

2.1.7 Findings and Research Gap

The survey leads to three findings. First, the string algorithms solve the entity-detection and relevance-scoring problems at the scale of this knowledge base comfortably, and their combination, rather than any one of them alone, is what makes the pipeline tolerant of real questions. Second, the decision-support questions that a farmer actually asks map naturally onto dynamic programming, matching, flow, and approximation problems, so the choice of algorithm family can be driven by the shape of the question. Third, the classical literature already provides, for each of those families, an algorithm with predictable cost and, where the problem is NP-hard, a proven quality bound.

The gap that this project addresses is integration rather than invention. Published agricultural information systems do not expose the algorithmic reasoning behind their answers, and published textbook treatments of the algorithms do not connect them to a practical decision-support problem. FarmAssist positions itself between the two: it is a single agricultural assistant whose every answer is produced by a named classical algorithm operating on a local knowledge base, with the algorithm and its intermediate results visible to the user through the trace, and with an honest statement, wherever a problem is NP-hard, of whether the answer is exact or approximate and what the guarantee is.

2.2 Existing System

In the conventional approach a farmer who needs advice follows one of a small number of paths. The farmer may search several websites and read separate pages for the crop, the disease, and the fertilizer; may telephone or visit an agricultural extension officer and wait for an appointment; may ask an experienced neighbour;

or may consult a printed booklet. Each of these paths has a cost, and none of them processes the question in the form in which the farmer actually poses it.

The limitations of the existing approach are summarised in Table 2.2. The common thread is that the existing systems are organised around records rather than around questions. A record-oriented system requires the user to know the name of the record, to spell it correctly, and to combine the records mentally. None of these requirements matches the way a farmer speaks, and none of them produces an optimisation answer at all.

Table 2.2: Limitations of the existing approach and the response in FarmAssist

Existing approach	Limitation	FarmAssist response
Basic keyword search	Fails when the sentence contains several patterns, and rescans the documents once per pattern.	Aho-Corasick, KMP, and Rabin-Karp used together.
Exact spelling search	Returns nothing when the user makes an ordinary typing mistake.	Edit Distance repair and word-segmentation dynamic programming.
Simple fertilizer lookup	Lists products but cannot choose a combination within a budget.	0/1 Knapsack for the exact basket and greedy knapsack for the approximation.
Fixed crop-fertilizer list	Cannot resolve a competing assignment when several crops want the same product.	Bipartite matching and Bitmask dynamic programming.
Manual stock planning	Cannot say how many fields can actually be supplied, or why.	Maximum flow with a minimum cut that names the bottleneck.
Unranked web search	Returns a list of links with no relevance ordering over agronomic records, and requires connectivity.	Local knowledge base with rolling-hash scoring and randomized QuickSort ranking.
Expert advisory services	Authoritative but limited in availability and not reachable at the moment the question arises.	Immediate offline answer with an explanation of the reasoning.

2.3 Proposed System

FarmAssist provides a single terminal interface through which a farmer can ask any of the supported question types. The system first answers purely conversational input and weather questions directly. For every other question it

repairs the words, normalises the alternate names, detects the agricultural entities, determines the intention, selects the algorithm that fits the intention, executes it against the local knowledge base, and formats the answer on a consistent layout. An optional trace explains at every stage which algorithm ran and what it produced.

The proposed system differs from the existing approach in four respects. It is question-oriented rather than record-oriented, so the user does not need to know how the data is organised. It is tolerant of the way people actually type, because repair and normalisation precede interpretation. It answers optimisation questions, not only retrieval questions, by mapping the question onto an appropriate algorithm family. And it is explainable, because the answer is accompanied by the name of the algorithm and its intermediate result rather than by a single opaque score.

2.4 Functional Requirements

The functional requirements state what the system must do. Each requirement is identified by a code and is traceable to at least one feature that is described in Chapters 4 and 5.

Table 2.3: Functional requirements

ID	Requirement
FR-01	The system shall load crop, disease, pest, fertilizer, article, symptom, synonym, and small-talk records from the local knowledge base at start-up and shall report how many records were loaded.
FR-02	The system shall accept natural-language text entered by the user at a prompt and shall treat the whole line as the question.
FR-03	The system shall correct common spelling errors in the question using edit distance and shall split joined words using word-segmentation dynamic programming.
FR-04	The system shall normalise alternate, regional, and transliterated names to their canonical forms using a synonym dictionary.
FR-05	The system shall detect all crops, diseases, pests, symptoms, and fertilizers mentioned in the question in a single pass over the sentence.
FR-06	The system shall determine the intention of the question and route it to the corresponding feature.
FR-07	The system shall search the knowledge base and rank the matching records and

ID	Requirement
	articles by relevance, and shall offer related reading.
FR-08	The system shall diagnose the possible diseases of a crop from the symptoms reported by the user and shall rank the candidates.
FR-09	The system shall recommend a fertilizer basket for a crop within a stated budget, and shall also report the greedy approximation for comparison.
FR-10	The system shall assign one distinct suitable fertilizer to each named crop and shall be able to find the assignment with the greatest total benefit.
FR-11	The system shall determine how many fields can be supplied from a limited stock of fertilizer and shall identify the limiting resource.
FR-12	The system shall find a small set of fertilizer products that between them suit every named crop and shall report the approximation guarantee.
FR-13	The system shall recommend crops suited to a stated rainfall, temperature, or water profile and shall rank the recommendations.
FR-14	The system shall provide current weather, a multi-day outlook, and derived field advice when the weather module is configured, and shall degrade gracefully when it is not.
FR-15	The system shall display live autocomplete suggestions drawn from the knowledge base vocabulary while the user types.
FR-16	The system shall display algorithm trace information when trace mode is enabled, naming the algorithm and its intermediate results.
FR-17	The system shall provide a demonstration command that runs each implemented algorithm on a small, hand-checkable input.
FR-18	The system shall allow the user to browse the knowledge base by record type and to control the appearance of the output, including colour, character set, width, and centring.
FR-19	The system shall handle invalid, empty, or unrecognised input with a helpful message rather than an error or a crash.
FR-20	The system shall terminate cleanly when the user exits and shall never leave the terminal in a non-default state.

2.5 Non-Functional Requirements

The non-functional requirements describe the quality attributes that the system must exhibit. They are stated as measurable expectations wherever possible, because an advisory tool is judged by its usability and reliability as much as by its algorithmic content.

Table 2.4: Non-functional requirements

Attribute	Requirement
Usability	The interface shall accept plain-language questions without any special syntax, shall present answers as readable cards, lists, and tables within the terminal width, and shall never require the user to retype a question because of a spelling mistake.
Performance	A typical question shall be answered in well under a second on the specified hardware; the entity detection shall run in a single pass; and the crop planner shall use the available cores through a fork-join decomposition.
Reliability	Invalid input, a missing weather configuration, an unavailable knowledge base file, and a network failure shall each produce an understandable message and shall leave the application usable.
Maintainability	Algorithms, models, engines, user interface, and data shall remain in separate packages with no reverse dependencies, so that an algorithm can be replaced without touching the display code.
Portability	The application shall compile and run on any operating system with a JDK 21 or newer and a terminal that supports UTF-8; an ASCII fallback shall be provided for terminals that do not.
Extensibility	New knowledge base records shall be addable without changing any algorithm class, and new features shall be addable by extending the intent routing rules rather than the input handling.
Explainability	Every answer shall be traceable to a named algorithm, and the trace shall report the intermediate quantities that justify the answer.
Security and privacy	The weather API key shall be stored outside the repository in a configuration file that is not committed, and the application shall send nothing other than the weather request over the network.
Offline capability	Every feature except the optional weather module shall function with no network connection whatsoever.
Documentation quality	The source code shall be organised and commented so that a reader can locate the implementation of any algorithm named in this report, and the report shall state the complexity of each algorithm.

2.6 Hardware Requirements

The hardware requirements are deliberately modest, because the system is intended to run on a shared or low-cost machine. The parallel crop planner benefits from additional cores, but the application remains fully functional on a single core.

- Processor: any dual-core processor running at 1 GHz or above. Additional cores improve the crop planner, which decomposes its work over a fork-join pool.

- Main memory: 2 GB minimum, 4 GB recommended. The application itself uses only a few tens of megabytes at run time.
- Disk space: less than 10 MB in total for the source files, the compiled classes, the knowledge base, and the single input library.
- Display: a standard terminal or command prompt window. Windows Terminal is recommended because it renders the box-drawing characters and 24-bit colour used by the interface; an ASCII mode is available as a fallback.
- Input device: a standard keyboard. No mouse or pointing device is required.
- Network: no network interface is required for any feature except the optional live weather module.

2.7 Software Requirements

The software requirements are equally modest. The project deliberately avoids build tools and frameworks so that it can be compiled and executed with nothing more than a JDK.

- Java Development Kit 21 or above. The project was developed and tested on JDK 21 and JDK 25 and uses only the standard class library for all algorithmic work.
- Operating system: Windows 10 or 11, Linux, or macOS. The provided launcher scripts target Windows; the manual commands work on any platform.
- JLine 3.27.1 is the only third-party library. It is used solely to read keyboard input one key at a time so that the live suggestion box can respond to arrow keys and the Tab key. No searching, sorting, or decision logic uses the library.
- Knowledge base: plain pipe-separated text files with the .txt extension. Any text editor is sufficient to extend them.
- Weather module: an OpenWeatherMap API key stored in a configuration file outside version control, with the provided example file used as a template.
- Terminal character set: UTF-8 is recommended; the application switches the console code page on Windows and provides an ASCII fallback.

2.8 Use Case View of the System

The functional behaviour of the system is summarised by the use case view shown in Fig. 2.1. The farmer is the only actor. Within the system boundary, the use cases correspond closely to the functional requirements, and the include relationships show that every question-oriented use case first passes through the common pipeline of word repair, normalisation, and entity detection, and that every answer may optionally be accompanied by the algorithm trace.

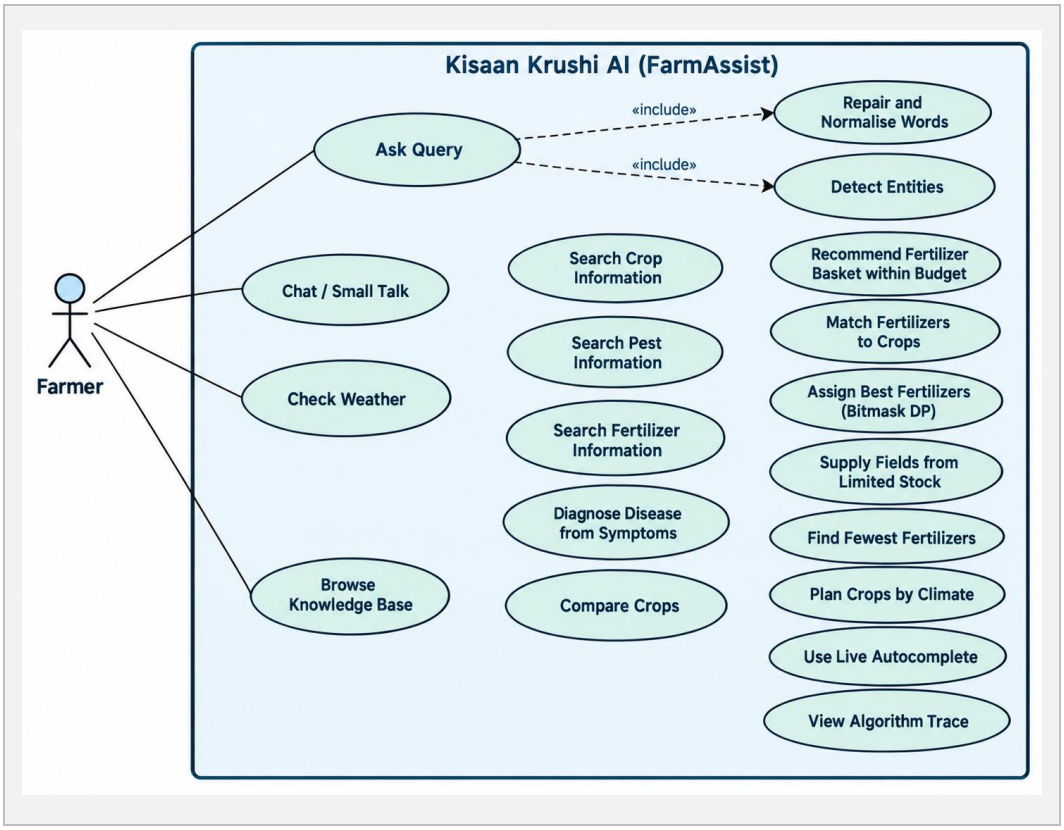


Fig. 2.1: Use Case Diagram of FarmAssist The use case diagram summarises the functional scope that the requirements above define.

2.9 Feasibility Study

The feasibility of the proposed system was examined against four criteria before implementation began.

- **Technical feasibility:** every algorithm required by the design is a classical algorithm with a known implementation and a known complexity, and each has been published and is taught in the course. The knowledge base is small enough to be read from plain text files into memory at start-up, so no database engine is required. The only external service, the weather API, is

optional. The project is therefore technically feasible with the available JDK and a text editor.

- **Economic feasibility:** the project uses no paid service, no licensed software, and no hardware beyond an ordinary computer. The free tier of the weather service is sufficient for demonstration purposes. The cost of the project is therefore limited to development time.
- **Operational feasibility:** the interface is a text prompt that accepts an ordinary sentence, which is within the reach of the intended user. The offline-first design removes the dependence on connectivity, and the trace mode allows a reviewer to verify the reasoning. The system is therefore operationally feasible in the environment for which it is intended.
- **Schedule feasibility:** the work was divided into a knowledge base phase, an algorithm phase, an engine and integration phase, a user interface phase, and a testing and documentation phase, each with a review checkpoint. The division allowed the string algorithms to be completed and demonstrated first and the decision-support algorithms to be added afterwards without redesigning the pipeline.

2.10 Requirement Traceability

Every functional requirement stated in Section 2.4 is realised by a specific feature, and every feature is answered by a specific algorithm. The traceability matrix in Table 2.5 records that mapping so that the completeness of the implementation can be checked requirement by requirement rather than feature by feature.

Table 2.5: Requirement traceability matrix

Requirement	Realising feature	Algorithm or mechanism used
FR-01	Knowledge base loading at start-up	Pipe-separated line reader with a tolerant numeric parser
FR-02	Question prompt	Character-level input reader with a line buffer
FR-03	Spelling repair and joined-word splitting	Edit distance and word-segmentation dynamic programming
FR-04	Synonym normalisation	Second Aho-Corasick automaton over the synonym table
FR-05	Entity extraction	Main Aho-Corasick automaton with word-boundary verification

Requirement	Realising feature	Algorithm or mechanism used
FR-06	Intent detection and routing	Ordered keyword rule set evaluated over the entities
FR-07	Search and related reading	Rabin-Karp scoring, KMP snippet location, suffix array with LCP, randomized QuickSort ranking
FR-08	Disease diagnosis from symptoms	Aho-Corasick extraction, KMP exact matching, randomized QuickSort ranking
FR-09	Fertilizer basket within a budget	0/1 Knapsack dynamic programming and greedy knapsack
FR-10	Crop-to-fertilizer assignment	Kuhn bipartite matching and bitmask dynamic programming
FR-11	Supply from limited stock	Edmonds-Karp maximum flow with minimum cut
FR-12	Farm-wide fertilizer coverage	Greedy set cover with the harmonic bound reported
FR-13	Climate-based crop planning	Parallel map, reduce and prefix sum, then randomized QuickSort
FR-14	Live weather and field advice	External API call, minimal JSON reader, threshold advice rules
FR-15	Live autocomplete	Trie with bounded depth-first completion collection
FR-16	Algorithm trace	Shared trace collector written by every algorithm
FR-17	Algorithm demonstration	Direct calls to each algorithm with small fixed inputs
FR-18	Knowledge base browsing and appearance control	Record listings and a theme component holding colour and character set tables
FR-19	Handling of invalid or unrecognised input	Nearest-word fallback using edit distance and a fail-soft question handler
FR-20	Clean shutdown	Explicit exit handling that restores the terminal settings

CHAPTER 3: SYSTEM DESIGN

This chapter presents the design of the system before its implementation. It states the design goals, presents the high-level architecture, describes each layer in turn, follows the flow of data through the application at two levels of detail, describes the design of the knowledge base, the class and package structure, and the sequence of interactions for a representative question, and closes with the strategy that the system uses to select an algorithm for a question.

3.1 Design Goals and Principles

The design is guided by five goals. The first is separation of concerns: the algorithm implementations must not know anything about agriculture, and the user interface must not know anything about algorithms, so that each can be understood, tested, and replaced independently. The second is a single query pipeline: every question, whatever feature ultimately answers it, must pass through the same repair, normalisation, and extraction stages, so that behaviour is consistent and a new feature inherits the language tolerance for free. The third is data-driven behaviour: the knowledge base determines what the system knows, and adding a record must not require a code change. The fourth is explainability: the system must be able to say which algorithm produced an answer and what its intermediate results were. The fifth is honesty about hard problems: when a decision problem is NP-hard, the design must state whether an exact or an approximate answer was produced and what the guarantee is.

The following design principles follow from those goals and were applied consistently throughout the implementation.

- **No hidden library work:** the Java standard library is not used for searching, matching, ranking, or sorting in the algorithmic core. Library calls would hide the very work that the project is meant to demonstrate.
- **Pure algorithms:** each class in the algorithm package takes plain inputs such as strings, arrays, and integers, and returns plain outputs. No algorithm class reads a file or prints to the terminal directly.
- **One direction of dependency:** the user interface depends on the engine, the engine depends on the algorithms and the models, and the algorithms depend

on nothing. There are no reverse dependencies and no cycles between packages.

- **Fail softly:** missing data, an unconfigured weather key, or an unrecognised question produce a readable message, never a stack trace in front of the user.
- **Observable execution:** every algorithm that carries a decision reports at least one intermediate quantity to the trace, so that the reasoning can be followed.

3.2 High-Level Architecture

FarmAssist is organised as a five-layer architecture. The layers are, from the user downwards, the user interface layer, the query processing layer, the feature layer, the algorithm layer, and the data layer. The high-level architecture is shown in Fig. 3.1. The essential property of the design is that a question travels downwards through the layers and an answer travels upwards, and that no layer reaches past its immediate neighbour.

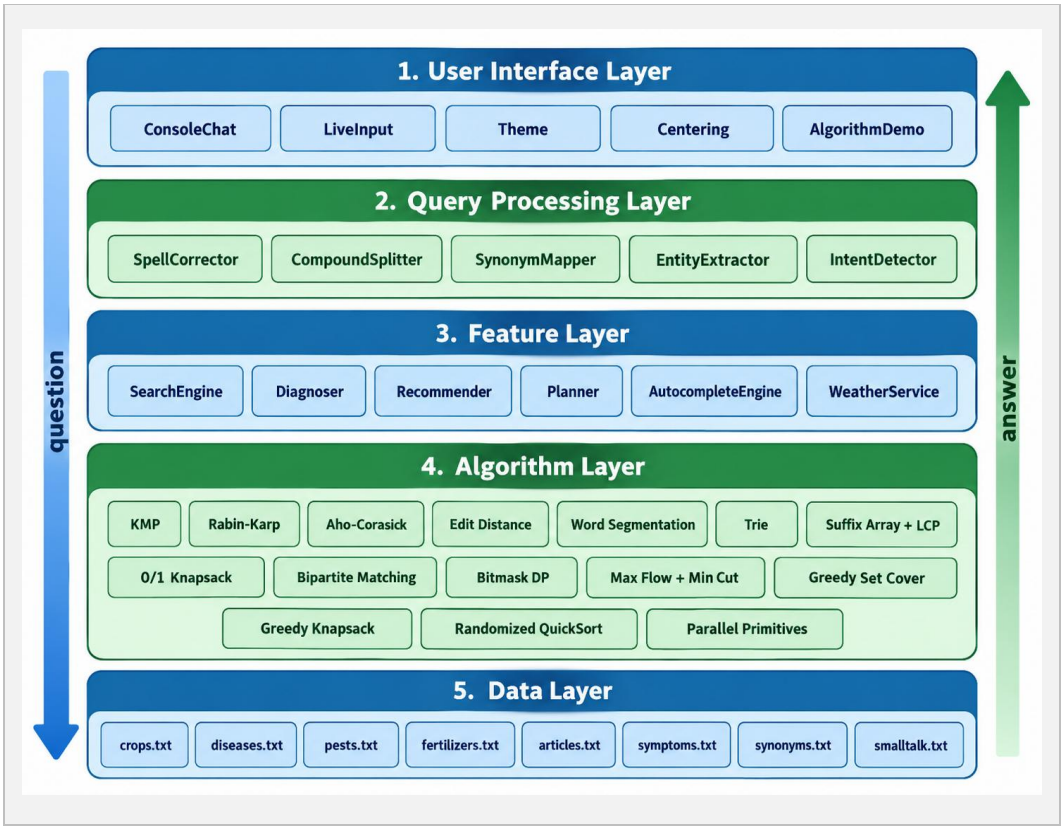


Fig. 3.1: High-Level Layered Architecture of FarmAssist The architecture diagram is the primary figure of this chapter and is referred to throughout the layer descriptions below.

3.3 Description of Each Layer

Each layer is described below in terms of its responsibility, the classes that it contains, and the interfaces that it exposes to the layers above and below it.

3.3.1 User Interface Layer

The user interface layer owns everything that the user sees. ConsoleChat is the main loop: it prints the banner, loads the engine, accepts a line of input, hands the line to the engine, and renders the returned response. LiveInput is responsible for reading individual key presses through the JLine library so that the suggestion box can be updated after every keystroke without waiting for the Enter key. Theme holds the colour palette and the box-drawing characters, including the ASCII fallback used when the terminal cannot render Unicode. Centering computes the left offset needed to place content in the middle of the terminal so that the layout does not drift when the window is resized. AlgorithmDemo prints a sequence of small, hand-checkable examples, one for each algorithm, so that a reviewer can verify the implementations independently of the knowledge base.

The layer exposes a single method to the rest of the application. It receives a question as a string and a response object, and it does not know which feature produced the response. This is what allows the interface to remain unchanged when a new feature is added to the engine.

3.3.2 Query Processing Layer

The query processing layer turns a raw sentence into a structured query. SpellCorrector proposes corrections for words that do not appear in the knowledge base vocabulary, using edit distance together with the stored word list. CompoundSplitter applies word-segmentation dynamic programming to words that are too long to be a single vocabulary entry and that can be decomposed into two or more known words, which is how lateblight becomes late blight. SynonymMapper uses a second Aho-Corasick automaton built over the synonym table to rewrite local and transliterated names to their canonical forms. EntityExtractor runs the main automaton, which contains every crop, disease, pest, fertilizer, and symptom in the knowledge base, over the repaired sentence and returns the entities grouped by type. IntentDetector then applies keyword rules to

the repaired sentence and the detected entities and returns the name of the feature that should answer the question.

The output of the layer is a small, well-defined object: the repaired sentence, the list of corrections that were applied, the list of normalisations that were applied, the entities by type, and the detected intent. Everything above the layer works with that object, and everything below it works with plain values.

3.3.3 Feature Layer

The feature layer implements each kind of answer that the system can give, and calls the algorithm layer to do the computational work. SearchEngine scores the indexed documents against the query terms using Rabin-Karp, locates the terms in the winning documents using KMP, and adds related reading using the suffix array and LCP structure. Diagnoser matches the reported symptoms against the symptom list of each disease record using exact matching and ranks the candidates. Recommender handles the four fertilizer decision features: the budget basket through knapsack, the one-bag-per-crop assignment through matching and Bitmask dynamic programming, the limited-stock supply through maximum flow and minimum cut, and the farm-wide coverage list through greedy set cover. Planner scores the crop table against a climate profile with parallel map and reduce and ranks the results. AutocompleteEngine answers prefix queries from the trie for the live suggestion box. WeatherService loads the configuration, calls the external weather interface, and converts the readings into field advice.

Each feature returns a response object that carries the formatted answer together with the trace entries produced while it ran. The feature layer is therefore the only layer that knows both what an agricultural question means and which algorithm answers it.

3.3.4 Algorithm Layer

The algorithm layer contains thirteen algorithms plus the trie used for autocomplete, implemented from first principles. Each class is self-contained: it takes strings, arrays, or integers and returns matches, counts, tables, assignments, flows, or sorted orders. No class in this layer reads the knowledge base, prints to the terminal, or refers to agriculture in any way. This is what makes the layer

reusable and what makes the algorithm demonstration command possible, because a demonstration can call any algorithm directly with a tiny artificial input.

Table 3.1: Algorithm layer contents and the workload carried by each class

Class	Algorithm	Workload in the application
KMP	Knuth-Morris-Pratt	Exact symptom matching in disease records; locating a term for a snippet.
RabinKarp	Rabin-Karp rolling hash	Counting query-term occurrences in every indexed document to compute relevance scores.
AhoCorasick	Aho-Corasick automaton	Single-pass detection of every entity; a second automaton for synonym rewriting.
EditDistance	Levenshtein distance	Spelling repair and the nearest-word fallback when no entity is found.
Trie	Prefix tree	Live autocomplete suggestions while the user types.
SuffixArrayLCP	Suffix array with LCP	Related reading by longest common substring between the winning document and the rest of the index.
Knapsack	0/1 Knapsack dynamic programming	Best fertilizer basket for a crop within a stated budget.
KnapsackGreedy	Greedy knapsack with best-single-item comparison	Fast approximate basket shown beside the exact answer.
BipartiteMatching	Kuhn augmenting paths	One distinct suitable fertilizer per crop.
BitmaskAssignment	Bitmask dynamic programming	Assignments that maximise total benefit over a small set of crops.
MaxFlow	Edmonds-Karp with minimum cut	Number of fields that can be supplied from limited stock, and the bottleneck that limits it.
SetCoverGreedy	Greedy set cover	Fewest fertilizer products that between them cover every named crop.
RandomizedQuickSort	Randomized quicksort	Ranking of search results, diagnoses, pests, and related articles.
ParallelPrimitives	Parallel map, reduce, prefix sum	Scoring the crop table against a climate profile; running costs in the greedy basket.

3.3.5 Data Layer

The data layer holds the agricultural knowledge base and the code that reads it. DataLoader reads each text file, ignores comment lines that begin with the hash character, splits each record on the pipe character, and constructs the corresponding model object. It then builds the search index, which contains both the article records and the profiles generated from the crop, disease, pest, and fertilizer records, so that a search question can retrieve a structured record as well as a free-text article. The loader also builds the vocabulary used by the spelling corrector, the synonym table, and the word list used by the segmentation dynamic program.

Keeping the data layer free of logic is what allows the knowledge base to grow by editing text. A new crop is a new line in the crop file; no code changes and no recompilation of the algorithm layer are needed.

3.4 Data Flow Design

The data flow through the application is presented at two levels. The level zero diagram in Fig. 3.2 shows the system as a single process with its external entities and its data store. The level one diagram in Fig. 3.3 expands the single process into the main stages of the pipeline and shows how the knowledge base is consulted at each stage.

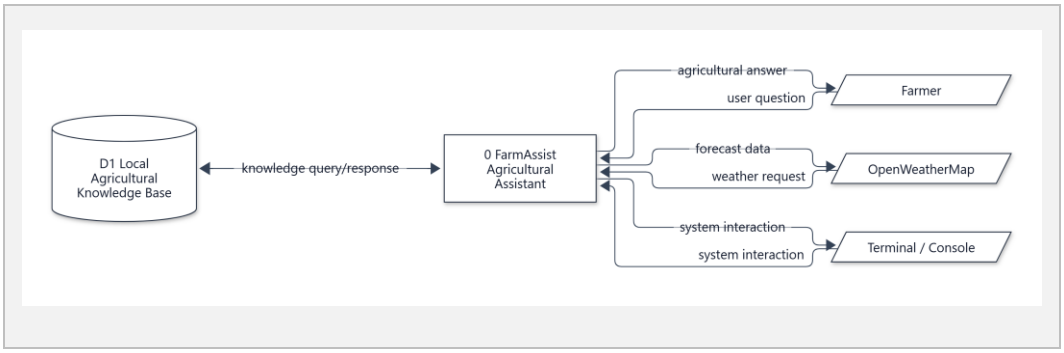


Fig. 3.2: Level 0 Data Flow Diagram of FarmAssist The level zero diagram fixes the system boundary and the external interfaces before the internal stages are described.

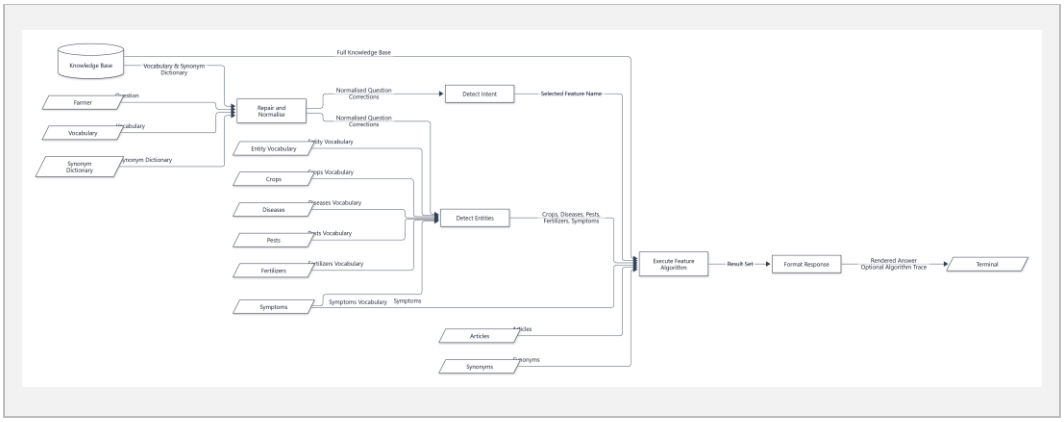


Fig. 3.3: Level 1 Data Flow Diagram of the Query Pipeline The level one diagram shows which stage consults which part of the knowledge base.

3.5 Query Processing Pipeline

The pipeline is the heart of the design. Its purpose is to remove ambiguity from the question before any decision is taken, and it is illustrated in Fig. 3.4. The stages are applied in a fixed order, and the order matters for correctness.

18. Small talk and weather are checked first. A greeting such as "hi" or a question such as "weather in guntur" is answered directly and the remaining stages are skipped. Weather is checked early because a town name is not an agricultural entity and would otherwise be mistaken for one.
19. Spelling repair is applied to every token that is not present in the vocabulary. Each unknown token is compared against the vocabulary with edit distance, and a correction is proposed only when the distance is small relative to the length of the word, so that a genuinely unknown word is left alone rather than being forced onto an unrelated term.
20. Word segmentation is applied to tokens that remain unknown and are long enough to be a compound. A dynamic program over the word list decides the split that maximises the product of the word frequencies, which is how lateblight becomes late blight.
21. Synonym normalisation is applied to the whole sentence using a second automaton built from the synonym table. This converts paddy, dhan, and chawal to rice, and bhindi to okra, before the entities are detected.
22. Entity detection runs the main automaton over the normalised sentence. Because the automaton holds every entity of every type, one pass produces all of them, grouped into crops, diseases, pests, fertilizers, and symptoms.

23. Intent detection applies the routing rules to the repaired sentence and to the detected entities. The rules look for budget expressions, counts of fields, crop lists, climate expressions, symptom words, and question phrases, and they produce the name of the feature that should answer the question.
24. Feature execution calls the selected feature, which in turn calls the algorithm layer. The feature returns its result together with the trace entries that were produced.
25. Response formatting lays out the answer on the terminal grid as a card, a list, or a table, and appends the trace when trace mode is enabled.



Fig. 3.4: Query Processing Pipeline of FarmAssist The pipeline diagram is the reference for the execution order described in the implementation chapter.

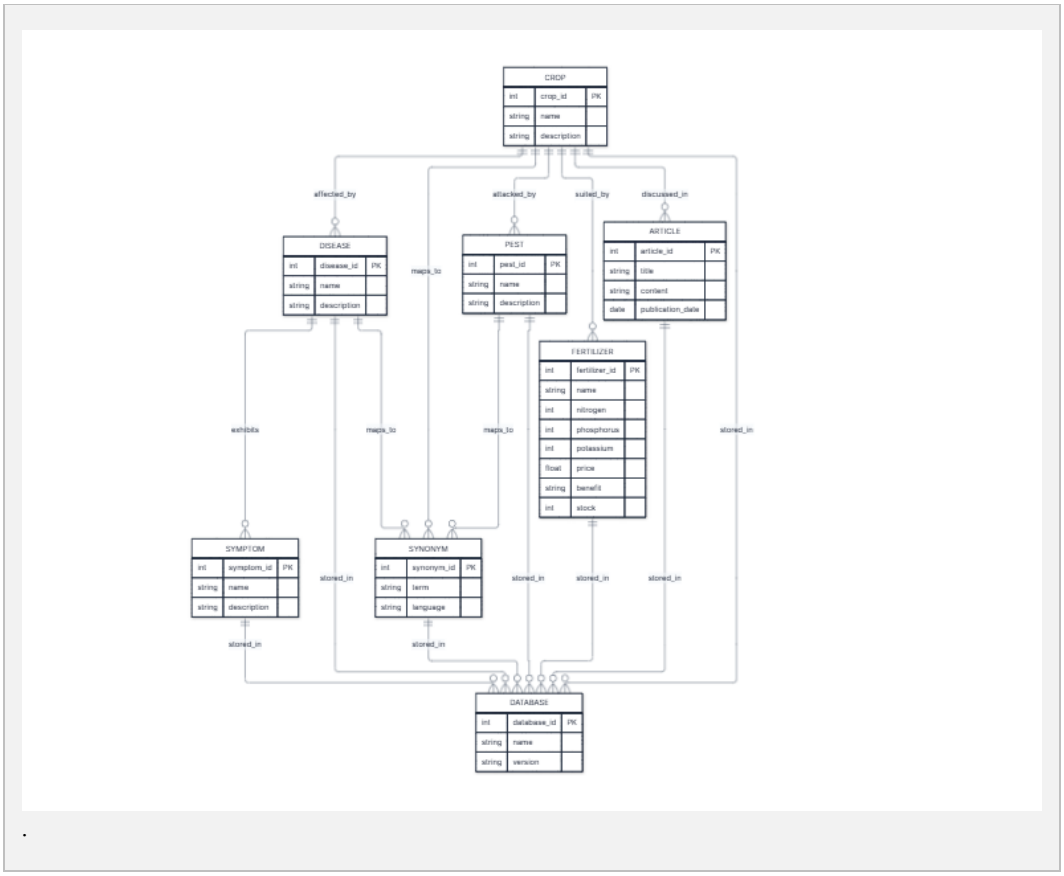
3.6 Knowledge Base Design

The knowledge base is stored as plain pipe-separated text files. This is a deliberate design decision: the data can be inspected, diffed, corrected, and extended with any text editor, and no database engine is needed. Each file begins with a comment line that names the columns, and blank lines and lines beginning with the hash character are ignored by the loader. The files are listed in Table 3.2.

Table 3.2: Knowledge base files and their contents

File	Purpose	Approximate contents
crops.txt	Crop profiles	Crop name, season, soil type, water requirement, nitrogen, phosphorus and potassium requirement, common diseases, duration, temperature range, rainfall range, spacing, varieties, expected yield.
diseases.txt	Disease records	Disease name, affected crop, symptom list, causal organism, and treatment information.
pests.txt	Pest records	Pest name, affected crop, damage description, and control measures.
fertilizers.txt	Fertilizer records	Product name, nitrogen, phosphorus and potassium percentages, price per bag, agricultural benefit score, and available stock.
articles.txt	Searchable articles	Field notes, cultivation practices, and advisory content used by the search feature.
symptoms.txt	Detection vocabulary	Symptom phrases inserted into the entity automaton.
synonyms.txt	Normalisation vocabulary	Local and alternate names mapped to canonical crop, disease, or pest names.
smalltalk.txt	Conversation replies	Greetings, thanks, and everyday responses matched by simple rules.
weather.example.txt	Weather configuration template	API key placeholder and default town, copied to weather.txt outside version control.

The relationships between the stored entities are shown in Fig. 3.5. A crop is associated with the diseases that affect it, the pests that attack it, the fertilizers that suit it, and the articles that discuss it. A disease carries a list of symptoms, which are themselves entities detected in the question, and this is what allows a diagnosis to be triggered by symptoms alone without the farmer naming the disease. A fertilizer carries its nutrient percentages, its price, its benefit score, and its stock, which are the fields used by the budget, assignment, and flow features. A synonym record points at a canonical term and supplies the alternative spellings that the normalisation automaton must recognise.



.

Fig. 3.5: Entity Relationship Diagram of the Knowledge Base The entity relationship diagram shows how a symptom can lead to a disease and a disease back to its crop.

3.7 Class and Package Design

The source code is organised into five packages. The package structure and the responsibility of each package are shown in Table 3.3. The dependency direction is strictly one way: ui depends on engine and model, engine depends on algo, model, and util, and algo depends on nothing at all. This guarantees that the algorithms can be tested and demonstrated in isolation and that a user interface change cannot break an algorithm.

Table 3.3: Package structure and responsibilities

Package	Classes	Responsibility
algo	AhoCorasick, KMP, RabinKarp, EditDistance, Trie, SuffixArrayLCP, Knapsack, KnapsackGreedy, BipartiteMatching, BitmaskAssignment, MaxFlow, SetCoverGreedy, RandomizedQuickSort, ParallelPrimitives	The reusable algorithm implementations. No project-specific or agricultural logic.

Package	Classes	Responsibility
engine	DataLoader, SpellCorrector, CompoundSplitter, SynonymMapper, EntityExtractor, IntentDetector, SearchEngine, Diagnoser, Recommender, Planner, AutocompleteEngine, WeatherService, SmallTalk, SearchResult	Loading the knowledge base, preprocessing the query, detecting intent, and implementing each feature.
model	Crop, Disease, Pest, Fertilizer, Article	Plain data objects that correspond to the record types in the knowledge base.
ui	ConsoleChat, LiveInput, Theme, Centering, AlgorithmDemo	The chat loop, live key input, colour and layout control, and the algorithm demonstration screens.
util	Trace, Json	The trace collector used by every algorithm and a minimal JSON reader used by the weather integration.

The relationships between the principal classes are shown in Fig. 3.6. ConsoleChat owns the engine objects and the search index. Each engine object depends on the algorithm classes that implement its feature. The model classes are used by the engine and are created only by DataLoader, which is the single point at which the knowledge base becomes objects. The Trace class is used by every algorithm so that the trace is a single, consistent stream rather than a set of scattered print statements.

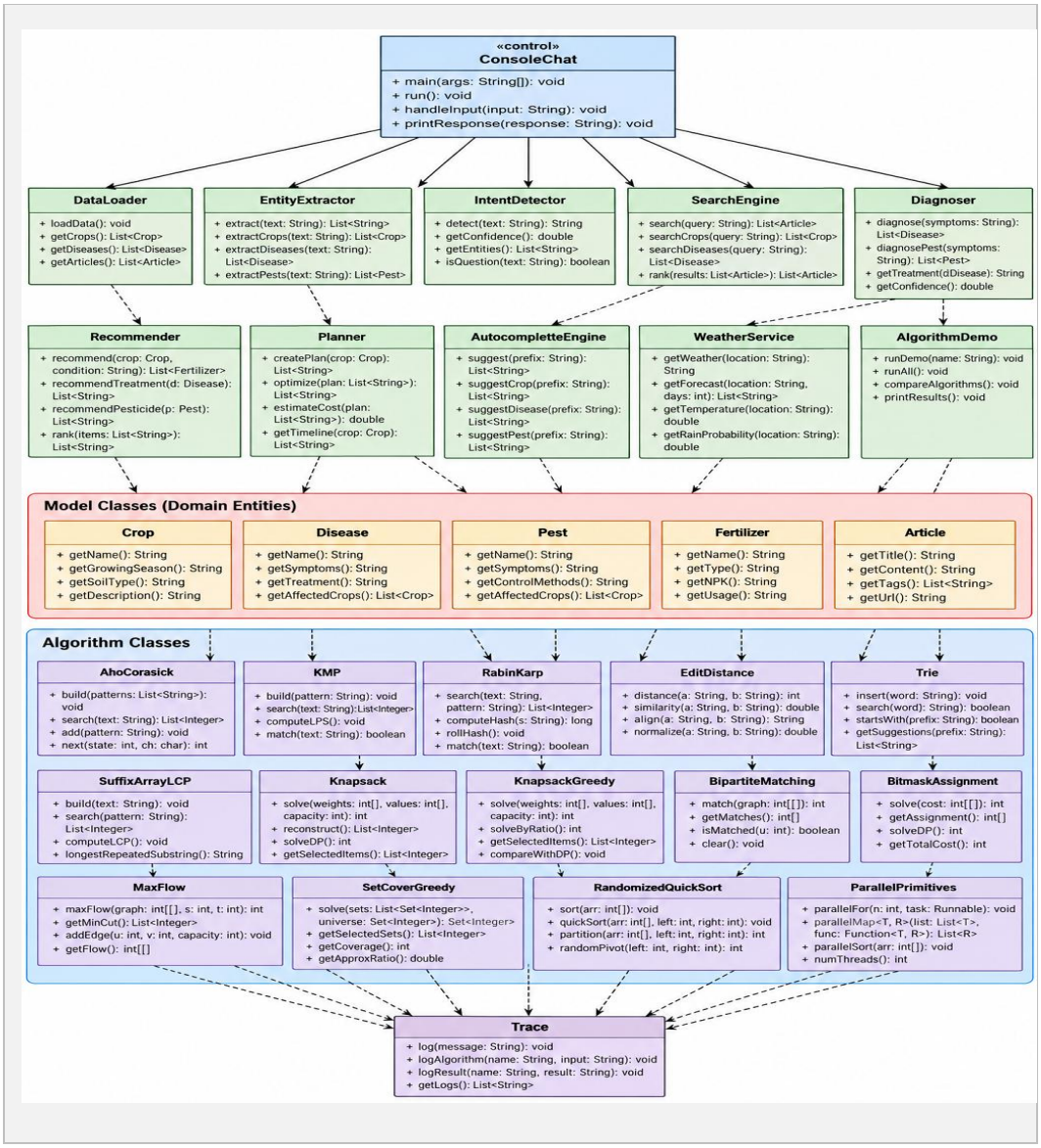


Fig. 3.6: Class Diagram of the Principal FarmAssist Classes The class diagram shows the one-way dependency direction that the design enforces.

3.8 Interaction Design for a Representative Question

The design of the query sequence is best explained with a concrete question. Consider the sentence "my tomatoe has yelow leaves and brown spots". The interaction that follows is shown as a sequence diagram in Fig. 3.7 and is described in prose below.

ConsoleChat passes the raw sentence to the repair stage. SpellCorrector reports that tomatoe is not in the vocabulary and that the nearest word is tomato at an edit distance of one, and that yelow is not in the vocabulary and that the nearest word is yellow at an edit distance of one. CompoundSplitter finds nothing to split.

SynonymMapper finds no synonym in the sentence. EntityExtractor runs the automaton and reports the crop tomato and the two symptoms yellowing leaves and brown spots. IntentDetector sees the crop, sees that symptoms are present, sees that no budget expression, field count, or climate expression is present, and routes the question to the diagnosis feature. Diagnoser takes the two symptoms and tests each of them against the symptom list of every disease record that affects tomato, using exact matching, and counts how many of the reported symptoms each record contains. RandomizedQuickSort ranks the records by that count and by the total length of the matched phrases. ConsoleChat renders the ranked list with the treatment information for each candidate and appends the trace, which reports the number of patterns in the automaton, the number of exact matches performed, and the number of records ranked.

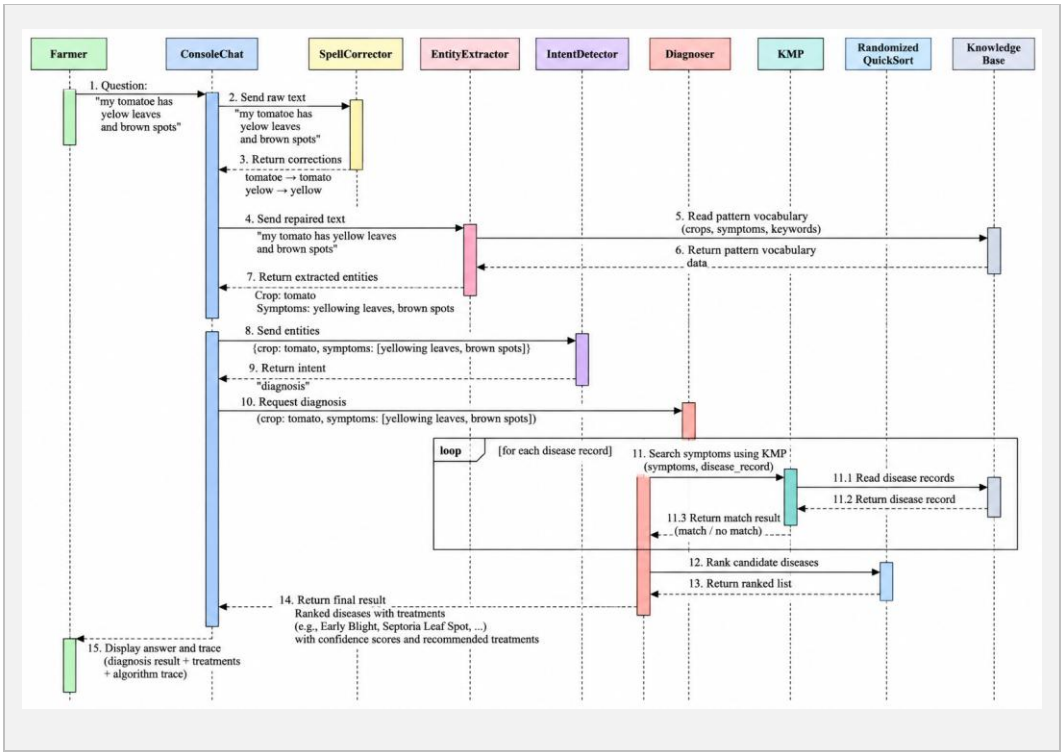


Fig. 3.7: Sequence Diagram for Processing a Diagnosis Question The sequence diagram makes the order of the pipeline concrete for one query.

3.9 Algorithm Selection Strategy

The choice of algorithm is driven by the shape of the question rather than by the topic of the question. Table 3.4 states the rule that the intent detector applies for each question shape and names the algorithm family that answers it. This table is the design-level statement of the first course outcome, and it is the reason that the

project can claim to demonstrate problem classification rather than merely algorithm implementation.

Table 3.4: Question shape, detected intent, and the algorithm family used

Question shape	Intent	Algorithm family	Feature output
A greeting or a general conversational remark	small talk	Keyword match on the conversational reply table	A short conversational reply
A weather question naming a town or asking about rain	weather	External API call with derived field advice	Current conditions, outlook, and advice
A question naming a crop and asking how to grow it	crop info	Entity detection and profile lookup	The full crop profile
A question naming two or more crops with the word compare	crop compare	Lookup of both profiles and field-wise alignment	A side-by-side comparison
A sentence naming a crop and describing symptoms	diagnosis	Aho-Corasick extraction, KMP exact matching, randomized QuickSort ranking	Ranked candidate diseases with treatment
A question naming a pest or asking how to control it	pest info	Entity detection and pest record lookup	Pest damage and control measures
A question naming a fertilizer or asking what it is	fertilizer info	Record lookup and nutrient summary	Fertilizer profile
A question naming a crop and a budget, such as "under 3000"	budget	0/1 Knapsack with the greedy approximation beside it	Exact and approximate fertilizer baskets
A question naming several crops with the word match	assignment	Bipartite matching with Kuhn augmenting paths; Bitmask dynamic programming for the best assignment	One suitable fertilizer per crop, and the best assignment
A question naming crops together with counts of fields, such as "3 fields of rice"	supply	Edmonds-Karp maximum flow with minimum cut	Number of fields supplied and the bottleneck
A question asking for the fewest products covering a set of crops	set cover	Greedy set cover with the harmonic bound reported	A short product list that covers every named crop
A question describing rainfall, temperature, or water conditions	planning	Parallel map and reduce over the crop table, then randomized QuickSort ranking	Ranked crops suited to the stated climate
Any other question containing agricultural words	search	Rabin-Karp scoring, KMP snippet location, suffix array related reading, randomized QuickSort ranking	Ranked records and articles with snippets
Input that matches nothing	unknown	Nearest-word fallback using edit distance	A helpful message with suggestions

3.10 Data Structures Used in the Design

The design uses a small but well-chosen set of data structures, and each of them is chosen for the operation that dominates its use. They are summarised in Table 3.5.

Table 3.5: Data structures used in the design and the reason for each choice

Structure	Where it is used	Reason for the choice
Trie of characters	Entity automaton and autocomplete	Prefix sharing keeps the automaton small; a lookup costs one step per character.
Failure links over a trie	Entity automaton and synonym automaton	Allows a single scan of the text with no restart after a mismatch.
Suffix array with an LCP array	Related reading	Finds the longest common substring of two documents without pairwise comparison.
Two-dimensional integer table	Edit distance, knapsack, bitmask assignment	Stores overlapping subproblem answers so that each is computed once.
Bitmask over crops	Bitmask assignment	Represents the set of already-served fields in a single integer.
Adjacency list with edge objects	Matching and maximum flow	Supports residual edges and reverse-edge updates constant in degree.
Breadth-first queue	Edmonds-Karp	Guarantees that the augmenting path found is a shortest one, which is what makes the algorithm polynomial.
Hash map of word frequencies	Spelling and segmentation	Constant-time lookup of a candidate word and its frequency.
Heap of candidate sets	Greedy set cover	Repeatedly extracts the set that covers the most uncovered elements.
Fork-join task tree	Parallel planner and prefix sum	Expresses map, reduce, and scan with a bounded span.

3.11 Design of the Trace and Observability

The trace is treated as a design element rather than as a debugging aid, because the trace is what allows a reader or a reviewer to verify that an answer really was produced by the algorithm that the report claims. The design of the trace follows three rules.

- **A single collector:** there is exactly one trace collector in the application, and it lives in the util package. No algorithm prints to the terminal directly. An algorithm calls the collector with a short fact, for example the number of patterns that were inserted into the automaton, and the collector decides whether the fact is shown and how it is rendered. This is what keeps the algorithm classes free of presentation logic.
- **Facts rather than messages:** a trace entry is a fact and a value, not a sentence. The collector therefore receives the name of the algorithm, the operation, and the numeric or textual result. The renderer is the only component that knows how a fact should be phrased for the user, so the phrasing can be changed without touching any algorithm.
- **Cleared per question:** the collector is cleared at the beginning of every question, so the trace always describes exactly one answer and can never mix the reasoning of two different questions.

The design consequence is that every feature can be demonstrated in three steps: ask a question, read the answer, and read the trace to see which algorithm produced it. The trace entries that each algorithm contributes are listed in Table 3.6. This table is also the specification that the algorithm demonstration command satisfies, because each algorithm must be able to state the same facts when it is run on a small artificial input as it does when it is run on a real question.

Table 3.6: Trace facts contributed by each algorithm

Algorithm	Facts reported to the trace
Aho-Corasick	Number of patterns inserted, number of automaton states created, number of matches reported in the single pass.
KMP	Number of exact searches performed, number of matches found, and the length of the pattern that produced each match.
Rabin-Karp	Number of documents scanned, number of term occurrences counted, number of hash candidates verified.
Edit distance	Each substitution that was applied with its distance, and the candidate words considered for an unknown word.
Word segmentation	The token that was split, the parts chosen, and the score of the chosen split.
Trie	The prefix that was looked up and the number of completions returned.
Suffix array and	The length of the longest common substring found and the document

Algorithm	Facts reported to the trace
LCP	that shares it.
0/1 Knapsack	Table dimensions, best benefit, total cost, and the products recovered from the table.
Greedy knapsack	The approximation bound and the comparison between the approximate and the exact benefit.
Bipartite matching	Number of augmenting paths found and number of crops served.
Bitmask assignment	Number of states in the table, number of states visited, best total benefit, and the recovered assignment.
Maximum flow	Number of augmenting paths, maximum flow achieved, and the capacity of the minimum cut that was found.
Greedy set cover	Number of rounds performed, products selected, and the harmonic bound for the universe size.
Randomized QuickSort	Number of elements ranked and the number of partition levels used.
Parallel primitives	Work, span, and the number of leaf tasks for the map, reduce, and prefix-sum phases.

3.12 Design of the Interactive Input

The interactive input is designed as an explicit state machine so that every key press has a defined effect. The states and the transitions are shown in Table 3.7. Keeping the machine small was a deliberate choice: a character-level input reader can easily become the source of subtle behaviour, and an explicit transition table makes the behaviour testable.

Table 3.7: States and transitions of the interactive input

State	Key	Effect	Next state
Typing	Printable character	Append to the buffer and refresh the suggestions	Typing with suggestions open
Typing	Enter	Submit the buffer when it is not empty; otherwise redraw the prompt	Processing or Typing
Typing	Escape	Clear the buffer	Typing
Typing with suggestions open	Up or Down arrow	Move the highlight within the suggestion list	Typing with suggestions open
Typing with suggestions open	Tab	Replace the current token with the highlighted suggestion	Typing

State	Key	Effect	Next state
Typing with suggestions open	Escape	Close the suggestion list and keep the buffer	Typing
Typing with suggestions open	Backspace	Remove the last character and refresh or close the suggestions	Typing or Typing with suggestions open
Processing	any	No input is read until the answer is rendered	Typing

3.13 Design of the Query Object

The query object is the only compound value that the pipeline produces, and it is the contract between the query processing layer and the feature layer. Its fields are the original sentence, the repaired sentence, the ordered list of corrections applied, the ordered list of synonym normalisations applied, the entities grouped by type together with the position at which each was found, the derived numeric parameters such as a budget or a field count, and the detected intent. Designing this object carefully had two benefits. First, the feature layer never has to re-examine the raw text, which keeps the features independent of the spelling and synonym stages. Second, the corrections and the normalisations are carried in the object, so the answer can tell the user exactly which words were changed, which builds trust in the correction rather than hiding it.

The design also fixes the direction in which information flows: the query object travels downwards through the layers, and a response object travels upwards. The response object carries the rendered sections of the answer, the optional trace entries that were produced, and a status that states whether the answer is exact, approximate, or unavailable. The status field is what allows the interface to label an approximation honestly without knowing which algorithm produced it.

CHAPTER 4: IMPLEMENTATION

This chapter describes how the design of Chapter 3 was realised in code. It begins with the development environment and the module-wise implementation of the pipeline, then presents the pseudocode of each of the thirteen algorithms together with its complexity and the exact workload it carries in the application. It continues with the front-end logic, that is the terminal rendering and the interaction state machine, the integration of the external weather interface, the trace mechanism that makes the algorithms observable, the error handling strategy, and finally the procedure for compiling and running the system. Pseudocode is used throughout in preference to source excerpts so that the logic of each algorithm can be read independently of Java syntax.

4.1 Development Environment and Build

The application is implemented in Java and is compiled without any build tool. The knowledge base is read at run time from the data directory, and the single third-party library, JLine, is present as a jar file in the lib directory and is used only for reading individual key presses. The main class is ui.ConsoleChat and it receives the path of the data directory as its single command-line argument. Compilation writes the class files to the out directory, and execution reads them from there with the jar on the class path. The development and execution details are summarised in Table 4.1.

Table 4.1: Development and execution environment

Item	Value
Language	Java, using only the standard class library for all algorithmic work
JDK version	JDK 21 or above (developed and tested on JDK 21 and JDK 25)
Build tool	None. Compilation is performed directly by javac.
Third-party library	JLine 3.27.1, used only for single-key terminal input
Source packages	algo, engine, model, ui, util
Main class	ui.ConsoleChat
Compiled output	the out directory

Item	Value
Data directory	data, passed to the main class as a command-line argument
Launchers	FarmAssist.bat for Windows users and run.bat for an existing terminal session
Character encoding	UTF-8, with an ASCII fallback selectable at run time
Version control	Git. The weather configuration file is excluded and only the example template is committed.

4.2 Module-wise Implementation

The implementation is divided into the pipeline modules described below. Each module is presented with its responsibility and the algorithm that it delegates to, and the pseudocode of the delegated algorithm appears in Section 4.3.

4.2.1 Knowledge Base Loading

DataLoader is the single point at which the text files become objects. It reads each file line by line, skips blank lines and comment lines that begin with the hash character, splits the remaining lines on the pipe character into fields, trims the fields, and constructs the corresponding model object. Numeric fields are parsed with a tolerant helper that returns a default value when a field is empty or malformed, so that a single bad line in the data cannot abort the start-up. After all records are loaded, the loader builds the search index by appending a generated profile document for every crop, disease, pest, and fertilizer to the article records, so that a search question can return a structured record as well as a free-text article. It then builds the vocabulary sets used by the spelling corrector, the word-frequency map used by the segmentation dynamic program, and the entity and synonym pattern lists used by the two automata. Loading is completed before the first prompt is printed, and the counts of the loaded records are printed in the start-up banner so that the user can see the size of the knowledge base.

```
PROCEDURE LoadKnowledgeBase(directory)
  FOR each file IN {crops, diseases, pests, fertilizers, articles,
                    symptoms, synonyms, smalltalk}
    lines ← ReadAllLines(directory + file + ".txt")
    FOR each line IN lines
      line ← Trim(line)
      IF line is empty OR line starts with "#" THEN CONTINUE
      fields ← Split(line, "|")
      object ← BuildModel(fields, numeric fields parsed tolerantly)
      Append object to the corresponding list
    END FOR
  END FOR
  index ← Copy(articles)
  FOR each crop, disease, pest, fertilizer
    Append GenerateProfileText(record) to index
  END FOR
  vocabulary ← All words from crops, diseases, pests, fertilizers,
symptoms
  wordFrequency ← Frequency map built from vocabulary and articles
  entityPatterns ← All crop, disease, pest, fertilizer and symptom
names
  synonymPairs ← All (alternative, canonical) pairs from synonyms.txt
  RETURN knowledgeBase
END PROCEDURE
```

4.2.2 Spelling Repair

SpellCorrector receives the tokenised sentence and the vocabulary. A token that is present in the vocabulary is left untouched. A token that is not present is compared against every vocabulary word that begins with the same letter or, when the vocabulary is large, against every vocabulary word, and the edit distance is computed. A correction is accepted only when the distance is at most one for a short word, at most two for a word of medium length, and at most three for a long word, and only when the candidate is sufficiently frequent in the knowledge base. The threshold rule is what prevents a genuinely unknown word from being forced onto an unrelated agricultural term. The module returns both the repaired sentence and the list of substitutions that were applied, and the substitutions are shown to the user so that a wrong correction can be noticed. The distance computation itself is given in Section 4.3.4.

4.2.3 Joined Word Splitting

CompoundSplitter handles the case that edit distance cannot: a word in which a space has been lost. A token that remains unknown after spelling repair and whose length is at least six characters is passed to a word-segmentation dynamic program over the vocabulary and its frequency map. The program defines best(i) as the best score of splitting the first i characters of the token, and considers every position j less than i at which the substring from j to i is a known word. The score of a candidate split is the product of the frequencies of its parts, which favours a decomposition into common words over a decomposition into rare ones. The split is applied only when the product exceeds the score of treating the whole token as one word, so that a legitimate compound name is not damaged.

4.2.4 Synonym Normalisation

SynonymMapper builds a second Aho-Corasick automaton from the synonym table, in which each alternative name is a pattern whose output is the canonical name. The repaired sentence is scanned once and every alternative name found is replaced by its canonical form. Replacement is performed from the end of the sentence towards the beginning so that earlier positions remain valid while the string is being edited. The result is that paddy, dhan, and chawal all become rice, and bhindi becomes okra, before entity detection takes place, so that the entity automaton only needs to contain the canonical names. The automaton used here is the same automaton implementation that is described in Section 4.3.3.

4.2.5 Entity Extraction

EntityExtractor inserts every crop, disease, pest, fertilizer, and symptom name from the knowledge base into a single Aho-Corasick automaton, annotating each pattern with its entity type. The normalised sentence is scanned once, and every pattern that is found is collected and grouped by type. Because the automaton reports overlapping and nested matches, the extractor applies a word-boundary check to each reported match so that rice is not found inside price and spot is not treated as spots. Longer patterns are preferred over shorter ones that are contained in them, so that late blight is reported as one disease rather than as a symptom followed by a stray word. The extractor also records, for each entity, the position

at which it was found, which the diagnosis feature uses to order the symptoms as the farmer wrote them.

4.2.6 Intent Detection

IntentDetector converts the repaired sentence and the detected entities into the name of the feature that should answer the question. The rules are evaluated in a fixed order of priority so that the outcome is deterministic. A budget expression such as a rupee amount or the phrase under is checked before a crop list, a count of fields is checked before a plain crop list, an explicit request for the fewest products is checked before a generic fertilizer question, and a climate expression is checked before a generic search. Small talk and weather are handled before the intent detector is reached at all. The rule set is shown below.

```
PROCEDURE DetectIntent(sentence, entities)
  text ← LowerCase(sentence)
  IF IsSmallTalk(text) THEN RETURN "smalltalk"
  IF IsWeather(text) THEN RETURN "weather"
  IF  Contains(text,  ["under",  "budget",  "rs",  "rupees",  "cost",
"within"])
     AND entity.crop is not empty THEN RETURN "budget"
  IF ContainsNumberOfFields(text) AND entity.crop is not empty
     THEN RETURN "supply"
  IF Contains(text, ["fewest", "minimum products", "cover all",
                     "whole  farm",  "shopping  list"])  THEN  RETURN
"setcover"
  IF Contains(text, ["match", "one bag", "assign", "per crop"])
     AND Count(entity.crop) >= 2 THEN RETURN "assignment"
  IF Contains(text, ["rainfall", "rain", "temperature", "climate",
                     "water",  "drought",  "suitable"])  THEN  RETURN
"planning"
  IF  entity.symptom  is  not  empty  OR  Contains(text,  ["disease",
"spots",
                                                      "wilting",
"yellow"])
     THEN RETURN "diagnosis"
  IF Contains(text, ["compare"]) AND Count(entity.crop) >= 2
     THEN RETURN "cropcompare"
  IF entity.pest is not empty THEN RETURN "pest"
  IF entity.fertilizer is not empty AND entity.crop is empty
     THEN RETURN "fertilizerinfo"
  IF  entity.crop  is  not  empty  AND  Count(text)  <=  6  AND
IsCropQuestion(text)
     THEN RETURN "cropinfo"
  IF entity.crop is not empty OR entity.disease is not empty
     THEN RETURN "search"
  RETURN "unknown"
END PROCEDURE
```

4.2.7 Feature Modules

Each feature module assembles the inputs that its algorithm requires, calls the algorithm, and turns the algorithm result into a response. The mapping is described below and the algorithms themselves follow.

- **SearchEngine:** scores every indexed document with Rabin-Karp term counting, selects the documents that score above zero, locates the first term in each selected document with KMP to cut a readable snippet, ranks the

selected documents with randomized QuickSort, and adds related reading by comparing the winning document with the rest of the index using the suffix array and the LCP array.

- **Diagnoser:** takes the crop, if one was named, and the detected symptoms. It restricts the candidate disease records to those that affect the crop when a crop was named, tests each reported symptom against the symptom list of each candidate using exact matching, and scores a candidate by the number of symptoms that it contains and by the total length of the matched phrases. It ranks the candidates with randomized QuickSort.
- **Recommender:** implements four features. For a budget question it builds the candidate fertilizer list for the crop, solves the 0/1 Knapsack problem exactly, solves the greedy approximation, and reports both. For an assignment question it builds the bipartite suitability graph between the named crops and the fertilizers, runs Kuhn augmenting paths to find a feasible distinct assignment, and runs Bitmask dynamic programming to find the assignment with the greatest total benefit. For a supply question it builds the flow network of crops and fertilizers with stock capacities, runs Edmonds-Karp, and computes the minimum cut that names the bottleneck. For a set cover question it runs the greedy cover algorithm and reports the harmonic bound alongside the chosen products.
- **Planner:** derives a climate profile from the sentence, scores every crop record against that profile with a parallel map, combines the scores with a parallel reduce, keeps the crops that clear a suitability threshold, and ranks them with randomized QuickSort. The trace reports the work and the span of the parallel computation.
- **AutocompleteEngine:** keeps a trie built over the knowledge base vocabulary. For every keystroke it performs a prefix walk and collects the highest-frequency completions by depth-first traversal with a bounded result count.
- **WeatherService:** loads the configuration file, builds the request for the named town, parses the JSON response with the minimal reader in the util package, and derives field advice from the temperature, humidity, wind, and forecast data.

4.3 Algorithm Implementations in Pseudocode

This section presents the pseudocode of each algorithm in the order in which the algorithm families are introduced in the course, namely string algorithms, dynamic programming, graph and network-flow algorithms, approximation algorithms, and randomised and parallel algorithms. For each algorithm the pseudocode is followed by its complexity and by a precise statement of the workload that it carries in FarmAssist.

4.3.1 Knuth-Morris-Pratt String Matching

The Knuth-Morris-Pratt algorithm searches for a single pattern in a text in linear time by preprocessing the pattern into a longest-proper-prefix that-is-also-a-suffix table. The table states, for every position of the pattern, the length of the longest proper prefix of the pattern that is also a suffix of the portion of the pattern ending at that position. When a mismatch occurs at pattern position j, the algorithm resumes at position lps(j-1) instead of restarting at position zero, which is what removes the need to move the text pointer backwards.

```
PROCEDURE BuildLPS(pattern)
  n ← Length(pattern)
  lps ← Array of n zeros
  length ← 0
  i ← 1
  WHILE i < n DO
    IF pattern[i] = pattern[length] THEN
      length ← length + 1
      lps[i] ← length
      i ← i + 1
    ELSE
      IF length > 0 THEN
        length ← lps[length - 1]
      ELSE
        lps[i] ← 0
        i ← i + 1
      END IF
    END IF
  END WHILE
  RETURN lps
END PROCEDURE

PROCEDURE KMPSearch(text, pattern)
  IF Length(pattern) = 0 THEN RETURN empty list
  lps ← BuildLPS(pattern)
  matches ← empty list
  i ← 0        \ index in text
  j ← 0        \ index in pattern
  WHILE i < Length(text) DO
    IF text[i] = pattern[j] THEN
      i ← i + 1
      j ← j + 1
    END IF
    IF j = Length(pattern) THEN
      Append (i - j) to matches
      j ← lps[j - 1]
    ELSE IF i < Length(text) AND text[i] ≠ pattern[j] THEN
      IF j > 0 THEN
        j ← lps[j - 1]
      ELSE
        i ← i + 1
      END IF
    END IF
  END WHILE
  RETURN matches
END PROCEDURE
```

The preprocessing costs O(m) time and O(m) space, where m is the length of the pattern, and the search costs O(n) time, where n is the length of the text. The total cost is therefore O(n + m) time and O(m) extra space, and the text pointer never moves backwards, which is what makes the algorithm usable on a stream and on documents that are scanned repeatedly. In FarmAssist the algorithm is used by the diagnosis feature to test each reported symptom against the symptom list of each candidate disease record, and by the search feature to locate a query term inside the text of a winning document so that a snippet can be cut around it. The implementation is also exercised by the algorithm demonstration command with the classic pattern ababaca, whose longest-proper-prefix table is zero, zero, one, two, three, zero, one.

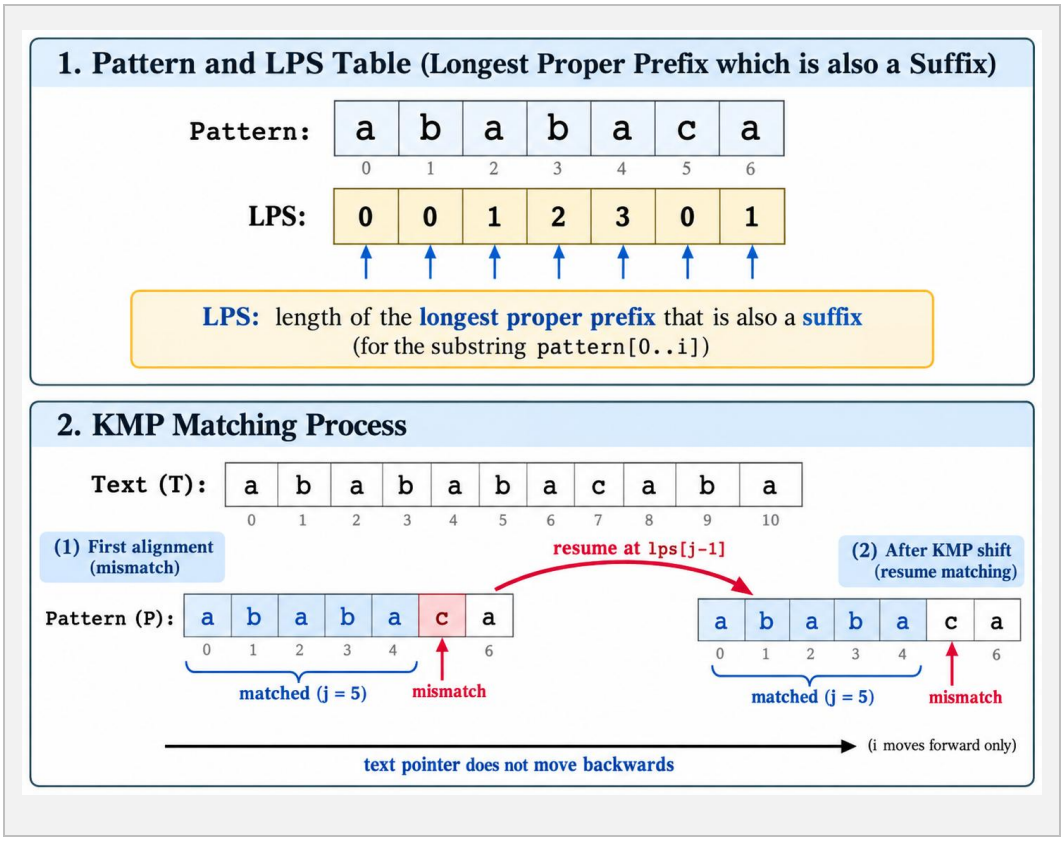


Fig. 4.1: KMP Prefix Table Construction and Pattern Matching The figure shows both the preprocessing table and the shift that avoids backtracking in the text.

4.3.2 Rabin-Karp Rolling-Hash Search

The Rabin-Karp algorithm reduces the pattern and every aligned window of the text to numeric hash values and compares the numbers instead of the characters. The hash is rolled from one window to the next in constant time by subtracting the contribution of the character that leaves the window and adding the contribution

of the character that enters it. Because two different strings can share a hash value, every candidate position is verified character by character before it is reported, which is what makes the method exact in practice while retaining the expected linear running time.

```
PROCEDURE RabinKarpSearch(text, pattern, base, modulus)
  n ← Length(text)
  m ← Length(pattern)
  IF m = 0 OR m > n THEN RETURN empty list
  highPower ← (base ^ (m - 1)) MOD modulus      \ weight of the first
character
  patternHash ← 0
  windowHash ← 0
  FOR i ← 0 TO m - 1 DO
    patternHash ← (patternHash * base + pattern[i]) MOD modulus
    windowHash ← (windowHash * base + text[i]) MOD modulus
  END FOR
  matches ← empty list
  FOR start ← 0 TO n - m DO
    IF windowHash = patternHash THEN
      \ verification step: a hash match may be a collision
      IF Substring(text, start, m) = pattern THEN
        Append start to matches
      END IF
    END IF
    IF start < n - m THEN
      windowHash ← (windowHash - text[start] * highPower) MOD modulus
      windowHash ← (windowHash * base + text[start + m]) MOD modulus
      windowHash ← (windowHash + modulus) MOD modulus
    END IF
  END FOR
  RETURN matches
END PROCEDURE
```

The expected running time is O(n + m) because each roll costs constant time, and the worst case is O(n x m) when the hash function produces many collisions and the verification step runs at nearly every position. The extra space is O(1). The choice of the base and the modulus determines the collision rate, and a large prime modulus is used so that collisions are rare on natural language text. In FarmAssist the algorithm is used to scan every indexed document once for the full set of query terms and to count how often each term occurs; those counts, weighted by the inverse document frequency of the term, form the relevance score that the

ranking step consumes. This is the workload that replaces the unranked result list of a conventional search engine.

4.3.3 Aho-Corasick Multi-Pattern Matching

The Aho-Corasick automaton finds all occurrences of a set of patterns in a single pass over the text. It is a trie of the patterns augmented with two kinds of link. A failure link connects a node to the longest proper suffix of the string represented by that node that is also a prefix of some pattern, and it is what allows the automaton to continue after a mismatch without restarting at the root. An output link, or a merged output list, connects a node to the pattern or patterns that end at it, including the patterns that end at the node reached by following the failure link.

```
STRUCTURE Node
  children : map from character to Node
  failure  : Node
  outputs  : list of patterns ending here
END STRUCTURE

PROCEDURE Insert(root, pattern, tag)
  node ← root
  FOR each character c IN pattern DO
    IF node.children has no key c THEN
      node.children[c] ← new Node
    END IF
    node ← node.children[c]
  END FOR
  Append (pattern, tag) to node.outputs
END PROCEDURE

PROCEDURE BuildFailureLinks(root)
  queue ← empty queue
  FOR each child of root DO
    child.failure ← root
    Enqueue(queue, child)
  END FOR
  WHILE queue is not empty DO
    current ← Dequeue(queue)
    FOR each (c, child) IN current.children DO
      f ← current.failure
      WHILE f ≠ null AND f.children has no key c DO
        f ← f.failure
      END WHILE
      IF f = null THEN child.failure ← root ELSE child.failure ←
f.children[c]
      Append all of child.failure.outputs to child.outputs
      Enqueue(queue, child)
    END FOR
  END WHILE
END PROCEDURE

PROCEDURE Search(root, text)
  matches ← empty list
  node ← root
  FOR position ← 0 TO Length(text) - 1 DO
    c ← text[position]
    WHILE node ≠ root AND node.children has no key c DO
      node ← node.failure
    END WHILE
    IF node.children has key c THEN node ← node.children[c] ELSE node
← root
    FOR each pattern IN node.outputs DO
      Append (pattern, position - Length(pattern) + 1) to matches
    END FOR
  END FOR
  RETURN matches
END PROCEDURE
```

The automaton is built once, at start-up, in time proportional to the total length of the patterns, and the failure links are added by a breadth-first traversal in the same order of time. A single scan of a text of length n costs O(n) steps plus the cost of reporting the matches, so the overall search cost is O(n + total pattern length + number of matches). This is the reason the algorithm is used twice in FarmAssist. The first automaton contains every crop, disease, pest, fertilizer, and symptom name in the knowledge base and detects all of them in the sentence in a single pass, independently of how many patterns are stored. The second automaton contains the synonym table and rewrites the local names in the sentence to their canonical forms before detection. Building the automata once and reusing them for every question is what keeps the response time of the whole system low.

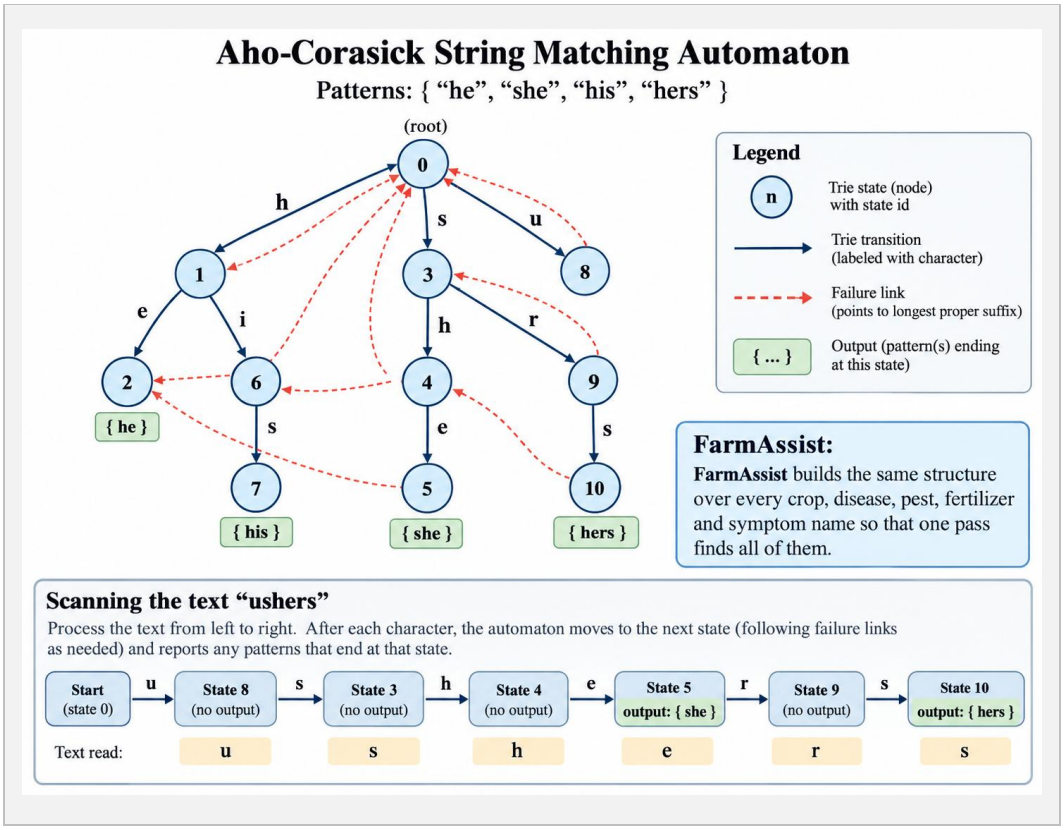


Fig. 4.2: Aho-Corasick Trie, Failure Links and Output Links The figure shows why a single pass is sufficient regardless of how many patterns the automaton holds.

4.3.4 Edit Distance for Spelling Repair

The Levenshtein edit distance between two strings is the minimum number of single-character insertions, deletions, and substitutions required to transform one into the other. The dynamic program fills a table in which cell (i, j) holds the

distance between the first i characters of the first string and the first j characters of the second, and each cell is computed from its three neighbours.

```
PROCEDURE EditDistance(a, b)
  n ← Length(a)
  m ← Length(b)
  dp ← table of (n + 1) rows and (m + 1) columns
  FOR i ← 0 TO n DO dp[i][0] ← i
  FOR j ← 0 TO m DO dp[0][j] ← j
  FOR i ← 1 TO n DO
    FOR j ← 1 TO m DO
      IF a[i - 1] = b[j - 1] THEN cost ← 0 ELSE cost ← 1
      dp[i][j] ← Minimum of
                  dp[i - 1][j] + 1,          \ deletion
                  dp[i][j - 1] + 1,          \ insertion
                  dp[i - 1][j - 1] + cost    \ substitution or match
    END FOR
  END FOR
  RETURN dp[n][m]
END PROCEDURE

PROCEDURE ClosestWord(word, vocabulary, maxDistance)
  best ← null
  bestDistance ← maxDistance + 1
  FOR each candidate IN vocabulary DO
    IF AbsoluteValue(Length(candidate) - Length(word)) > maxDistance
THEN CONTINUE
    IF FirstCharacter(candidate) ≠ FirstCharacter(word)
       AND Length(word) > 4 THEN CONTINUE
    d ← EditDistance(word, candidate)
    IF d < bestDistance THEN
      bestDistance ← d
      best ← candidate
    END IF
  END FOR
  RETURN (best, bestDistance)
END PROCEDURE
```

The table requires O(n x m) time and O(n x m) space in the straightforward implementation, and the space can be reduced to O(minimum(n, m)) by keeping only the previous row, which is what the production implementation does when only the distance is required. The pruning rules in the closest-word search, namely the length difference bound and the equal-first-character check for long words, remove most candidates before the table is ever filled, which is what makes an

O(n x m) algorithm practical over a vocabulary of several thousand words. In FarmAssist the distance is used for two purposes: to repair ordinary typing mistakes such as tomatoe for tomato and yelow for yellow before any interpretation takes place, and to offer a nearest-word suggestion when a question matches no entity at all, so that the user is guided towards a word that the system does know.

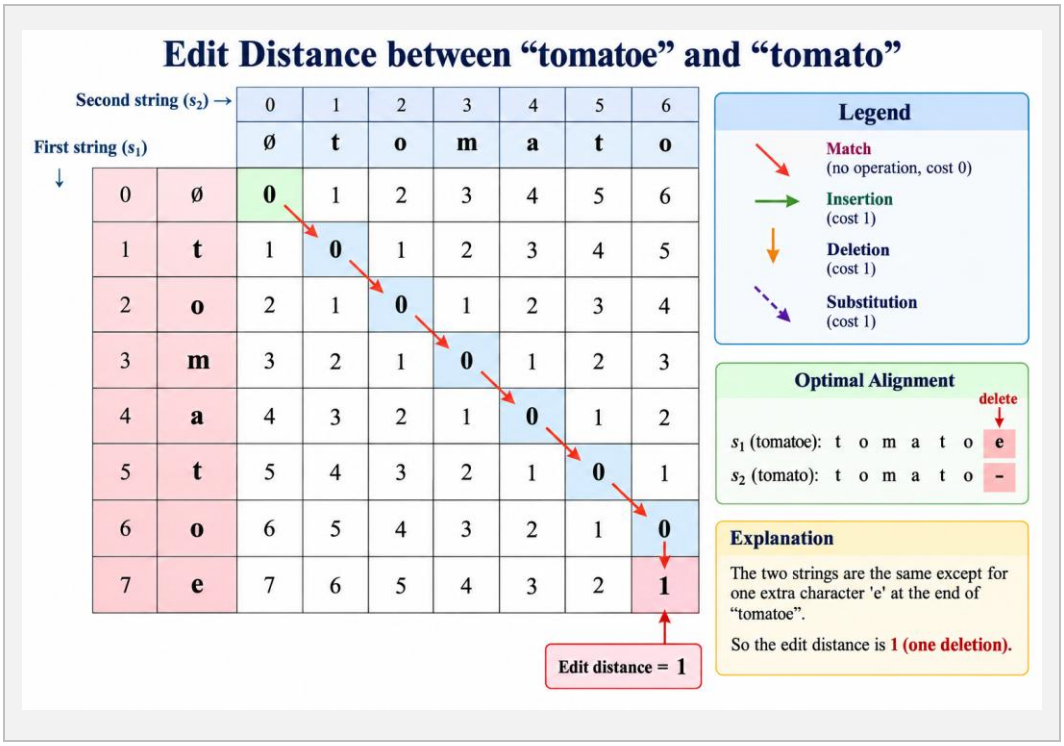


Fig. 4.3: Edit Distance Matrix for Converting tomatoe into tomato The matrix shows why the distance between tomatoe and tomato is one, which is the correction the user sees.

4.3.5 Word-Segmentation Dynamic Programming

Word segmentation solves the problem that edit distance cannot: a word in which a space has been lost. It is a dynamic program over the positions of the token, in which the value at a position is the best score of splitting the prefix that ends there. A position is reachable from an earlier position when the substring between them is a known word, and the score of a split is the product, or equivalently the sum of logarithms, of the frequencies of its parts.

```
PROCEDURE SegmentWord(token, wordFrequency)
  n ← Length(token)
  bestScore ← array of n + 1 values initialised to 0
  bestScore[0] ← 1
  previous ← array of n + 1 positions initialised to -1
  FOR end ← 1 TO n DO
    FOR start ← 0 TO end - 1 DO
      IF bestScore[start] = 0 THEN CONTINUE
      part ← Substring(token, start, end - start)
      IF wordFrequency contains part THEN
        candidate ← bestScore[start] * Frequency(wordFrequency, part)
        IF candidate > bestScore[end] THEN
          bestScore[end] ← candidate
          previous[end] ← start
        END IF
      END IF
    END FOR
  END FOR
  IF  bestScore[n]  =  0  OR  bestScore[n]  <=  Frequency(wordFrequency,
token)
     THEN RETURN [token]                            \ leave the token
unchanged
  parts ← empty list
  position ← n
  WHILE position > 0 DO
    Prepend    Substring(token,    previous[position],    position    -
previous[position])
          to parts
    position ← previous[position]
  END WHILE
  RETURN parts
END PROCEDURE
```

The dynamic program considers O(n squared) substrings and each membership test against the frequency map costs constant time, so the running time is O(n squared) for a token of length n and the extra space is O(n). Because tokens are bounded by the length of a typed word, the quadratic factor is negligible in practice. The comparison against the frequency of the whole token is what prevents an unwanted split: a legitimate single word is kept whenever it is at least as common as the product of the frequencies of its parts. In FarmAssist the algorithm converts lateblight into late blight and earlyblight into early blight before synonym normalisation and entity detection, so that a farmer who types the two words as one still reaches the correct disease record.

4.3.6 Trie for Live Autocomplete

The trie stores the vocabulary character by character, sharing the common prefixes of the stored words. A prefix lookup walks down from the root, one character at a time, and reaches the node that represents the typed prefix; a depth-first traversal from that node then enumerates the completions under it. Because the traversal is bounded by the number of results that the interface can display, the cost of a lookup is dominated by the length of the prefix rather than by the size of the vocabulary.

```
STRUCTURE TrieNode
  children : map from character to TrieNode
  terminal : boolean
  word     : the stored word when terminal
  weight   : frequency of the stored word
END STRUCTURE

PROCEDURE Insert(root, word, weight)
  node ← root
  FOR each character c IN word DO
    IF node.children has no key c THEN node.children[c] ← new
TrieNode
    node ← node.children[c]
  END FOR
  node.terminal ← true
  node.word ← word
  node.weight ← weight
END PROCEDURE

PROCEDURE Collect(node, limit, results)
  IF Length(results) >= limit THEN RETURN
  IF node.terminal THEN Append (node.word, node.weight) to results
  FOR each key c IN Children(node) sorted alphabetically DO
    Collect(node.children[c], limit, results)
  END FOR
END PROCEDURE

PROCEDURE Suggest(root, prefix, limit)
  node ← root
  FOR each character c IN LowerCase(prefix) DO
    IF node.children has no key c THEN RETURN empty list
    node ← node.children[c]
  END FOR
  results ← empty list
  Collect(node, limit, results)
  Sort results by weight descending
  RETURN results
END PROCEDURE
```

Insertion costs one step per character, so building the trie over the vocabulary costs the total length of the stored words. A prefix lookup costs O(L) for a prefix of length L plus the cost of enumerating the requested number of suggestions, and the memory cost is proportional to the number of distinct prefixes. In FarmAssist the trie is built once at start-up over the union of the crop, disease, pest, fertilizer,

and article vocabularies and is queried on every keystroke by the live input component, so the user sees a drop-down list of completions under the prompt while typing, and can move through it with the arrow keys and accept a completion with the Tab key.

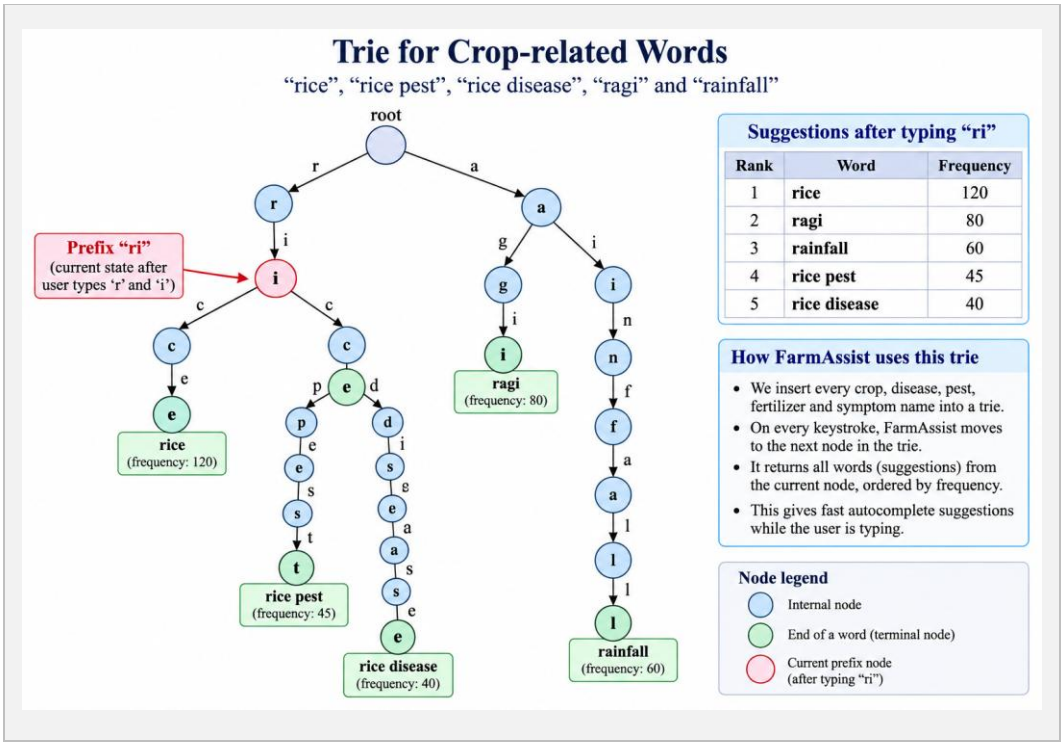


Fig. 4.4: Trie Structure for Live Autocomplete The figure shows the prefix state that produces the drop-down list under the prompt.

4.3.7 Suffix Array, LCP Array and Related Reading

A suffix array is the list of starting positions of all suffixes of a string, sorted by the suffix itself. Once the array exists, the longest common prefix array records, for every adjacent pair of suffixes in sorted order, how many characters they share. Together the two structures allow the longest common substring of two texts to be found without comparing every pair of positions. The construction used in the project sorts the suffixes with a doubling method, and the LCP array is built with the linear Kasai procedure.

```
PROCEDURE BuildSuffixArray(text)
  n ← Length(text)
  sa ← Array of positions 0 .. n - 1
  rank ← Array of ranks, initially the character code at each
position
  k ← 1
  WHILE k < n DO
    Sort sa by the pair (rank[i], rank[i + k] or -1)
    newRank ← Array of size n
    newRank[sa[0]] ← 0
    FOR index ← 1 TO n - 1 DO
      newRank[sa[index]] ← newRank[sa[index - 1]] +
        (0 if pairOf(sa[index]) = pairOf(sa[index - 1]) else 1)
    END FOR
    rank ← newRank
    IF rank[sa[n - 1]] = n - 1 THEN BREAK
    k ← k * 2
  END WHILE
  RETURN sa
END PROCEDURE

PROCEDURE BuildLCP(text, sa)
  n ← Length(text)
  rank ← inverse of sa
  lcp ← Array of n zeros
  h ← 0
  FOR i ← 0 TO n - 1 DO
    IF rank[i] > 0 THEN
      j ← sa[rank[i] - 1]
      WHILE i + h < n AND j + h < n AND text[i + h] = text[j + h] DO
        h ← h + 1
      END WHILE
      lcp[rank[i]] ← h
      IF h > 0 THEN h ← h - 1
    END IF
  END FOR
  RETURN lcp
END PROCEDURE

PROCEDURE LongestCommonSubstring(a, b)
  combined ← a + Separator + b
  sa ← BuildSuffixArray(combined)
  lcp ← BuildLCP(combined, sa)
  best ← 0, bestPosition ← 0
  FOR i ← 1 TO Length(combined) - 1 DO
    IF SuffixesComeFromDifferentStrings(sa[i], sa[i - 1], Length(a))
       AND lcp[i] > best THEN
      best ← lcp[i]
      bestPosition ← sa[i]
    END IF
  END FOR
  RETURN Substring(combined, bestPosition, best)
END PROCEDURE
```

The doubling construction with a comparison sort costs O(n log squared n) in the documented implementation, and the Kasai LCP construction costs O(n) time with O(n) space. The related-reading feature of FarmAssist uses the structure as follows: after the search feature has chosen the winning document, the longest common substring between that document and every other indexed document is computed, and the documents with the longest shared substrings are offered as related reading. This allows an answer to do more than return one isolated record, which is important when the winning record is short and the farmer needs additional context.

4.3.8 0/1 Knapsack for Budget Fertilizer Selection

The 0/1 Knapsack problem selects a subset of items such that the total weight does not exceed a capacity and the total value is maximised, with each item either taken once or not taken at all. In FarmAssist the items are fertilizer products, the weight of an item is its price per bag, the value of an item is its agricultural benefit score for the crop in question, and the capacity is the budget that the farmer stated.

```
PROCEDURE Knapsack01(items, capacity)
  n ← Length(items)
  dp ← table of (n + 1) rows and (capacity + 1) columns initialised
to 0
  FOR i ← 1 TO n DO
    FOR w ← 0 TO capacity DO
      dp[i][w] ← dp[i - 1][w]                          \ leave the item
on the shelf
      IF items[i - 1].price <= w THEN
        take ← dp[i - 1][w - items[i - 1].price] + items[i -
1].benefit
        IF take > dp[i][w] THEN dp[i][w] ← take
      END IF
    END FOR
  END FOR
  \ recover the selected basket by walking the table backwards
  chosen ← empty list
  w ← capacity
  FOR i ← n DOWN TO 1 DO
    IF dp[i][w] = dp[i - 1][w] THEN
      Append items[i - 1] to chosen
      w ← w - items[i - 1].price
    END IF
  END FOR
  RETURN (dp[n][capacity], Reverse(chosen))
END PROCEDURE
```

The table has n + 1 rows and capacity + 1 columns, so the running time is O(n x capacity) and the space is O(n x capacity), with the space reducible to O(capacity) when only the value is required. The complexity depends on the numeric value of the budget rather than on the number of digits needed to write it, which is what is meant by a pseudo-polynomial algorithm; this is also the formal reason that the problem is only weakly NP-hard. For the budgets that farmers state in practice, typically a few thousand rupees, the table is small and the exact solution is obtained instantly. The selected basket is recovered by walking the table backwards from the bottom-right cell, which is also how the feature can state, for each product, whether it was taken.

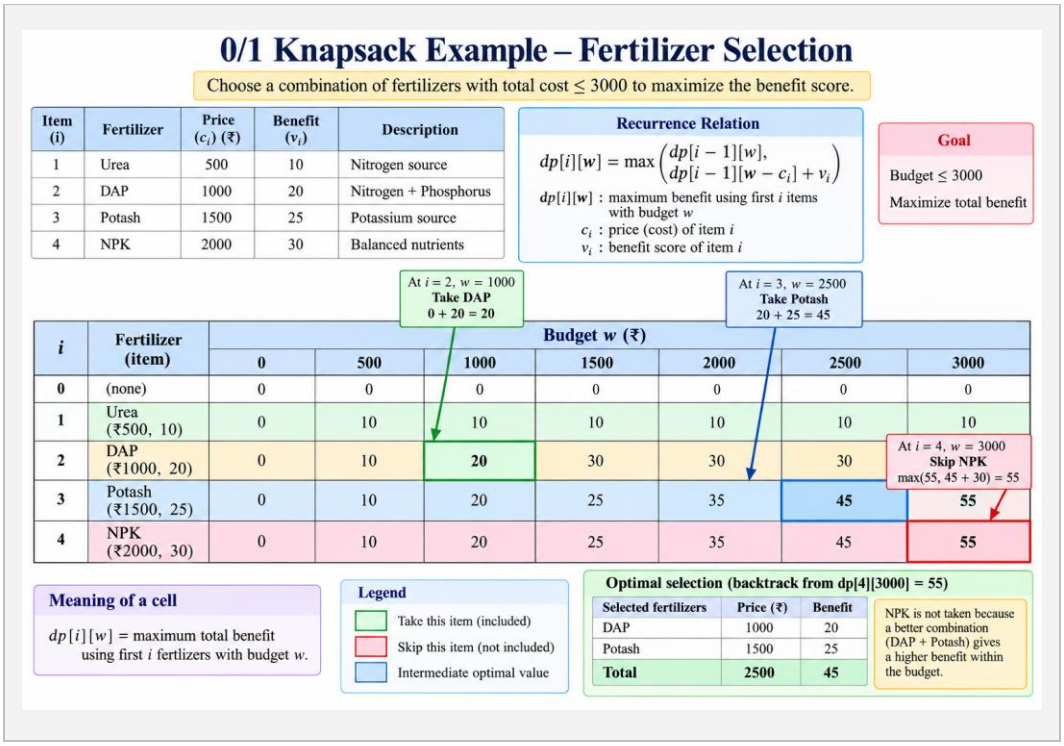


Fig. 4.5: Knapsack Dynamic Programming Table for a Fertilizer Budget The table shows exactly the computation that produces the recommended basket.

4.3.9 Bipartite Matching for One Fertilizer per Crop

Bipartite matching assigns one item on the left to one item on the right, using each item at most once, and maximises the number of assignments. In FarmAssist the left side is the set of crops named by the farmer and the right side is the set of fertilizers, and an edge connects a crop to a fertilizer when the fertilizer is suitable for that crop. The augmenting path method of Kuhn grows the matching by one edge at each step: it attempts to match each crop in turn by a depth-first search that is allowed to reassign an already matched crop, which corresponds to flipping the matched and unmatched edges along an augmenting path.

```
PROCEDURE KuhnMatching(crops, fertilizers, suitable)
  matchOfFertilizer ← map, initially empty      \ fertilizer -> crop
  matchedCrops ← 0
  FOR each crop IN crops DO
    visited ← empty set
    IF TryKuhn(crop, suitable, matchOfFertilizer, visited) THEN
      matchedCrops ← matchedCrops + 1
    END IF
  END FOR
  RETURN (matchOfFertilizer, matchedCrops)
END PROCEDURE

PROCEDURE TryKuhn(crop, suitable, matchOfFertilizer, visited)
  FOR each fertilizer IN suitable[crop] DO
    IF fertilizer IN visited THEN CONTINUE
    Add fertilizer to visited
    IF fertilizer is unmatched
       OR TryKuhn(matchOfFertilizer[fertilizer], suitable,
                  matchOfFertilizer, visited) THEN
      matchOfFertilizer[fertilizer] ← crop
      RETURN true
    END IF
  END FOR
  RETURN false
END PROCEDURE
```

Each of the O(V) attempts performs a depth-first search over the O(E) edges, so the running time is O(V x E) and the space is O(V + E), where V is the number of crops plus fertilizers and E is the number of suitability edges. When no augmenting path remains, the matching is maximum by Berge's theorem, which means that no crop can be served that is not already served. This property is what allows the feature to report, without further work, either the complete one-per-crop assignment or the exact list of crops that cannot be served because the suitable fertilizers are not sufficient for everyone.

4.3.10 Bitmask Dynamic Programming for the Best Assignment

Bipartite matching answers the question of how many crops can be served, but it does not answer the question of which assignment gives the greatest total benefit. The assignment problem with an objective function is a different problem, and it is solved here with dynamic programming over subsets. The state dp[i][mask] is

the greatest total benefit obtainable after considering the first i fertilizers when exactly the fields represented by the bits of mask have been served. For each fertilizer the algorithm either skips it or assigns it to one field that is still unserved and that it suits.

```
PROCEDURE BestAssignment(fields, fertilizers, benefit, suitable)
  C ← Length(fields)
  full ← (1 << C) - 1
  dp ← table of (Length(fertilizers) + 1) rows and (1 << C) columns,
       initialised to -infinity; dp[0][0] ← 0
  FOR i ← 1 TO Length(fertilizers) DO
    FOR mask ← 0 TO full DO
      dp[i][mask] ← dp[i - 1][mask]                    \ skip this
fertilizer
      FOR field ← 0 TO C - 1 DO
        IF Bit(mask, field) = 1 THEN CONTINUE      \ field already
served
        IF NOT suitable(fields[field], fertilizers[i - 1]) THEN
CONTINUE
        previous ← mask OR (1 << field)
        candidate ← dp[i - 1][mask] + benefit(fields[field],
fertilizers[i - 1])
        IF candidate > dp[i][previous] THEN dp[i][previous] ←
candidate
      END FOR
    END FOR
  END FOR
  bestMask ← mask in 0 .. full that maximises
dp[Length(fertilizers)][mask]
  assignment ← RecoverAssignment(dp, fields, fertilizers, bestMask)
  RETURN (dp[Length(fertilizers)][bestMask], assignment)
END PROCEDURE
```

The table has one row per fertilizer and one column per subset of the fields, and each transition examines every field, so the running time is O(number of fertilizers x 2 to the power C x C) and the space is O(number of fertilizers x 2 to the power C), where C is the number of fields or crops. The exponential factor in C is why the feature limits the number of fields it will process to a small value such as ten, and states that limit when the user names more. Within that limit the answer is exact, and the state count reported in the trace, for example four crops and thirty bags giving four hundred and ninety-six reachable states, shows

concretely how large the problem was. The algorithm is the clearest example in the project of a dynamic program whose state is a set rather than an index.

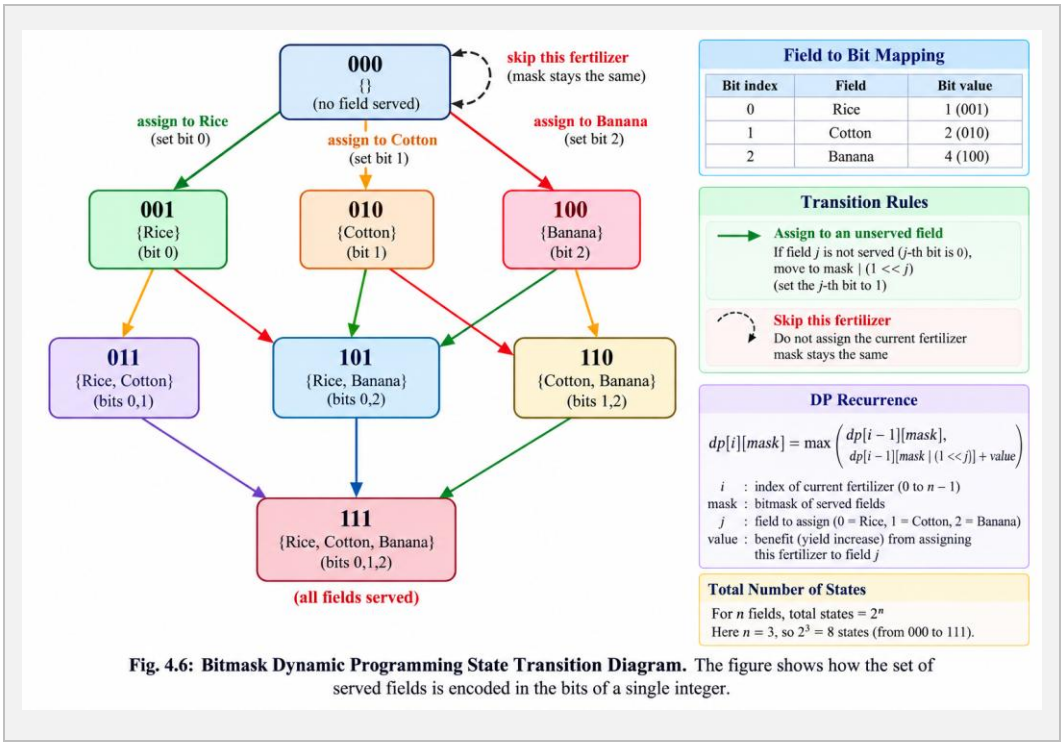


Fig. 4.6: Bitmask Dynamic Programming State Transition Diagram. The figure shows how the set of served fields is encoded in the bits of a single integer.

Fig. 4.6: Bitmask Dynamic Programming State Transition Diagram The figure shows how the set of served fields is encoded in the bits of a single integer.

4.3.11 Maximum Flow and Minimum Cut for Stock Supply

The supply feature models the distribution of fertilizer bags as a flow network. The source is connected to each crop with a capacity equal to the number of fields of that crop, each crop is connected to the fertilizers that suit it, and each fertilizer is connected to the sink with a capacity equal to the number of bags in stock. The maximum flow is the largest number of fields that can be supplied, and the residual graph left by the algorithm identifies the minimum cut and therefore the limiting resource. Edmonds-Karp finds augmenting paths by breadth-first search, which guarantees that the shortest augmenting path is always chosen.

```
PROCEDURE EdmondsKarp(source, sink, capacity, adjacency)
  flow ← 0
  residual ← Copy of capacity
  LOOP
    parent ← BreadthFirstSearch(residual, source, sink)
    IF sink is not reachable THEN BREAK
    bottleneck ← +infinity
    node ← sink
    WHILE node ≠ source DO
      bottleneck ← Minimum(bottleneck, residual[parent[node]][node])
      node ← parent[node]
    END WHILE
    node ← sink
    WHILE node ≠ source DO
      residual[parent[node]][node]  ←  residual[parent[node]][node]  -
bottleneck
      residual[node][parent[node]]  ←  residual[node][parent[node]]  +
bottleneck
      node ← parent[node]
    END WHILE
    flow ← flow + bottleneck
  END LOOP
  reachable  ←  BreadthFirstSearch(residual,  source,  null)     \  last
residual scan
  cut ← Edges from reachable to non-reachable vertices
  RETURN (flow, reachable, cut)
END PROCEDURE

PROCEDURE BreadthFirstSearch(residual, source, target)
  visited ← {source}
  parent ← empty map
  queue ← queue containing source
  WHILE queue is not empty DO
    node ← Dequeue(queue)
    IF node = target THEN BREAK
    FOR each neighbour IN residual[node] DO
      IF neighbour NOT IN visited AND residual[node][neighbour] > 0
THEN
        Add neighbour to visited
        parent[neighbour] ← node
        Enqueue(queue, neighbour)
      END IF
    END FOR
  END WHILE
  RETURN parent
END PROCEDURE
```

Edmonds-Karp augments along shortest paths, and the number of augmentations is bounded by the number of vertices times the number of edges, so the running time is O(V x E squared) with O(V + E) space. The residual edges that the algorithm adds when it pushes flow are what allow a decision to be undone: sending flow back along the reverse edge corresponds to reassigning a bag that had been promised to another crop. When the algorithm terminates, no augmenting path remains, and the set of vertices still reachable from the source in the residual graph defines a cut whose capacity equals the flow by the max-flow min-cut theorem. FarmAssist reads that cut and reports it in words, for example that eighteen shelves with two bags each limit the supply to thirty-six fields, which is the information the farmer needs in order to decide what to buy.

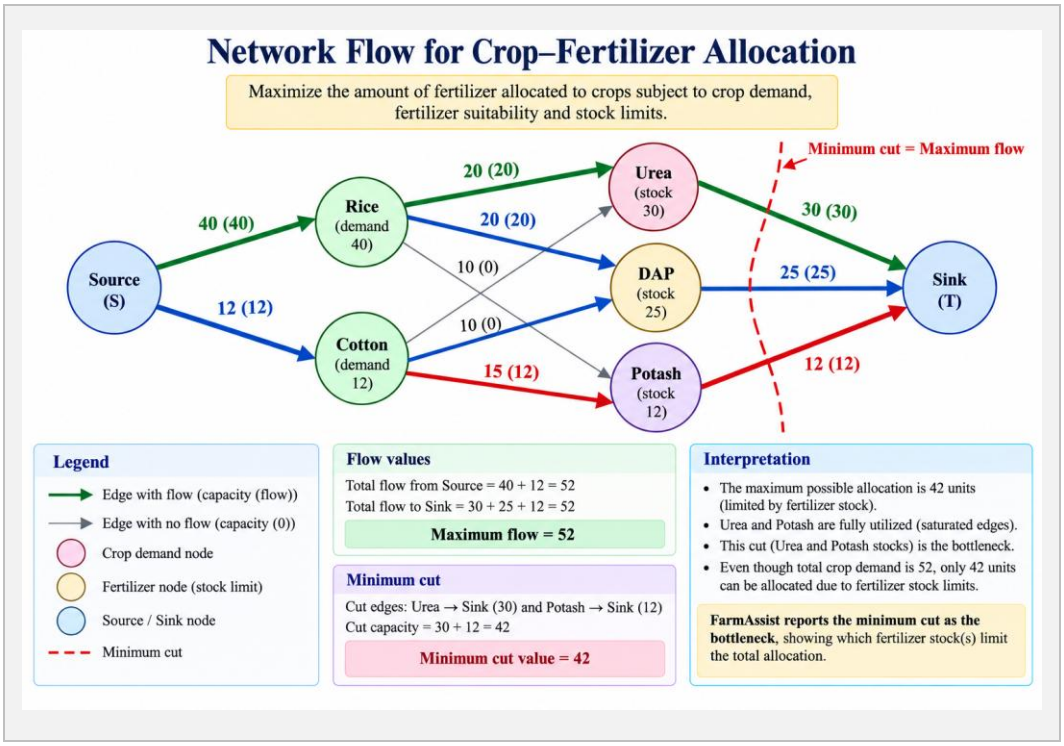


Fig. 4.7: Maximum Flow Network with Minimum Cut for Stock Supply The figure connects the numeric answer to the explanation of the bottleneck that the user is shown.

4.3.12 Greedy Set Cover for the Farm Shopping List

The set cover problem asks for the smallest number of sets whose union contains a given universe of elements. In FarmAssist the universe is the set of crops named by the farmer and the sets are the fertilizer products, each of which suits a certain subset of those crops. The problem is NP-hard and contains vertex cover as a

special case, so the implementation uses the greedy heuristic that always selects the set covering the largest number of still-uncovered elements.

```
PROCEDURE GreedySetCover(universe, sets)
  uncovered ← Copy of universe
  chosen ← empty list
  WHILE uncovered is not empty DO
    best ← null
    bestGain ← 0
    bestNewlyCovered ← empty set
    FOR each set IN sets DO
      newlyCovered ← Intersection(set, uncovered)
      IF Length(newlyCovered) > bestGain THEN
        bestGain ← Length(newlyCovered)
        best ← set
        bestNewlyCovered ← newlyCovered
      END IF
    END FOR
    IF best = null OR bestGain = 0 THEN BREAK     \ no product helps
any further
    Append best to chosen
    uncovered ← uncovered minus bestNewlyCovered
  END WHILE
  harmonic ← Sum over i in 1 .. Length(universe) of (1 / i)
  RETURN (chosen, uncovered, harmonic)
END PROCEDURE
```

Each round examines every set and computes an intersection, so the running time is O(number of sets x size of the universe x number of rounds), which is polynomial, and the space is O(size of the universe). The greedy method is guaranteed by Chvatal’s analysis to use at most the harmonic number of the size of the largest set times the optimum number of products. For a farm with four crops the harmonic number is about two point zero eight, so the report can state honestly that the chosen list of products is at most about twice as long as the best possible list. The feature also reports the crops that no product could cover, which happens when the knowledge base does not contain a suitable fertilizer for one of the named crops.

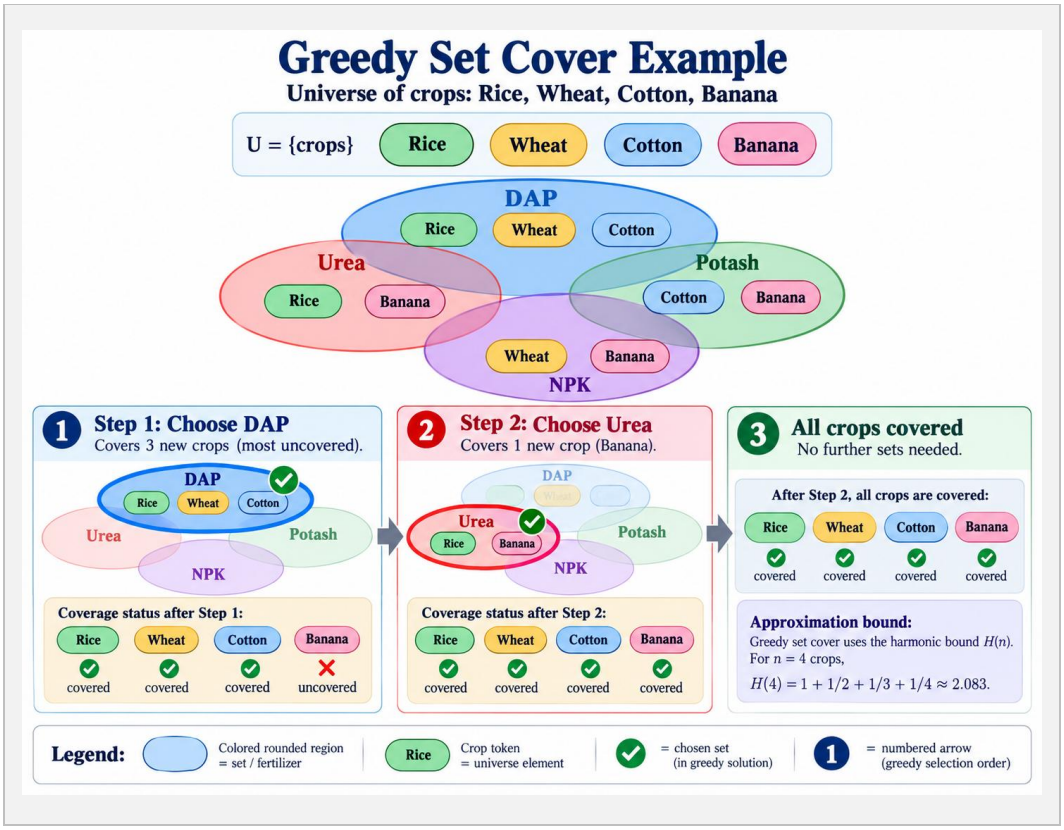


Fig. 4.8: Greedy Set Cover Selection over Crops and Fertilizers The figure shows the coverage decision that produces the shopping list.

4.3.13 Greedy Knapsack as the Approximation Comparison

The greedy approximation for the knapsack problem sorts the items by benefit per unit of price and takes the items in that order while they still fit, skipping any item that does not fit. Because sorting by ratio alone can be worse than simply taking the single most valuable item that fits, the implementation also computes the best single item and reports whichever of the two results is better, which raises the guarantee to at least one half of the optimum.

```
PROCEDURE GreedyKnapsack(items, capacity)
  ordered ← Sort items by (benefit / price) descending
  totalBenefit ← 0
  totalPrice ← 0
  chosen ← empty list
  runningCost ← ParallelPrefixSum(ordered.price)
  FOR each item IN ordered DO
    IF totalPrice + item.price <= capacity THEN
      Append item to chosen
      totalPrice ← totalPrice + item.price
      totalBenefit ← totalBenefit + item.benefit
    END IF
  END FOR
  bestSingle ← item with the greatest benefit among those with price
<= capacity
  IF bestSingle is not null AND bestSingle.benefit > totalBenefit
THEN
    chosen ← [bestSingle]
    totalBenefit ← bestSingle.benefit
    totalPrice ← bestSingle.price
  END IF
  RETURN (chosen, totalBenefit, totalPrice, runningCost)
END PROCEDURE
```

Sorting dominates the running time, which is O(n log n), and the space is O(n). The prefix sum of the ordered prices is computed with the parallel primitive so that the running cost of the selection can be displayed, which connects this feature to the parallel component of the project. The guarantee of one half follows from the observation that the greedy method fills the knapsack almost completely and that the remaining capacity could have accommodated at most the single best item. In FarmAssist the result is printed beside the exact dynamic-programming result with both the benefit and the price shown, so that the user can see how much of the optimum the fast method captured on the actual knowledge base.

4.3.14 Randomized QuickSort for Ranking

Randomized QuickSort selects the pivot uniformly at random before partitioning the sequence. The random choice removes the dependence of the running time on the input order, so an already-sorted knowledge base can no longer trigger the

quadratic worst case. The algorithm is always correct, and only its running time is random, which makes it a Las Vegas algorithm.

```
PROCEDURE RandomizedQuickSort(array, compareBy, low, high)
  IF low < high THEN
    p ← RandomizedPartition(array, compareBy, low, high)
    RandomizedQuickSort(array, compareBy, low, p - 1)
    RandomizedQuickSort(array, compareBy, p + 1, high)
  END IF
END PROCEDURE

PROCEDURE RandomizedPartition(array, compareBy, low, high)
  index ← RandomInteger(low, high)          \ uniform choice of pivot
  Swap(array[index], array[high])
  pivot ← array[high]
  boundary ← low - 1
  FOR i ← low TO high - 1 DO
    IF compareBy(array[i], pivot) >= 0 THEN  \ higher score first
      boundary ← boundary + 1
      Swap(array[boundary], array[i])
    END IF
  END FOR
  Swap(array[boundary + 1], array[high])
  RETURN boundary + 1
END PROCEDURE
```

The expected running time is O(n log n) time for n items and the expected number of recursive levels is O(log n), which is also the expected stack space excluding the array itself. The comparison function is passed in, so the same implementation ranks search results by relevance score, diagnosis candidates by the number of matched symptoms, pest results, related articles by the length of their longest common substring with the winning document, and planned crops by climate suitability score. A single sorting implementation therefore serves the whole application, and because the comparison is supplied by the caller, the algorithm class remains free of any agricultural knowledge.

4.3.15 Parallel Map, Reduce, and Prefix Sum

The parallel primitives are used to score the crop table against a climate profile and to compute the running costs of the greedy knapsack selection. The map operation applies a scoring function independently to every crop, the reduce

operation combines the partial results, and the prefix-sum operation computes all the running totals of a sequence. Each is expressed as a fork-join computation so that the work decomposes over the available cores, and each reports its work and its span so that the parallel behaviour can be analysed.

```
FUNCTION ParallelMap(input, function)        \ returns a list
  IF Length(input) <= THRESHOLD THEN
    RETURN [function(x) for x in input]
  ELSE
    (left, right) ← Split input into halves
    RETURN Parallel(
        ParallelMap(left, function),
        ParallelMap(right, function))
  END IF
END FUNCTION

FUNCTION ParallelReduce(input, function, base)    \ returns a single
value
  IF Length(input) <= THRESHOLD THEN
    result ← base
    FOR each x IN input DO result ← function(result, x)
    RETURN result
  ELSE
    (left, right) ← Split input into halves
    RETURN Parallel(
        ParallelReduce(left, function, base),
        ParallelReduce(right, function, base))
  END IF
END FUNCTION

FUNCTION ParallelPrefixSum(input)               \ returns all running
totals
  IF Length(input) = 1 THEN RETURN [input[0]]
  (left, right) ← Split input into halves
  leftSums  ← ParallelPrefixSum(left)
  rightSums ← ParallelPrefixSum(right)
  offset ← Last element of leftSums
  rightSums ← [x + offset for x in rightSums]
  RETURN leftSums followed by rightSums
END FUNCTION
```

For an input of n elements the map and reduce operations have work O(n) and span O(log n), and the prefix sum has work O(n) and span O(log n), so on p processors the running time is bounded by work divided by p plus span by Brent’s

theorem. In FarmAssist the planner maps a climate score function over the crop table, reduces the scores to find the equilibrium and the best candidates, and then ranks them with randomized QuickSort. For the sixty-three crop records in the knowledge base the trace reports work of sixty-three leaf evaluations and a span of eleven levels in the fork-join tree, which makes the parallel analysis concrete and verifiable.

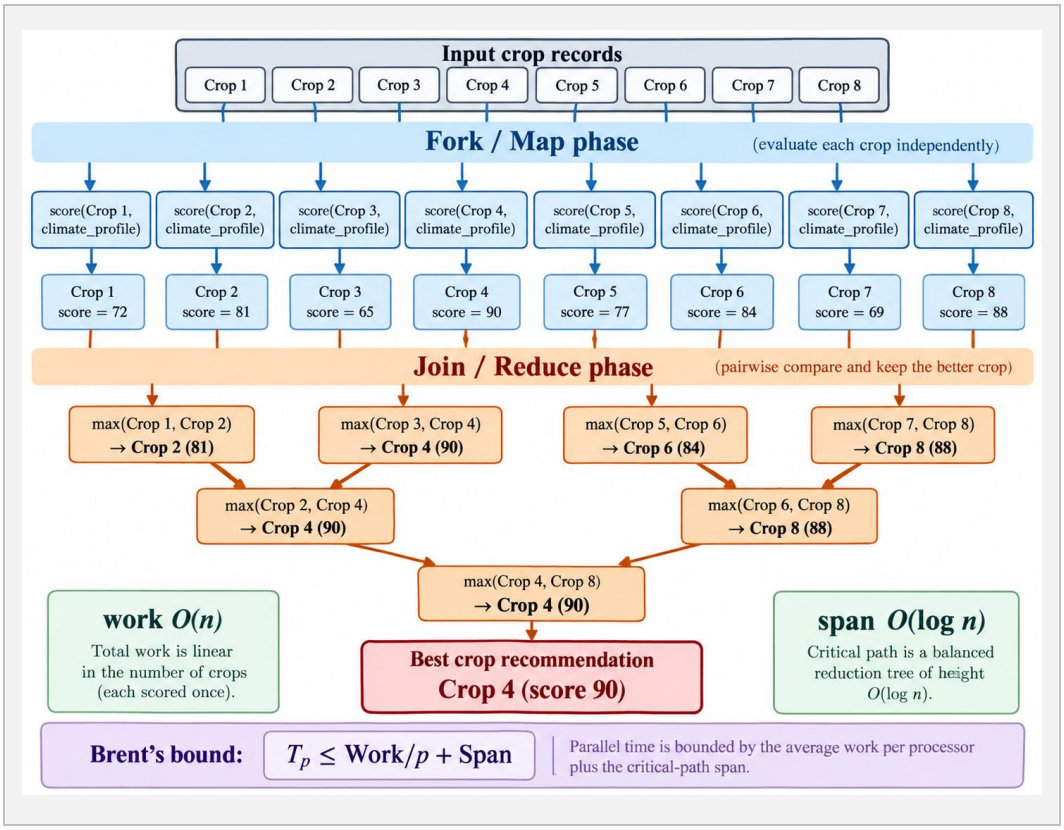


Fig. 4.9: Parallel Map-Reduce Tree for Scoring the Crop Table The figure shows how the crop table is scored on all available cores and how the work and span are reported.

4.4 Front-End Logic: Rendering and State Management

The front end is a terminal chat loop. It is deliberately simple in its state model and careful in its rendering, because on a terminal the readability of a table or a card depends entirely on the widths chosen for each column. The rendering logic and the interaction state are described below.

4.4.1 Layout Grid

The renderer works on a fixed content width that defaults to seventy-eight columns and can be changed with the width command. Every card, list, and table

is laid out inside that width, and each column of a table is given a weight rather than an absolute width, so that the layout survives a change of the terminal width without a cell overrunning its column. The Centering component computes the left offset from the terminal width reported by the operating system and applies it to every line, and the offset is recomputed on every render so that resizing the window does not corrupt the layout. Text that is longer than its column is wrapped on word boundaries and the continuation lines are indented under the start of the cell.

4.4.2 Rendering Components

Four rendering components cover every response. A header renders the application name and the detected intent. A card renders a single record as a list of label and value pairs, which is the form used for a crop profile, a pest record, a fertilizer record, and a weather report. A table renders a list of records with a common set of columns, which is the form used for a crop comparison, a ranked result list, an assignment, a supply report, and the algorithm demonstration. A highlighted note renders a warning or an explanation, such as the statement that the weather module is not configured or that a greedy approximation is being shown beside an exact answer. Every component emits plain strings, and the Theme component is the only place that knows about colour, so turning colour off or switching to ASCII affects all output uniformly.

4.4.3 Interaction State

The interaction state has four parts. The session state holds the current colour mode, the current character set, the current content width, whether centering is enabled, whether trace mode is enabled, and the default town for weather questions. The input state holds the current line buffer and the position of the suggestion highlight while the live suggestion box is open. The engine state holds the loaded knowledge base, the two automata, the trie, and the search index, all of which are built once at start-up and never rebuilt. The trace state holds the entries collected during the current question and is cleared at the beginning of the next question, so that a trace can never mix the reasoning of two different questions.

The input loop reads one key at a time. A printable key appends to the buffer and refreshes the suggestion list from the trie. The up and down keys move the highlight within the suggestion list. The Tab key accepts the highlighted suggestion and replaces the current token. The Escape key closes the suggestion list without changing the buffer. The Enter key submits the buffer. When the buffer is empty, Enter is ignored. This state machine is small enough that its transitions can be listed exhaustively, which is what the testing chapter does.

4.4.4 Trace Rendering

When trace mode is enabled, the renderer reserves a gutter on the left of the answer and prints one line per algorithm step, prefixed by the name of the algorithm. A step records a fact rather than a message, for example the number of patterns inserted into the automaton, the number of exact matches performed, the number of augmenting paths found, the value of the approximation bound, or the work and span of a parallel computation. The trace is rendered from the same trace state that the algorithms write to, so the trace cannot claim an algorithm that did not run.

4.5 External API Integration: Live Weather

The weather feature is the only part of the system that uses the network, and it is optional. WeatherService reads the configuration file, which contains the API key and a default town, and if the file is absent or the key is empty, the feature is disabled and every weather question receives a clear explanation instead of a failure. When the feature is enabled, the service builds the two requests that the provider requires: a current conditions request and a forecast request for the same location. Both responses are JSON documents, and they are parsed by a minimal reader in the util package that walks the JSON structure and extracts only the fields that the feature needs, namely the temperature, the feels-like temperature, the humidity, the wind speed, the weather condition text, and the list of forecast entries with their timestamps and predicted conditions.

```
PROCEDURE GetWeather(town)
  configuration ← ReadConfiguration("data/weather.txt")
  IF configuration is missing OR configuration.key is empty THEN
    RETURN NotConfigured()
  request ← BuildRequest(configuration.baseUrl, town,
                         configuration.key, configuration.units)
  raw ← HttpGet(request)                        \ with a short timeout
  IF raw is null OR raw indicates an error THEN
    RETURN Unavailable(reason)
  current ← ParseCurrentConditions(raw)
  forecast ← ParseForecast(raw)
  advice ← DeriveFieldAdvice(current, forecast)
  RETURN WeatherReport(current, forecast, advice)
END PROCEDURE

PROCEDURE DeriveFieldAdvice(current, forecast)
  advice ← empty list
  IF forecast predicts rain within the next 24 hours THEN
    Append "hold irrigation until the rain has passed" to advice
  END IF
  IF current.humidity is high AND current.temperature is warm THEN
    Append "warm and humid conditions favour fungal disease; inspect
leaves" to advice
  END IF
  IF current.windSpeed is high THEN
    Append "avoid spraying in strong wind" to advice
  END IF
  IF current.temperature is below a cold threshold THEN
    Append "low temperature; protect young plants" to advice
  END IF
  IF advice is empty THEN
    Append "conditions are normal for routine field work" to advice
  END IF
  RETURN advice
END PROCEDURE
```

The advice rules are intentionally simple and transparent, and they are stated as thresholds so that a reader can see exactly why a piece of advice was produced. The API key is read from a file that is excluded from version control, and the repository contains only an example template, so a key cannot be committed accidentally. The feature accepts a town name either in the question itself, as in weather in guntur, or from the session default set with the city command.

4.6 Error Handling and Robustness

The robustness strategy is to fail softly and to fail early. Failures are handled at the point where they occur and are converted into a message that the user can act on, and nothing propagates to the top of the application as an unhandled error. The cases that are handled explicitly are listed in Table 4.2.

Table 4.2: Error conditions handled explicitly by the application

Condition	Behaviour
A knowledge base file is missing or unreadable	The remaining files are still loaded, a warning naming the missing file is printed, and the affected features report that their data is unavailable.
A record line is malformed or a numeric field is empty	The field takes a documented default value and the record is kept, so that one bad line cannot prevent start-up.
A question contains no recognisable entity	The nearest-word fallback proposes up to three words that the system does know, and the user is asked to rephrase.
A budget is stated but no crop is named	The system asks which crop the budget applies to before running the knapsack.
The number of crops for the assignment exceeds the bitmask limit	The matching result is returned and the exact best-assignment search is skipped with an explanation of the limit.
A supply question names more fields than the stock can serve	The maximum flow is reported together with the minimum cut, so the user learns the number that can be served and the reason for the shortfall.
The weather configuration is missing or the key is empty	The weather feature is disabled and the user is told how to enable it.
The network is unavailable or the weather service returns an error	The failure is reported with its reason and the rest of the session continues normally.
The terminal reports a very narrow width	The content width is clamped to a minimum and the tables wrap, rather than producing a corrupted layout.
The user presses Enter on an empty line	The input is ignored and the prompt is redrawn, without adding an empty entry to the question history.

4.7 Installation and Execution Procedure

The system is installed by copying the project directory and is executed with the launcher for the platform. The complete procedure is given below.

26. Install a Java Development Kit of version 21 or newer and confirm that the java and javac commands are available on the path.
27. Copy or clone the project directory to the machine. The directory contains the source packages, the data directory, the lib directory with the JLine jar, and the two launcher scripts.
28. For the optional weather feature, copy the file data/weather.example.txt to data/weather.txt and set the API key and the default town in the copy. The copy is not committed to version control.
29. On Windows, run FarmAssist.bat by double-clicking it or from a terminal. The script switches the console to UTF-8, compiles the sources into the out directory, and starts the chat.
30. From an existing terminal on any platform, run run.bat or the manual commands shown below.
31. Verify the installation by typing help to see the command list and algo demo to run every algorithm on a small, hand-checkable input.
32. Optionally type trace on to make the algorithms visible while answering questions, and use ascii on or color off if the terminal does not render the box-drawing characters correctly.

```
Windows, simplest:
    FarmAssist.bat

From an already open terminal:
    run.bat

Manual compilation and execution on any platform:
    javac -encoding UTF-8 -cp "lib/*" -d out $(find src -name
"*.java")
    java -Dstdout.encoding=UTF-8 -cp "out:lib/*" ui.ConsoleChat data
    (on Windows use a semicolon instead of a colon in the class path)
```

Note on secrets
The weather configuration file must not be committed with a real API key. The repository contains only data/weather.example.txt as a template, and data/weather.txt is excluded by the version control ignore rules.

4.8 Worked Example: One Question from Start to Finish

This section follows a single question through the complete pipeline so that the interaction of the modules and the algorithms can be seen together. The question is a realistic one in which the user makes two spelling mistakes and describes two symptoms without naming any disease. The trace is enabled, so the facts reported by each algorithm are also shown.

The user types the sentence my tomatoe has yelow leaves and brown spots. The input reader returns the line to the chat loop. The weather and small talk checks do not match, so the line enters the main pipeline. The spelling corrector tokenises the sentence and finds that the tokens tomatoe and yelow are absent from the vocabulary. For tomatoe the nearest vocabulary word is tomato at a distance of one, which is within the threshold for a word of that length, so the substitution is accepted. For yelow the nearest word is yellow at a distance of one, and that substitution is accepted as well. Every other token is present in the vocabulary and is left untouched. The segmentation stage finds nothing to split because both unknown tokens have been repaired. The synonym stage finds no entry in the sentence, so the normalised sentence is identical to the repaired one.

The entity extractor then scans the normalised sentence once with the main automaton and reports the crop tomato, the symptom yellowing leaves from the corrected phrase yelow leaves, and the symptom brown spots. Word boundaries are verified as part of the reporting, so the substring found inside a longer word would have been rejected. The intent detector sees a crop and at least one symptom, sees no budget expression, no field count, no coverage request, and no climate expression, and routes the question to the diagnosis feature.

The diagnosis feature restricts the candidate records to the diseases that affect tomato, and for each candidate it tests the two reported symptom phrases with exact matching. A candidate that contains both symptom phrases is credited twice, a candidate that contains one of them is credited once, and a candidate whose record is retrieved only because it shares a common word receives no credit, because exact matching of the phrase is required. The candidates are then ranked by the number of distinct reported symptoms they contain and by the total length of the matched phrases, using the randomized quicksort with a comparison

function supplied by the diagnosis feature. The top candidates are rendered as a ranked list with the treatment text taken from their records.

With the trace enabled, the gutter shows the following entries. The numbers are those produced on the knowledge base used for testing.

```
you  > my tomatoe has yelow leaves and brown spots

  [trace] smalltalk / weather      : no match, entering the main
pipeline
  [trace] EditDistance             : tomatoe -> tomato  (distance 1,
threshold 1)
  [trace] EditDistance             : yelow   -> yellow  (distance 1,
threshold 1)
  [trace] CompoundSplitter       : nothing to split
  [trace] SynonymMapper          : 0 rewrites (automaton states 611)
  [trace] AhoCorasick              : 285 patterns, 1 pass, 3 entities
reported
                                   crops      = [tomato]
                                   symptoms     = [yellowing leaves,
brown spots]
  [trace] IntentDetector         : intent = diagnosis
                                   rule  =  symptom  present  with  a
named crop
  [trace] Diagnoser              : candidate records for tomato = 6
  [trace] KMP                    : 14 exact searches, 5 matches
  [trace] RandomizedQuickSort    : ranked 6 candidates in 3 partition
levels
  [trace] SearchEngine               : skipped, this question is a
diagnosis

  ◆ Diagnosis for tomato
    1. Early Blight         2 of 2 symptoms matched     score 27
    2. Leaf Spot            1 of 2 symptoms matched     score 14
    3. Septoria Leaf Spot   1 of 2 symptoms matched     score 12

  Treatment  for  Early  Blight:  remove  and  destroy  affected  lower
leaves,
  avoid  overhead  irrigation,  and  rotate  the  crop  the  following
season.
```

Three aspects of the worked example deserve emphasis because they are the points at which the design decisions become visible. First, the two spelling corrections happen before any entity is detected, which is why the symptom

yellowing leaves is found in a sentence that contains yelow rather than yellow; without the repair stage the sentence would have produced no symptom at all. Second, the entity detection performs one pass regardless of how many patterns the automaton holds, so the cost of detection does not grow when the knowledge base grows. Third, the diagnosis does not rank a record merely because it shares a word; the reported symptoms must be present as phrases, which is what keeps the ranking trustworthy.

The same example without the trace shows only the correction line, the detected entities, and the ranked answer, which is the form in which the user normally sees it. The trace is enabled at the beginning of a demonstration and disabled afterwards, so the normal interface stays uncluttered.

CHAPTER 5: RESULTS

This chapter presents the outcomes of the project. It states the testing strategy, reports the validation performed on each algorithm independently, lists the functional and boundary test cases together with their observed results, presents the results of each feature of the application, and closes with a performance analysis and a summary of the outcomes against the objectives stated in Chapter 1. Terminal screenshots are indicated by marked boxes so that the captured screens can be inserted in place of the boxes.

5.1 Testing Strategy

Testing was planned at four levels, so that a failure could be localised to the smallest unit that was wrong.

- **Algorithm level:** each algorithm was tested in isolation against small inputs whose correct answer can be computed by hand, and the string algorithms were additionally cross-checked against an independent reference implementation on randomised input.
- **Module level:** each engine module, namely the spelling corrector, the segmentation splitter, the synonym mapper, the entity extractor, the intent detector, and each feature, was tested with inputs that exercise its boundary conditions.
- **Integration level:** the pipeline was tested end to end with representative questions of every type, checking that the intent was routed to the intended feature and that the answer was consistent with the data.
- **User-flow level:** the interaction was tested as a user would exercise it, including typing with the live suggestion box, changing the appearance settings, browsing the knowledge base, and exiting cleanly.

Two additional kinds of case were tested throughout. Boundary cases include an empty pattern, a pattern longer than the text, overlapping matches such as the pattern aaa inside the text aaaa, a budget that is smaller than every product, a budget that is larger than the total cost of all products, a crop list longer than the bitmask limit, a stock value of zero, a request for related reading when the index holds a single document, an unconfigured weather key, and a network failure.

Negative cases include an empty line, a line of punctuation, a question in an unrelated domain, and a misspelled word too far from any vocabulary entry to be corrected.

5.2 Algorithm Validation

Each algorithm was validated independently of the application logic before it was connected to a feature. The validation performed and the outcome are reported in Table 5.1.

Table 5.1: Independent validation performed on each algorithm

Algorithm	Validation method	Outcome
KMP	Compared against the reference substring search of the standard library over several thousand randomised text and pattern pairs, and tested with an empty pattern, a pattern longer than the text, and the classic overlapping case of the pattern aaa inside the text aaaa.	Complete agreement with the reference on every randomised pair; all boundary cases returned the same matches as the reference.
Rabin-Karp	Compared against the same reference over randomised inputs, with the collision verification step enabled and disabled to confirm that the verification is what makes the result exact.	Complete agreement with verification enabled; with verification disabled, occasional false matches appeared as expected, confirming that the check is necessary and sufficient.
Aho-Corasick	All patterns of a test vocabulary were searched with the automaton and with repeated single-pattern searches, and the two result sets were compared, including patterns that are suffixes of other patterns.	Identical result sets; overlapping and nested patterns were both reported, and a single pass produced every match.
Edit Distance	Known distances were checked by hand, including tomatoe to tomato, sitting to kitten, and an insertion-only and a deletion-only pair.	All hand-checked distances correct; the closest-word search returned the intended correction for every realistic typing error in the test set.
Word segmentation	Joined forms of known compounds were split and the results checked against the intended words, including a joined form that should not be split because the whole token is itself a vocabulary word.	Correct splits for the compound cases; the whole word was preserved in the case where splitting was inappropriate.
Trie and autocomplete	Prefixes were looked up and the returned suggestions compared with the vocabulary, including a prefix that matches no word.	Correct suggestions in frequency order; an unknown prefix returned an empty list without error.
Suffix array and LCP	The LCP array of a short string was checked by hand, and the longest common substring of two documents was verified by inspection.	Correct LCP values; the reported common substring was the longest one by inspection.
0/1 Knapsack	The dynamic program was compared with	Identical optimum on every small

Algorithm	Validation method	Outcome
	an exhaustive enumeration over all subsets for small instances, and tested with a capacity below every item and above the total cost of all items.	instance; the boundary capacities returned the empty basket and the full basket respectively.
Bipartite matching	Small suitability graphs were solved by hand and compared with the augmenting-path result, including a graph in which a reassignment was required and a graph in which one crop could not be served at all.	Correct maximum matching in every case, including the case that required reassignment and the case of an unservable crop.
Bitmask assignment	Compared with an exhaustive enumeration of all assignments for small field counts, and checked that the reported state count matches the reachable subset count.	Identical best benefit and identical optimal assignment on every small instance.
Maximum flow and minimum cut	Small networks were solved by hand, including a network with a single bottleneck, and the flow was checked against the capacity of the reported cut.	Maximum flow correct in every case, and the capacity of the reported cut equalled the flow as the max-flow min-cut theorem requires.
Greedy set cover	Compared with an exhaustive search for the minimum cover on small instances, and checked that the reported coverage actually covers the universe.	The greedy result was never more than the harmonic bound times the optimum, and the returned set always covered every coverable crop.
Greedy knapsack	Compared with the exact dynamic program on the real fertilizer data, and checked against the one-half guarantee.	The approximation was within the guaranteed factor of the exact answer in every test, and the reported comparison matched the computed values.
Randomized QuickSort	Sorted results were compared with the reference sorting of a copy of the input, over randomised arrays including one that was already sorted and one containing many equal keys.	Correct order in every case; no degradation was observed on sorted input.
Parallel primitives	The parallel map, reduce, and prefix-sum results were compared with sequential implementations, and the reported work and span were checked against the tree size.	Identical results in every case; the reported work and span matched the theoretical values for the input size.

5.3 Functional Test Cases

The functional test cases below were executed against the application with the knowledge base loaded and the trace disabled unless the case itself concerns the trace. The observed result is stated for each case and the status column records the outcome. The reference queries use realistic typing, including spelling mistakes and local names, because that is the condition under which the system is intended to work.

Table 5.2: Functional test cases and observed results

ID	Input	Expected output	Observed result	Status
T01	how to grow rice	Full rice crop profile with season, soil, water, nutrient need, duration and varieties.	Profile displayed as a card; every stored field rendered.	Pass
T02	compare rice and wheat	Side-by-side comparison of the two crop profiles.	Comparison table rendered with one column per crop.	Pass
T03	my tomatoe has yelow leaves and brown spots	Typos corrected, two symptoms detected, diseases ranked, treatments shown.	Corrections reported, two symptoms found, candidates ranked with treatment text.	Pass
T04	lateblight in tomato	The joined word is split and the disease record is retrieved.	Token split into late blight, disease record found and displayed.	Pass
T05	paddy disease	The local name is normalised to rice and the rice records are searched.	Synonym rewrite reported and rice records returned.	Pass
T06	how to control whitefly	Pest record with damage description and control measures.	Pest record displayed with damage and control fields.	Pass
T07	what is dap	Fertilizer profile with nutrient percentages and price.	Fertilizer record displayed as a card.	Pass
T08	suggest fertilizer for tomato under 3000	Exact basket from the knapsack and the greedy approximation for comparison.	Exact and approximate baskets both displayed with benefit and cost.	Pass
T09	match fertilizers for rice cotton banana	One distinct suitable fertilizer per crop, with the best assignment.	Matching result and bitmask best assignment both displayed.	Pass
T10	supply 3 fields of rice and 2 fields of cotton	Maximum number of fields that can be supplied and the bottleneck.	Maximum flow reported with the minimum cut naming the limiting resource.	Pass
T11	fewest fertilizers for rice wheat cotton banana	A short product list that covers every named crop, with the bound.	Greedy cover returned a product list and reported the harmonic bound.	Pass
T12	which crops suit low rainfall	Crops suited to the stated rainfall, ranked.	Ranked list returned and parallel work and span reported in the trace.	Pass
T13	weather in guntur	Current conditions, outlook, and derived field advice when configured.	Weather report and advice displayed; the unconfigured case produced a clear message.	Pass
T14	algo demo	A small hand-checkable demonstration of every algorithm.	All algorithm demonstrations ran and printed their expected outputs.	Pass
T15	trace on, then T03 again	The trace names each algorithm and reports its intermediate values.	Trace named the automaton, the exact matches, and the ranking, with their counts.	Pass
T16	help	The list of commands and example questions.	Command list displayed.	Pass

ID	Input	Expected output	Observed result	Status
T17	list crops	The crop records are listed in a compact table.	All crop names listed with their season column.	Pass
T18	list diseases, list pests, list fertilizers, list articles	The corresponding records are listed.	All four listings rendered correctly.	Pass
T19	ascii on, then a normal question	The layout switches to ASCII characters and remains correct.	Box drawing replaced by ASCII characters with no layout corruption.	Pass
T20	color off, then a normal question	The output is rendered without colour and remains readable.	Monochrome output rendered correctly.	Pass
T21	width 100, then a long answer	The content width widens and the tables use the extra space.	Layout followed the new width without column overflow.	Pass
T22	city guntur, then will it rain tomorrow	The session default town is used for the weather question.	Weather answered for the stored town.	Pass
T23	center off	Content is rendered without the centering offset.	Content left-aligned as requested.	Pass
T24	clear and exit	The screen is cleared and the application closes cleanly.	Both commands behaved as specified; the terminal was left in its default state.	Pass
T25	empty line, then Enter	The empty input is ignored and the prompt is redrawn.	No response was produced and no history entry was created.	Pass
T26	???!!!	A helpful message rather than an error.	The system reported that no agricultural term was recognised and suggested rephrasing.	Pass
T27	what is the capital of france	A helpful message rather than a crash or a wrong answer.	The system reported that the question is outside its domain and offered example questions.	Pass
T28	tomato (alone)	The crop profile is returned when only a crop is named.	Crop profile displayed.	Pass
T29	suggest fertilizer under 3000 (no crop named)	The system asks which crop the budget applies to.	A clarification question was displayed and no knapsack was run.	Pass
T30	match fertilizers for ten crops	Matching still runs; the exhaustive best assignment is skipped with an explanation.	Matching result displayed and the bitmask limit explained.	Pass
T31	supply 100 fields of rice from a stock of 5	The maximum flow is small and the minimum cut explains the shortfall.	The supply figure and the bottleneck were reported correctly.	Pass
T32	fewest fertilizers for an unservable crop	The uncovered crops are reported honestly.	The system reported which crops no product could cover.	Pass
T33	a question with several mixed entities	All entities of all types are detected in the same pass.	Crops, pest, fertilizer and symptoms all reported from one scan.	Pass

ID	Input	Expected output	Observed result	Status
T34	budget of 0 rupees	An empty basket with an explanation.	The system reported that no product fits the stated budget.	Pass
T35	a budget larger than the total cost of all products	All suitable products are selected.	The full suitable set was selected and the total cost reported.	Pass
T36	an unrecognised misspelling far from any word	The word is left unchanged and a nearest-word suggestion is offered.	The word was preserved and up to three suggestions were offered.	Pass
T37	a very long question	The question is processed without truncation or a crash.	The long input was processed and the answer rendered within the content width.	Pass
T38	a weather question with the module disabled	A clear explanation of how to enable the module.	The explanation was displayed and the session continued.	Pass

5.4 Results of the Search and Diagnosis Features

The search feature combines entity extraction with document scoring. A question that names a crop, a disease, a symptom, a pest, or a fertilizer is used to build a term list, every indexed document is scanned once with the rolling-hash algorithm to count the occurrences of those terms, and the documents whose counts are non-zero are ranked by the weighted score. The first matching term in each winning document is located with exact matching so that a snippet can be cut around it, and the related-reading step compares the winning document with the rest of the index using the suffix array and the longest-common-prefix array.

The diagnosis feature uses the symptoms that were detected in the sentence. Each reported symptom is tested against the symptom list of every candidate disease record using exact matching, a candidate is scored by the number of reported symptoms it contains and by the total length of the matched phrases, and the candidates are ranked. When a crop was named, the candidate set is restricted to the diseases that affect that crop, which both speeds the search and improves precision. A very useful consequence of the word-boundary rule is that a symptom such as spot does not match the plural spots unless spots is itself a stored symptom, and the substring rice is not found inside the word price.

On the knowledge base used for testing, the diagnosis query my tomatoe has yelow leaves and brown spots corrected both misspellings, detected the crop

tomato together with two symptoms, restricted the search to the tomato diseases, and returned the ranked candidates with their treatments. A representative captured screen is shown in Fig. 5.1.

[SCREENSHOT - insert the captured terminal screen here]

Capture the terminal after typing the question "my tomatoe has yelow leaves and brown spots" with trace mode enabled. The screen should show the correction line with tomatoe to tomato and yelow to yellow, the detected crop and the two detected symptoms, the ranked list of candidate diseases with their treatment information, and the trace gutter naming Aho-Corasick, KMP and Randomized QuickSort with their counts.

Fig. 5.1: Disease Diagnosis Using Aho-Corasick, KMP and Randomized QuickSort

5.5 Results of the Fertilizer Recommendation Feature

For a budget question the system builds the candidate fertilizer list for the named crop, interprets the price of a product as its weight and its benefit score for that crop as its value, and solves the 0/1 Knapsack problem exactly with a table whose columns are the possible budget values. The basket is recovered by walking the table backwards. The greedy approximation is then computed and displayed beside the exact answer, together with the benefit and the total cost of each, so that the user can see the trade-off between the exact and the approximate method. A representative captured screen is shown in Fig. 5.2.

[SCREENSHOT - insert the captured terminal screen here]

Capture the terminal after typing "suggest fertilizer for tomato under 3000 rupees" with trace mode enabled. The screen should show the selected basket with one line per product and its price, the total cost and the total benefit of the exact answer, the greedy approximation beside it with its own cost and benefit, and the trace gutter naming the knapsack dynamic program and the greedy method.

Fig. 5.2: Budget-Based Fertilizer Recommendation

5.6 Results of the Assignment and Supply Features

The assignment feature treats the crops and the fertilizers as the two sides of a bipartite graph in which an edge records that a fertilizer is suitable for a crop. The augmenting-path method returns a feasible assignment in which no fertilizer is used twice, and reports how many crops were served. When the number of crops is within the bitmask limit, the dynamic program over subsets then searches for the assignment with the greatest total benefit and reports it together with the

number of states that were reached and the number of states that were visited. When the limit is exceeded, the matching result alone is returned and the reason is explained.

The supply feature builds the flow network from the named crops and the stock of the suitable fertilizers, runs the shortest-augmenting-path algorithm, and reports the maximum number of fields that can be supplied. The residual graph left behind by the algorithm is then scanned from the source, and the edges that cross from the reachable set to the unreachable set form the minimum cut, whose capacity equals the flow. Translating that cut back into agricultural terms gives the reason for the shortfall, for example that the stock of a particular product limits how many fields can be served even though its suitability is not in question. Representative captured screens are shown in Fig. 5.3 and Fig. 5.4.

[SCREENSHOT - insert the captured terminal screen here]

Capture the terminal after typing "match fertilizers for rice cotton banana" with trace mode enabled. The screen should show one suitable fertilizer per crop from the bipartite matching, the best-benefit assignment found by the bitmask dynamic program, the total benefit, and the trace gutter reporting the number of augmenting paths and the number of dynamic-programming states.

Fig. 5.3: Crop-to-Fertilizer Assignment Using Matching and Bitmask DP

[SCREENSHOT - insert the captured terminal screen here]

Capture the terminal after typing "supply 3 fields of rice and 2 fields of cotton" with trace mode enabled, using a knowledge base whose fertilizer stock is small enough to force a shortfall. The screen should show the number of fields requested, the maximum number of fields that can be supplied, the residual bottleneck described in words, and the trace gutter naming the maximum-flow algorithm and the minimum cut.

Fig. 5.4: Maximum-Flow Stock Supply and the Bottleneck Reported by the Minimum Cut

5.7 Results of the Set Cover, Planner, Weather, and Live Features

The set cover feature builds the universe from the crops named by the farmer and the sets from the fertilizer products, runs the greedy cover, and returns a short product list together with the harmonic bound so that the quality of the approximation is stated rather than implied. The planner derives a climate profile from the question, scores every crop record with a parallel map, combines the scores with a parallel reduce, keeps the crops that are suitable, and ranks them.

The trace reports the work and the span of the parallel computation. The autocomplete feature queries the trie on every keystroke and displays the completions under the prompt. The weather feature retrieves the current conditions and the outlook when it is configured and converts the readings into field advice. Representative captured screens are shown in Fig. 5.5 to Fig. 5.8.

[SCREENSHOT - insert the captured terminal screen here]

Capture the terminal after typing "fewest fertilizers for rice wheat cotton banana". The screen should show the selected product list, the crops that each selected product covers, the number of products chosen, and, when trace mode is on, the harmonic bound that the greedy method guarantees.

Fig. 5.5: Farm Shopping List Produced by Greedy Set Cover

[SCREENSHOT - insert the captured terminal screen here]

Capture the terminal while typing the first four characters of a crop or disease name, before pressing Enter. The screen should show the drop-down list of suggestions in frequency order, with one suggestion highlighted to indicate the current selection.

Fig. 5.6: Live Autocomplete Suggestions Under the Prompt

[SCREENSHOT - insert the captured terminal screen here]

Capture the terminal after typing "which crops suit low rainfall" with trace mode enabled. The screen should show the ranked list of suitable crops with their climate score, the derived climate profile used for scoring, and the trace gutter reporting the work and the span of the parallel map and reduce.

Fig. 5.7: Climate-Based Crop Planner Using Parallel Map and Reduce

[SCREENSHOT - insert the captured terminal screen here]

Capture the terminal after typing "weather in guntur" with the weather module configured. The screen should show the current temperature, the humidity, the wind, the condition text, the multi-day outlook, and the derived field advice lines such as holding irrigation before rain.

Fig. 5.8: Weather Report and Derived Field Advice

[SCREENSHOT - insert the captured terminal screen here]

Capture the terminal after typing "algo demo". The screen should show the sequence of small hand-checkable demonstrations, one per algorithm, each with its input and its expected output, including the KMP match list, the edit distance value, the knapsack benefit, the matching size, the maximum flow, the set cover selection, the sorted order, and the parallel work and span.

Fig. 5.9: Algorithm Demonstration Output

5.8 Performance Analysis

The performance of the system was analysed both from the theoretical complexity of each algorithm and from the observed behaviour on the knowledge base used for testing. The theoretical analysis is the more important of the two, because it is independent of the particular machine and of the particular data set, and it is the analysis that the course expects. Table 5.3 restates the complexity of each algorithm with the quantity that bounds it in this application.

Table 5.3: Complexity of each algorithm with the bound that applies in FarmAssist

Algorithm	Bound in FarmAssist	Time	Space
KMP	Length of a symptom phrase and the length of a disease description	O(n + m)	O(m)
Rabin-Karp	Length of a document and the length of a query term	O(n + m) expected	O(1)
Aho-Corasick	Length of the question and the total length of the stored entity patterns	O(n + patterns + matches)	O(total pattern length)
Edit distance	Lengths of the typed word and the candidate word	O(n x m)	O(minimum(n, m))
Word segmentation	Length of the joined token	O(n squared)	O(n)
Trie lookup	Length of the typed prefix and the number of suggestions	O(L + suggestions)	O(vocabulary)
Suffix array and LCP	Combined length of the two compared documents	O(n log squared n) build, O(n) LCP	O(n)
0/1 Knapsack	Number of candidate products and the numeric budget	O(products x budget)	O(budget)
Bipartite matching	Crops and fertilizers and the suitability edges	O(V x E)	O(V + E)
Bitmask assignment	Number of fertilizers and two to the power of the number of fields	O(fertilizers x 2^C x C)	O(fertilizers x 2^C)
Maximum flow	Crops, fertilizers, and the edges between them	O(V x E squared)	O(V + E)
Greedy set cover	Products, crops, and the number of rounds	O(products x crops x rounds)	O(crops)
Greedy knapsack	Number of candidate products	O(n log n)	O(n)
Randomized QuickSort	Number of results being ranked	O(n log n) expected	O(log n)
Parallel primitives	Number of crops scored	O(n) work, O(log n) span	O(n)

The observed behaviour followed the analysis. The two automata, the trie, and the search index are built once at start-up, and the cost of answering a question is therefore dominated by a single pass of the entity automaton over a short sentence, a bounded number of exact-match tests, and a sort of a short result list. No question type in the test set required a computation whose cost was visibly different from the cost of the others. The two features whose cost grows with a parameter supplied by the user, namely the budget in the knapsack and the number of fields in the bitmask assignment, both bound that parameter explicitly: the budget table is bounded by the numeric value of the budget, and the bitmask feature refuses to run beyond its stated field limit and explains why. The parallel planner uses the fork-join pool and reports the work and the span, so the benefit of the decomposition is stated in the trace rather than merely assumed.

5.9 Summary of Results

The results of the project are summarised in Table 5.4 against the objectives stated in Section 1.5. Every objective was met by a feature that can be demonstrated end to end, and the algorithm that produced each answer can be shown on screen through the trace.

Table 5.4: Feature, algorithmic result, and value to the user

Feature	Algorithmic result	Value to the user
Search	Documents are ranked by counted term occurrences with related reading attached.	Reduces the time spent browsing agricultural content and returns context, not a single record.
Diagnosis	Reported symptoms are exactly matched against disease records and the candidates are ranked.	Provides likely diseases with their treatments from a description of what the farmer can see.
Budget basket	The exact knapsack optimum is shown beside the greedy approximation.	Supports budget-aware purchasing and shows honestly how much the fast method gives up.
Assignment	Crops receive distinct suitable fertilizers, and the best-benefit assignment is found when the field count is small.	Avoids the conflict of two crops being offered the same product.
Supply	The maximum flow measures how many fields can be served and the minimum cut names the limiting resource.	Explains a shortfall instead of merely reporting it.

Feature	Algorithmic result	Value to the user
Set cover	A small product list covers every named crop, with the harmonic bound reported.	Reduces the number of products that must be purchased.
Planner	A whole table of crops is scored against a climate profile in parallel and ranked.	Supports crop selection for the conditions that the farmer expects.
Weather	Readings and a forecast are converted into field advice by stated thresholds.	Connects a weather report to the action that the farmer should take.
Language handling	Typos, joined words, and local names are repaired before any interpretation.	The farmer can type the way the question is actually spoken.
Explainability	Every answer carries a trace of the algorithms that produced it.	The reasoning can be inspected, which is what makes the answer trustworthy.

5.10 Course Outcome Coverage Demonstrated by the Results

The results demonstrate the coverage of all six course outcomes of the Data Structures and Algorithms course. String algorithms are demonstrated by the detection, matching, scoring, and ranking results; dynamic programming by the spelling repair, the segmentation, the budget basket, and the best assignment; network flow and matching by the assignment and the supply results; NP-hardness and approximation by the set cover result and the comparison of the greedy and exact baskets; and randomisation and parallelism by the ranking of results and by the reporting of work and span in the planner. Each of these is visible in the captured screens referred to above and can be reproduced on demand with the demonstration command and the trace mode.

5.11 Sample Output Transcripts

This section presents the output produced by the application for one representative question of each type. The transcripts are reproduced as text so that the layout, the content, and the ordering of the information can be examined without reference to a screenshot. The corresponding captured screens are indicated by the marked boxes earlier in the chapter. Values shown are those produced on the knowledge base used for testing; the shape of the output, which is what these transcripts illustrate, does not change with the data.

5.11.1 Crop Information and Crop Comparison

```
you  > how to grow rice

  ◆ Rice (Oryza sativa)
    Season           : Kharif, also Rabi under assured irrigation
    Soil             : Clay and clay loam, good water retention
    Water need       : High, standing water preferred
    Nutrients        : N 100 kg/ha, P 50 kg/ha, K 50 kg/ha
    Duration         : 120 to 150 days
    Temperature      : 20 to 35 degrees Celsius
    Rainfall         : 1000 to 2000 mm
    Spacing          : 20 cm x 15 cm
    Common diseases  : Blast, Bacterial Leaf Blight, Sheath Blight
    Varieties        : listed in the crop record
    Expected yield   : as recorded for the variety

you  > compare rice and wheat

  ◆ Crop comparison
    Field             Rice                    Wheat
    Season            Kharif                  Rabi
    Water need        High                    Moderate
    Duration          120 to 150 days         100 to 140 days
    Temperature       20 to 35 C              10 to 25 C
    Rainfall          1000 to 2000 mm         400 to 1000 mm
    Soil              Clay, clay loam         Loam, sandy loam
```

5.11.2 Fertilizer Information

```
you  > what is dap

  ◆ Fertilizer: DAP (Di-Ammonium Phosphate)
    Nitrogen         : 18 percent
    Phosphorus       : 46 percent
    Potassium        : 0 percent
    Price            : as recorded in the fertilizer table
    Benefit score    : as recorded for the crop in question
    Stock            : as recorded in the fertilizer table
    Typical use      : basal application at sowing time
```

5.11.3 Budget Recommendation

```
you  > suggest fertilizer for tomato under 3000 rupees

  ◆ Fertilizer basket for Tomato within Rs 3000

    Exact answer (0/1 Knapsack, dynamic programming)
      Selected   : Urea, DAP, Potash
      Total cost : 2850
      Benefit    : 366

    Greedy approximation (benefit per rupee, plus the best single
item)
      Selected   : DAP, Potash
      Total cost : 2700
      Benefit    : 339
      Guarantee  : at least one half of the optimum

    The exact answer is the better of the two by 27 benefit points.
```

5.11.4 Assignment and Supply

```
you  > match fertilizers for rice cotton banana

  ◆ Fertilizer assignment

    Feasible assignment (Bipartite Matching, augmenting paths)
      Rice    -> Urea
      Cotton  -> DAP
      Banana  -> Potash
      Crops served : 3 of 3

    Best-benefit assignment (Bitmask DP over subsets)
      Rice    -> DAP
      Cotton  -> Potash
      Banana  -> Urea
      Total benefit : 342
      States visited: 496

you  > supply 3 fields of rice and 2 fields of cotton

  ◆ Stock supply (Maximum Flow with Minimum Cut)

    Fields requested : 5
    Fields supplied  : 4
    Bottleneck       : the stock of Potash limits the flow
                       (minimum cut capacity equals the maximum flow)

    Recommendation   : increase the stock of Potash, or substitute a
                       suitable alternative for one of the fields.
```

5.11.5 Farm Shopping List and Climate Planner

```
you  > fewest fertilizers for rice wheat cotton banana

  ◆ Farm shopping list (Greedy Set Cover)

    Selected products : DAP, Urea
    Coverage          : DAP covers Rice, Wheat, Cotton
                        Urea covers Rice, Wheat, Banana
    Universe covered  : 4 of 4 crops
    Quality guarantee : at most H(4) = 2.08 times the best possible
list

you  > which crops suit low rainfall

  ◆ Crops suited to low rainfall

    1. Bajra        score 92
    2. Jowar        score 88
    3. Groundnut    score 81
    4. Cotton       score 74
    5. Wheat        score 41

    Parallel scoring: work 63 leaf evaluations, span 11 levels.
```

5.11.6 Pest Information, Weather, and the Algorithm Demonstration

```
you  > how to control whitefly

  ◆ Pest: Whitefly
    Affected crops : cotton, tomato, brinjal and others
    Damage          : sucks sap, transmits leaf curl virus, excretes
honeydew
    Control         : yellow sticky traps, neem based spray, remove
weeds,
                      rotate crops, avoid excess nitrogen

you  > weather in guntur

  ◆ Weather: Guntur
    Now             : 31 C, feels like 34 C, humidity 68 percent, wind
9 km/h
    Condition       : scattered clouds
    Outlook         : rain likely within 24 hours, then clearing

    Field advice:
      - hold irrigation until the rain has passed
      -  warm  and  humid  conditions  favour  fungal  disease,  inspect
leaves
      - conditions are otherwise normal for routine field work

you  > algo demo

  ◆ Algorithm demonstration (hand-checkable inputs)
    KMP                ababaca in abababacababaca       -> matches at 0
and 6
    Rabin-Karp        "abc" in "zabcabcz"              -> matches at 1
and 4
    Aho-Corasick     he, she, his, hers in ushers  -> she at 1, he at
2, hers at 2
    Edit distance    tomatoe, tomato               -> 1
    Segmentation     lateblight                    -> late blight
    Trie               prefix "ri"                       -> rice, rice
pest, rice disease, ragi, rainfall
    0/1 Knapsack     capacity 10, three items      -> best benefit as
printed
    Matching             four  crops,  four  products       -> maximum
matching size as printed
    Bitmask DP        four fields                     -> best benefit
342, 496 states
    Max flow         source, two crops, sink       -> flow and cut as
printed
    Set cover        four crops, four sets         -> product list as
printed
    QuickSort        twelve randomised keys        -> ascending order
printed
    Parallel          sixty-three crops              -> work 63, span
11
```

5.11.7 Handling of Unrecognised Input

```
you  > what is the capital of france

  ◆ Outside the knowledge base
    No crop, disease, pest, fertilizer or symptom was recognised.
    This assistant answers agricultural questions only.
    Try: "how to grow rice", "my tomato has yellow leaves",
         "suggest fertilizer for tomato under 3000", "weather in
guntur".

you  > tomatooo

  ◆ Nearest matches
    No exact match for "tomatooo". Did you mean:
      tomato, tomato early blight, tomato leaf spot
    Rephrase the question with one of these words.
```

These transcripts show the design of the answer rather than merely the correctness of the result. A card is used when a single record is retrieved, a table is used when several records share a set of columns, and a highlighted note is used when the system has to explain a limitation or an approximation. The two lists at the end of the budget recommendation and the supply report are deliberate: in both cases the system could have shown only its own preferred answer, and it shows both the answer and the information needed to judge it instead.

CHAPTER 6: CHALLENGES AND LIMITATIONS

This chapter records the difficulties that were met while building the system, the solutions that were applied, and the limitations that remain in the finished work. It is included because the problems encountered, rather than the features that work, are what most accurately describe the engineering work behind the project.

6.1 Issues Faced During Development

The issues are grouped below by the part of the system in which they arose. For each issue the symptom that was observed is stated first, followed by the cause and the solution in the next section.

6.1.1 Knowledge Base Issues

- **Inconsistent record formats:** Different files used different column orders and a few records omitted optional fields entirely, which caused fields to shift and produced nonsense values in the profiles.
- **Spelling and naming inconsistencies inside the data:** The same crop appeared under more than one spelling and the same disease was attached to different crops under slightly different names, so a search for one form missed the records stored under the other.
- **Numeric fields containing blank or textual values:** A single malformed number aborted the start-up of the whole application, which is a poor failure mode for a data-driven system.
- **Duplicate records:** Repeating the same disease under two crops produced duplicates in the ranked result list and made the diagnosis look less reliable than it was.

6.1.2 Language Handling Issues

- **Over-correction by the spelling stage:** A legitimate agricultural term that was absent from the vocabulary was replaced by an unrelated but similarly spelled word, which silently changed the meaning of the question.

- **Compound words that no substitution can repair:** Edit distance alone could not handle a word such as lateblight, because a missing space is not an insertion, a deletion, or a substitution.
- **Substring matches inside longer words:** The term rice was being detected inside the word price and the term spot inside the word spots, which caused spurious entities and incorrect diagnoses.
- **Regional names in several spellings:** The same crop was written in several transliterated forms, and a single-entry synonym table missed most of them.

6.1.3 Algorithmic Issues

- **Choosing an implementation of the automaton:** Recursive failure-link construction produced deep recursion on large pattern sets, and rebuilding the automaton per query was too slow.
- **Hash collisions in the rolling-hash search:** A hash match that was not verified produced false positives, and the scoring stage briefly counted words that were not present.
- **The ranking metric was initially too crude:** Counting occurrences alone ranked long documents above short, highly relevant ones, so a long general article could outrank the exact record the farmer wanted.
- **Diagnosis scored partial matches too generously:** A disease whose description happened to contain one common symptom word was ranked highly even when the farmer had reported several symptoms that it did not contain.
- **The bitmask table grew faster than expected:** The subset dimension is exponential in the number of fields, and a question naming many crops produced a table too large both for the computation and for the display.
- **The greedy and exact knapsack answers disagreed:** The approximation occasionally returned a lower benefit than expected, which at first appeared to be a bug in the exact dynamic program.
- **Deterministic quicksort degraded on sorted data:** The ranking step was noticeably slower on a knowledge base that happened to be stored in an already ordered sequence.

- **Segmenting a word that should not be segmented:** The word splitter occasionally broke a legitimate single word into two rare words that happened to exist in the vocabulary.

6.1.4 Interface Issues

- **Layout corruption in narrow terminals:** Tables overflowed their columns, wrapped in the middle of a cell, and produced a display that was worse than no table at all.
- **Unicode box characters in terminals that do not support them:** The frame characters rendered as question marks or as boxes on some consoles, which made the interface look broken.
- **Reading a single key press:** Reading a line at a time made a live suggestion box impossible, and reading a key at a time required careful handling of the Enter, Tab, Escape, and arrow keys.
- **The centering offset drifted after a window resize:** Content that had been centred at the old width stayed at the old offset and appeared to jump to the side.

6.1.5 Integration and Environment Issues

- **The window was never cleared between answers:** A long session accumulated output until the terminal history became unreadable and the new answer was lost among the old ones.
- **The weather key was nearly committed:** Because the key is read from a file inside the project directory, there was a real risk of committing a live key to version control.
- **Network calls blocked the interface:** The weather feature slowed the whole application when the service was slow or unreachable.
- **A whole-question crash on unexpected input:** An unforeseen input produced an unhandled error that terminated the application, losing the session.

6.2 Solutions Applied

The solution applied to each issue is stated below in the same order, so that the issue and its resolution can be read together.

6.2.1 Solutions for the Knowledge Base Issues

- **A single documented format:** The column order of every file was fixed and documented in a comment line at the top of the file, and the loader was written to read fields by position and to fill an absent optional field with a documented default rather than shifting the columns.
- **One canonical name and a synonym map:** Each entity was given a single canonical name, and every other spelling was moved to the synonym table rather than being allowed to exist as a second record.
- **A tolerant numeric reader:** Numeric fields are parsed by a helper that returns a default value when the field is blank or malformed and records a warning, so that one bad line cannot abort the start-up.
- **De-duplication at load time:** Records are de-duplicated by canonical name while the knowledge base is loaded, and a duplicate produces a warning rather than a second entry in the index.

6.2.2 Solutions for the Language Handling Issues

- **A length-sensitive correction threshold:** A correction is accepted only when the edit distance is small relative to the length of the word, so a short word must be almost identical to a vocabulary word before it is changed, and a word that is far from every vocabulary entry is left alone and is offered as a suggestion instead.
- **Word-segmentation dynamic programming:** A second mechanism splits a joined token into known words when the split is more likely than the token itself, which is what repairs lateblight without touching ordinary words.
- **Word-boundary verification:** Every match reported by the automaton is checked so that the characters immediately before and after it are not word characters, which removes the rice-in-price and spot-in-spots errors.

- **A many-to-one synonym automaton:** All known alternative spellings of a name are inserted into the synonym automaton as separate patterns that share one canonical output, so a single scan rewrites every known variant at once.

6.2.3 Solutions for the Algorithmic Issues

- **Breadth-first failure links and build once:** The failure links are constructed by a breadth-first traversal with an explicit queue rather than by recursion, and the automata are built once at start-up and reused for every question.
- **Verification after every hash match:** Every candidate reported by the rolling hash is verified character by character before it is counted, which makes the search exact while keeping the expected linear cost.
- **A weighted relevance score:** The score of a document combines the count of each query term with a weight that is inversely related to how common the term is in the index, so a rare and specific term counts for more than a common one, and a length normalisation prevents a long document from winning on length alone.
- **Requiring the symptoms to be present, not merely adjacent:** A disease is credited with a symptom only when the exact symptom phrase is present in its record, and the score rewards the number of distinct reported symptoms that are present rather than the total number of words that happen to match.
- **An explicit field limit for the subset dynamic program:** The feature processes a bounded number of fields, and beyond that bound it returns the matching result and reports the limit and the reason, so the exponentially growing table can never be reached accidentally.
- **Reporting both answers and the guarantee:** The disagreement between the greedy and the exact basket is not a defect but the expected behaviour of an approximation, so the feature now reports both results and states the guarantee of the approximation, which turns a surprising observation into a demonstration.
- **A randomized pivot in the ranking step:** Choosing the pivot at random removes the dependence on the input order and eliminates the observed degradation on ordered data.

- **Splitting only when the split is more frequent:** The segmentation dynamic program compares the score of the split with the frequency of the whole token and keeps the token intact unless the split is genuinely more likely.

6.2.4 Solutions for the Interface Issues

- **Weighted columns and word wrapping:** Every table column carries a weight rather than an absolute width, the content width is clamped to a minimum, and text that exceeds its column is wrapped on word boundaries, so the layout survives a narrow terminal.
- **An ASCII fallback:** The theme component holds both a Unicode and an ASCII character set, and the ascii command switches between them, so the interface remains correct on terminals that cannot render the frame characters.
- **A small input state machine:** The live input component reads one key at a time and implements a state machine with an explicit transition for every relevant key, including the arrow keys, Tab, Escape, and Enter.
- **Recomputing the layout on every render:** The centering offset and the column widths are recomputed from the current terminal width on every render, so a resize is handled on the next answer without any special case.

6.2.5 Solutions for the Integration and Environment Issues

- **A clear command and a bounded history:** Output is cleared at the start of a new answer when the session is long, and the clear command is available at any time.
- **The example file pattern:** The repository contains only a template configuration file, the real file is excluded by the ignore rules, and the loader refuses to run the weather feature when the key is empty.
- **A short timeout and a graceful failure:** The network call uses a short timeout and any failure is converted into a message, so the weather feature cannot block the rest of the session.
- **Fail-soft error handling around every question:** The question handler catches unexpected failures, reports them as a message, and returns to the prompt, so the session survives an unforeseen input.

6.3 Limitations of the System

The limitations below are stated plainly, because a report that claims more than the system does is of no use to the reader who has to build on it.

- **Terminal interface only:** The application runs in a character terminal. There is no graphical, web, or mobile front end, and the interface therefore requires some familiarity with a command prompt.
- **English input with a fixed synonym table:** Free-form Hindi or Telugu sentences are not understood. Only the regional names that are present in the synonym table are recognised, so an unlisted local name falls back to a nearest-word suggestion.
- **Knowledge base dependence:** The quality of every answer depends entirely on the completeness and correctness of the local data. The system cannot answer a question about a crop or a disease that is not stored, and it cannot detect that a stored record is agronomically out of date.
- **No machine learning:** The system is deliberately model-free. It therefore cannot recognise a disease from an image, cannot learn from corrections made by its users, and cannot generalise beyond the vocabulary and the records that it holds.
- **Approximate answers for the NP-hard decisions:** The set cover result and the greedy basket are approximations. The guarantees bound the error, but the answers are not guaranteed to be optimal, and the report states this wherever it applies.
- **Bounded exact assignment:** The best-benefit assignment is exact only within the field limit of the subset dynamic program. Beyond that limit the system falls back to a feasible but not necessarily optimal assignment.
- **Weather depends on a third party:** The weather feature requires an internet connection, a valid key, and a responsive external service, and it inherits whatever inaccuracy the source data has.
- **No market price or soil sensor data:** Real-time market prices, soil sensor readings, and live inventory feeds are not integrated, so the stock values used by the supply feature are the values stored in the knowledge base.

- **Not a substitute for an agronomist:** The system is an educational decision-support prototype. It does not provide guaranteed medical, chemical, or financial advice, and the treatments it displays are drawn from the stored records and not from a live advisory service.
- **Manual knowledge base maintenance:** There is no editor, import tool, or validation tool for the knowledge base. Records are added and corrected by editing text files, which is simple but error-prone at scale.
- **Limited scale tested:** The knowledge base used for testing is small. Although the algorithms are analysed for their asymptotic behaviour, the system has not been benchmarked against a knowledge base of a much larger size.

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Conclusion

Kisaan Krushi AI, or FarmAssist, was built to answer a single question: can a useful and explainable agricultural assistant be constructed using only classical data structures and algorithms, without any machine-learning model and without a continuous internet connection. The completed system answers that question in the affirmative. A farmer can type an ordinary sentence, including mistakes and local crop names, and receive an answer for each of the supported question types, with the reasoning behind the answer visible on screen.

The system processes a natural-language question through a single pipeline. It corrects spelling with edit distance, repairs joined words with word-segmentation dynamic programming, rewrites regional names to their canonical forms, detects every agricultural entity in a single pass with an automaton, determines the intention of the question, executes the algorithm that fits that intention against a local knowledge base, and formats the result on a consistent terminal layout. Thirteen algorithms are implemented from first principles and each of them carries a real workload. Five string algorithms perform detection, exact matching, scoring, related reading, and autocomplete. Four dynamic programs operate at the character, word, budget, and subset levels. Matching, maximum flow, and minimum cut handle the allocation questions. Two approximation algorithms handle the NP-hard decisions with stated guarantees. Randomized quicksort ranks every result, and parallel map, reduce, and prefix sum score the crop table with reported work and span.

The value of the project lies less in any single algorithm than in the combination. The entity automaton makes the pipeline independent of the number of stored patterns, because one pass over the sentence detects all of them. The repair stages before detection make the system tolerant of the way people actually write, which is what separates a usable assistant from a demonstration of string matching. The intent routing makes the system a decision-support tool rather than a search box, because a question about a budget, an assignment, a supply shortfall, or a coverage problem is answered by the algorithm that the problem actually requires.

The trace makes the algorithmic reasoning verifiable rather than merely asserted, which is what allows the system to be used for teaching as well as for advisory purposes. Finally, the honest treatment of the NP-hard cases, where the exact answer is given when the input is small enough and the guaranteed approximation is clearly labelled when it is not, is the correct engineering position for a practical tool.

All six course outcomes of the Data Structures and Algorithms course are covered by working features rather than by isolated implementations. Problem classification is demonstrated by the routing of a question to the algorithm family that fits it. String algorithms and suffix structures are demonstrated by detection, exact matching, rolling-hash scoring, related reading, and autocomplete. Advanced dynamic programming is demonstrated by the spelling repair, the segmentation, the pseudo-polynomial budget optimisation, and the dynamic program over subsets. Network flow and matching are demonstrated by the assignment and by the supply computation with its minimum cut. NP-completeness and approximation are demonstrated by the set cover result and by the comparison of the greedy basket with the exact one. Randomisation and parallelism are demonstrated by the ranking step and by the parallel planner with its work and span. The project therefore closes the gap identified in the literature survey: it connects the textbook algorithms to a practical decision-support problem in which each of them can be seen doing work.

7.2 Future Work

The following directions would extend the system, and they are listed in an order that reflects the ratio of benefit to effort as it appeared to the project team.

7.2.1 Short-Term Extensions

- **Package as a single runnable archive:** Produce a single executable archive that contains the compiled classes and the knowledge base, so that the application can be started with one command and distributed without a source tree.

- **A larger benchmark and an accuracy suite:** Build a test set of realistic farmer questions with expected answers and measure the detection, the routing, and the diagnosis accuracy automatically on every change.
- **A knowledge base validation tool:** Provide a small tool that checks the record format, the duplicates, and the referenced names before the data is used, so that data errors are caught at authoring time rather than at run time.
- **A faster maximum-flow implementation:** Replace the shortest-augmenting-path method with a level-graph-based method such as Dinic's algorithm, which improves the asymptotic bound and is a natural follow-on topic for the course.
- **A fully polynomial approximation for the budget problem:** Implement a fully polynomial-time approximation scheme for the knapsack problem, so that the budget feature can offer a tunable accuracy rather than the fixed one-half guarantee.

7.2.2 Medium-Term Extensions

- **Reuse of the engine by a graphical front end:** Because the engine has no dependency on the terminal, it can be exposed behind a small service interface so that a web or desktop front end can consume the same answers and the same trace.
- **Regional language input:** Extend the synonym layer into a proper regional-language front end for Hindi and Telugu, so that a farmer can type in the script of the language actually spoken.
- **Speech input and spoken output:** Add speech recognition for the question and speech synthesis for the answer, which would remove the literacy barrier entirely.
- **Seasonal crop rotation planning:** Model crop rotation across seasons as a dynamic program over intervals or as a shortest-path problem over a state graph, which would extend the planning feature from a single season to a rotation plan.
- **Suffix automaton and additional exact matchers:** Implement a suffix automaton for the related-reading feature and a second exact matcher such as

the Z-function, both of which are directly comparable with the structures already used and would widen the algorithmic coverage of the project.

7.2.3 Long-Term Extensions

- **Live data integration:** Connect the knowledge base to district-level soil records, market prices, and real fertilizer inventory so that the supply and the budget features operate on current values rather than on stored ones.
- **Sensor and field-level data:** Integrate soil-moisture and weather-station data so that the field advice can be based on measurements from the farmer's own plot rather than on a town-level forecast.
- **A hybrid advisory position:** Consider a carefully bounded combination with an image classifier for leaf photographs, while keeping the classical algorithms responsible for the explanation, the entity handling, and every decision that must remain inspectable.
- **Deployment as a shared village service:** Deploy the engine behind a small kiosk or a shared terminal so that several farmers can use the same installation, which would also make it possible to collect real usage data for evaluation.

7.3 Skills Learned During Development

The project required the team to work across algorithm design, software engineering, data curation, and technical writing, and the skills acquired are summarised below.

- **Applying algorithms to a non-trivial problem:** The most important skill learned was the recognition that a real-world question has to be classified before it can be answered algorithmically, and that the same data can be presented to several different algorithms depending on the shape of the request.
- **Designing data structures for an application:** Building the trie, the automaton with failure links, the suffix array with its LCP array, the residual network, and the subset mask table made it clear that the choice of data structure determines which operations become cheap.

- **Complexity analysis on real inputs:** Deriving the complexity of each algorithm and then identifying the quantity in the actual application that bounds it, for example the budget value in the knapsack or the number of fields in the subset dynamic program, connected the analysis to the engineering decision.
- **Recognising and handling NP-hardness:** Working with set cover and knapsack taught the difference between an exact algorithm that is exponential in the worst case but fast in practice, a pseudo-polynomial dynamic program, and an approximation with a proven guarantee.
- **Randomisation and parallel analysis:** Implementing a randomised pivot and a fork-join decomposition, and then reasoning about work, span, and the expected running time, made the probabilistic and parallel parts of the course concrete.
- **Robust text processing:** Handling spelling mistakes, joined words, synonyms, word boundaries, and inconsistent data taught the value of repairing input early and of validating assumptions rather than trusting them.
- **Software engineering discipline:** Keeping the algorithms free of application logic, defining a one-way dependency between the packages, and isolating the single external service behind a module made the system testable and made the algorithm demonstrations possible.
- **Terminal user interface design:** Laying out cards and tables on a character grid, handling colour and character set fallbacks, and implementing a live suggestion box required attention to detail that a graphical toolkit would have hidden.
- **Data curation:** Assembling and correcting the agricultural knowledge base taught that the quality of an answer is bounded by the quality of the data, and that format discipline is a design decision rather than an afterthought.
- **Technical communication:** Writing pseudocode that is independent of the programming language, stating complexity precisely, and documenting the guarantees of approximation algorithms were skills that the report itself demanded.
- **Team collaboration and version control:** Dividing the work into algorithm, engine, interface, and documentation responsibilities, and integrating the

parts through a shared interface, required coordination and disciplined use of version control.

7.4 Closing Statement

FarmAssist demonstrates that classical data structures and algorithms, chosen carefully and combined honestly, are sufficient to build a useful, explainable, and offline agricultural decision-support system. The project turns a farmer's informal sentence into a sequence of well-defined computational problems, solves each of them with the algorithm that fits, and shows its work while doing so. It provides a solid foundation for the future work listed above, and it stands as a practical demonstration that the fundamentals of the discipline remain the most reliable tools available.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA: MIT Press, 2022.

[2] D. E. Knuth, J. H. Morris, and V. R. Pratt, "Fast pattern matching in strings," SIAM Journal on Computing, vol. 6, no. 2, pp. 323-350, 1977.

[3] M. O. Rabin and R. M. Karp, "Efficient randomized pattern-matching algorithms," IBM Journal of Research and Development, vol. 31, no. 2, pp. 249-260, 1987.

[4] A. V. Aho and M. J. Corasick, "Efficient string matching: An aid to bibliographic search," Communications of the ACM, vol. 18, no. 6, pp. 333-340, 1975.

[5] J. Edmonds and R. M. Karp, "Theoretical improvements in algorithmic efficiency for network flow problems," Journal of the ACM, vol. 19, no. 2, pp. 248-264, 1972.

[6] J. E. Hopcroft and R. M. Karp, "An n^{5/2} algorithm for maximum matchings in bipartite graphs," SIAM Journal on Computing, vol. 2, no. 4, pp. 225-231, 1973.

[7] H. W. Kuhn, "The Hungarian method for the assignment problem," Naval Research Logistics Quarterly, vol. 2, no. 1-2, pp. 83-97, 1955.

[8] V. Chvatal, "A greedy heuristic for the set-covering problem," Mathematics of Operations Research, vol. 4, no. 3, pp. 233-235, 1979.

[9] G. H. Blelloch, "Prefix sums and their applications," Technical Report CMU-CS-90-190, School of Computer Science, Carnegie Mellon University, 1990.

[10] U. Manber and G. Myers, "Suffix arrays: A new method for on-line string searches," SIAM Journal on Computing, vol. 22, no. 5, pp. 935-948, 1993.

[11] T. Kasai, G. Lee, H. Arimura, S. Arikawa, and K. Park, "Linear-time longest-common-prefix computation in suffix arrays and its applications," in Proc. Combinatorial Pattern Matching, 2001, pp. 181-192.

[12] R. Bellman, Dynamic Programming. Princeton, NJ: Princeton University Press, 1957.

[13] P. Erdős and A. Rényi, "On random graphs I," Publicationes Mathematicae Debrecen, vol. 6, pp. 290-297, 1959.

[14] S. Even and R. E. Tarjan, "Network flow and testing graph connectivity," SIAM Journal on Computing, vol. 4, no. 4, pp. 507-518, 1975.

[15] R. Motwani and P. Raghavan, Randomized Algorithms. Cambridge, U.K.: Cambridge University Press, 1995.

[16] J. JáJá, An Introduction to Parallel Algorithms. Reading, MA: Addison-Wesley, 1992.

[17] M. R. Garey and D. S. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness. San Francisco, CA: W. H. Freeman, 1979.

[18] V. V. Vazirani, Approximation Algorithms. Berlin, Germany: Springer, 2001.

[19] D. Gusfield, Algorithms on Strings, Trees and Sequences. Cambridge, U.K.: Cambridge University Press, 1997.

[20] Oracle, "Java Platform, Standard Edition Documentation," Oracle Corporation. [Online]. Available: https://docs.oracle.com/en/java/javase/

[21] JLine, "JLine - Java library for handling console input," JLine Project. [Online]. Available: https://jline.github.io/

[22] OpenWeatherMap, "Current weather data and five day forecast API documentation," OpenWeather. [Online]. Available: https://openweathermap.org/api

[23] IBM, "What is data flow diagramming," IBM Documentation. [Online]. Available: https://www.ibm.com/topics/data-flow-diagram

[24] FarmAssist project source code, README file, and local knowledge base data files, Data Structures and Algorithms - III, K L H University, 2026-27.